

Manuel Babet

Table des matières

Babet — Manuel utilisateur	11
Pour démarrer	12
Trouver une fonctionnalité	12
Modules de référence	13
Organisation d’une page de module	14
Génération du PDF	15
Méthodologie de vérification	15
Pour démarrer	15
Téléchargement	15
Compilation depuis les sources	15
Trois manières de lancer un script Lua	16
1. Mode dossier	16
2. Mode embarqué (binaire autonome)	16
3. Embarqué via PATH (dossier + auto-détection)	16
Invocation en ligne de commande	16
Premier script	17
Pour aller plus loin	17
Modèle de sécurité	17
Ce contre quoi le durcissement <i>protège</i>	17
Ce contre quoi il <i>ne protège pas</i>	18
Pattern recommandé : moindre privilège	19
Archive — opérations ZIP et TAR sécurisées avec gzip, xz, bzip2 ou zstd	19
Périmètre	19
Table des matières	20
babet.archive.create	20
Format de sortie	20
Options d’inspection/extraction et limites anti-bombe	23
babet.archive.list	24
babet.archive.extract	25
babet.archive.extractFile	26
Règles de chemins	27
Protection de la destination	28
Publication atomique et nettoyage	28
Permissions	29
Types d’entrées pris en charge	29
Erreurs et limites connues	30
argparse — arguments de ligne de commande	30
Table des matières du module	30
Chargement et métadonnées	31
API	31
Déclarer des flags et des options	31

Déclarer des positionnels	32
Table opts	32
Champs communs	32
Champs réservés aux valeurs	33
Destination	33
Source des arguments	33
Table globale arg	33
Array explicite	33
Formes reconnues	33
Valeurs par défaut, choix et conversion	34
Résultats de parse	35
Succès normal	35
Aide intégrée	35
Erreur utilisateur	35
Erreur de programmation	35
Exemple complet	36
Texte produit par get_usage	36
Limites de la version actuelle	36
EXEC - programmes externes, capture et streaming	36
Table des matières du module	37
Conventions essentielles	38
Aucun shell implicite	38
Une sortie non nulle n'est pas une erreur de lancement	38
Les chaînes de données sont binaires	38
Vue d'ensemble de l'API	38
Signature et arguments	39
cmd et recherche du programme	39
args et absence de shell	40
Utiliser explicitement un shell	40
Options	40
opts.cwd	40
opts.env	41
opts.stdin	41
opts.timeout	42
opts.max_output	42
Table de résultat	42
Code de sortie normal	43
Fin par signal	43
Timeout	43
Troncature	43
Exemples complets	44
Capture simple	44
Arguments contenant des espaces	44
Traiter un code non nul	44
Capturer stderr	44
Envoyer du texte ou du binaire sur stdin	45
Modifier l'environnement	45
Changer de répertoire	45
Borner le temps	45
Borner la sortie	46
Créer un helper applicatif	46
Processus en streaming avec <code>babet.spawn</code>	46
Signature et options	47
Méthodes du processus	47
Lire stdout et stderr	48
Écrire sur stdin	48

Attendre, terminer et fermer	49
Exemple de boucle de streaming	49
Limites et prévention des blocages	50
Contrat d'erreur	50
Erreurs Lua levées	50
Erreurs renvoyées (nil, err)	50
Situations renvoyant une table	50
Processus, signaux et sécurité	51
Groupe de processus	51
Interaction avec babet.signal	51
Mémoire	51
Décisions et limites	51
Pipelines de processus	51
Syntaxe commune des commandes	52
babet.pipeline() — exécution complète	52
Signature	52
Options	53
Résultat	53
Timeout, gros volumes et troncature	54
babet.spawnPipeline() — streaming	54
Signature	54
Méthodes de lecture	55
Écriture et fermeture de stdin	55
État et identifiants	55
Attente et résultat final	56
Arrêt explicite	56
close(), GC et Lua <close>	56
Erreurs de lancement et réutilisation	57
FS — fichiers, dossiers, chemins, recherche, copie, permissions et sommes de contrôle	57
Table des matières du module	57
Conventions générales	58
Chemins	58
Liens symboliques	58
Ordre des résultats	59
Valeurs de retour	59
Vue d'ensemble de l'API	59
Vérifier et inspecter	59
Manipuler des chemins	59
Agir sur le système de fichiers	59
Lister et rechercher	60
Copier et déplacer	60
Modifier les attributs	60
Existence, type et taille	60
babet.fileExists(path)	60
babet.isFile(path) et babet.isfile(path)	61
babet.isDir(path) et babet.isdir(path)	61
babet.fileSize(path)	61
Répertoire courant	61
babet.currentDir()	61
babet.chdir(path)	61
Manipulation lexicale des chemins	62
babet.getBasename(path)	62
babet.getFilename(path)	62

babet.getExtension(path)	62
babet.getPath(path)	62
babet.joinPath(...)	62
Créer, supprimer, renommer et lier	63
babet.touch(path)	63
babet.mkdir(path)	63
babet.remove(path)	64
babet.rmdir(path)	64
babet.rmdirAll(path)	64
babet.rename(source, destination)	64
babet.link(target, linkpath)	65
Lister, itérer et rechercher	65
babet.listFiles(path, recursive?)	65
babet.createFileIterator(path, recursive?)	66
babet.find(path, opts?)	67
Copier et déplacer	69
babet.copy(source, destination)	69
babet.copyTree(source, destination, continue_on_error?)	70
babet.moveTree(source, destination)	71
Permissions et attributs Unix	71
babet.getMode(path) et babet.setMode(path, mode)	71
babet.getAttributes(path)	72
babet.setAttributes(path, uid, gid, mode?)	72
babet.symlinkAttr(path, uid, gid) et babet.symlinkattr(...)	73
Sommes de contrôle de fichiers	73
CRC-32 en mémoire	74
babet.crc32(data)	74
Contrat d'erreur	74
Erreurs Lua levées	74
Erreurs renvoyées	74
Opérations partielles	74
Décisions et limites	74

HTTP — requêtes web synchrones, headers, corps, TLS et limites **75**

Table des matières du module	75
Conventions essentielles	76
Client synchrone	76
Statut HTTP et erreur de transport	76
Corps binaires	76
Options inconnues	76
Vue d'ensemble de l'API	76
Table d'options	77
babet.http.request(opts)	77
babet.http.get(url, opts?)	77
babet.http.download(url, destination, opts?)	78
babet.http.post et ses formes d'appel	79
Sans corps	79
Corps positionnel	79
Corps dans opts	79
Priorité	79
URL, fragments et query	79
URL absolue	79
Ajouter une query	79
Méthodes HTTP	80
Headers de requête	80
Noms	81
Valeurs	81

Doublons	81
Casse	81
Corps de requête et Content-Type	81
Corps absent et corps vide	81
JSON	81
Formulaire URL-encoded	82
Multipart	82
opts.timeout	82
Limite DNS	82
Toujours en fournir un	82
HTTPS, vérification et CA	82
Vérification par défaut	82
CA spécifique	83
Désactiver la vérification	83
follow_redirects	83
max_body_size	83
max_file_size et sécurité du fichier	84
Tables de réponse	84
Réponse en mémoire	84
status	84
body	84
headers	84
headers_multi	85
Résultat d'un téléchargement	85
Exemples complets	85
GET JSON	85
POST JSON authentifié	85
PUT binaire	86
Télécharger un artefact de release	86
Gérer 404 séparément des erreurs réseau	86
Requêtes parallèles avec WORKERS	87
Contrat d'erreur	87
Erreurs Lua levées	87
Erreurs (nil, err)	87
Pas une erreur Babet	87
Sécurité et limites	88
babet.inotify — surveillance du système de fichiers	88
Table des matières du module	88
API	88
Créer un watcher	89
Ajouter une surveillance	89
Option onlydir	89
Lire les événements	89
Débordement de la file	90
Interruption par signal	90
Retirer et fermer	90
Exemple	91
Contrat d'erreur	91
Limites	91
babet.json — encodage et décodage JSON	91
Table des matières du module	92
API	92
Correspondance des types	92
Lua vers JSON	92
JSON vers Lua	93

null	93
Objet vide, tableau vide et as_array	93
Options de encode	94
Chaînes, UTF-8 et octets NUL	94
Nombres	94
Ordre des objets et clés dupliquées	94
Contrat d'erreur	95
Limites	95
logging — journalisation à niveaux	95
Table des matières du module	95
Émettre un message	96
Garantie de non-propagation	96
Format de sortie	96
Seuil	96
Destination	97
Couleurs ANSI	97
État du module	98
Contrat d'erreur	98
Limites	98
SIGNAL — arrêt propre, rechargement et signaux POSIX	98
Table des matières du module	99
Conventions essentielles	99
Les succès renvoient une seule valeur	99
Les noms sont stricts	99
Un callback ne reçoit aucun argument	99
Vue d'ensemble de l'API	100
Signaux pris en charge	100
handle(name, fn) — installer ou remplacer	100
handle(name, nil) — retirer un callback	101
ignore(name) — ignorer	101
default(name) — restaurer le comportement système	101
Exécution des callbacks	102
Thread principal et contexte sûr	102
Ordre de dispatch	102
Coalescence des occurrences	102
Erreurs du callback	103
Interaction avec les appels bloquants	103
Interaction avec debug.sethook	103
Exemples complets	104
Arrêt propre sur TERM et INT	104
Recharger une configuration avec HUP	104
Interrompre une attente socket	105
Ignorer temporairement PIPE	105
Observer les erreurs du callback	105
Contrat d'erreur	106
Erreurs Lua levées	106
Erreur système renvoyée	106
Workers, concurrence et limites	106
Fonctionnalités absentes	107
SOCKET — client et serveur TCP, flux binaires, lignes et timeouts	107
Table des matières du module	107
Conventions essentielles	108
TCP synchrone et bloquant	108
Chaînes binaires	108

Timeouts	108
Valeurs de retour	108
Vue d'ensemble de l'API	109
Constructeurs	109
Méthodes communes	109
Méthode d'un socket d'écoute	109
babot.socket.connect(host, port, timeout?)	109
Résolution et adresses multiples	110
Timeout de connexion non conservé	110
Erreurs typiques	110
babot.socket.listen(host, port, backlog?)	110
Écouter uniquement en loopback	110
Écouter sur toutes les interfaces	110
Demander un port libre au noyau	110
Backlog	111
server:accept(timeout?)	111
sock:set_timeout(seconds)	111
sock:send(data)	111
sock:recv(count, timeout?)	112
Lire exactement N octets	112
EOF et timeout	112
sock:recv_line(timeout?)	113
Limite de ligne	113
EOF avant LF	113
Timeout avant LF	113
sock:recv_all(timeout?, max_bytes?)	113
Limite mémoire	113
Mélanger recv, recv_line et recv_all	114
sock:peer() et sock:sockname()	114
sock:close() et nettoyage automatique	114
Signaux et interruptions	115
Exemples complets	115
Petit serveur ligne par ligne	115
Client avec longueur préfixée	116
Lire une réponse délimitée par EOF	116
Contrat d'erreur	116
Erreurs Lua levées	116
Erreurs renvoyées	116
Sécurité, performances et limites	117
babot.sqlite — base de données SQL embarquée	117
Table des matières du module	117
API	117
Ouverture et fermeture	117
Exécution directe	118
Statements préparés réutilisables	119
Transactions assistées	120
Paramètres SQL	121
BLOB explicites	121
Texte SQL et octets NUL	121
Lecture des lignes et mapping de types	122
Cycle de vie	122
Contrat d'erreur	122
Exemples complets	123
Inserts préparés et BLOB	123
Requête réutilisable et transaction	123
Fonctions non exposées	124

babet strings — découpage de chaînes	124
Table des matières du module	124
API	124
Mode séparateur	124
Limiter le nombre de coupes	125
Mode octets	125
Chaînes binaires et octets NUL	126
Chaîne vide	126
Contrat d'erreur	126
Limites et choix de conception	126
SYS - processus, machine, environnement, exécutables, version et mémoire Lua	127
Table des matières du module	127
Conventions générales	127
Fonctions plates	127
Chaînes strictes	128
État global du processus	128
Vue d'ensemble de l'API	128
Constantes de version	128
Identifier le processus et la machine	129
babet.pid()	129
babet.hostname()	129
babet.uname()	130
Rechercher un exécutable	130
Recherche dans PATH	130
Chemin direct	131
Programme absent ou non exécutable	131
Variables d'environnement	131
babet.env(name)	131
babet.setenv(name, value)	132
Valeur vide et variable absente	132
Interaction avec les workers	133
Mémoire de la VM Lua	133
babet.getMemoryUsage()	133
babet.getDetailedMemoryUsage()	133
Ce qui est mesuré et ce qui ne l'est pas	134
Contrat d'erreur	134
Décisions et limites	135
babet tables — fusion et copie de tables Lua	135
Table des matières du module	135
API	135
babet.mergeTables(t1, t2, ...)	135
Clés entières positives : concaténation	136
Toutes les autres clés : dernier écrivain gagnant	136
Fusion superficielle	136
babet.deepCopyTable(t)	136
Valeurs et graphe partagé	137
Les clés ne sont pas copiées	137
Métatables et accès brut	137
Profondeur maximale	137
Contrat d'erreur	137
Limites et choix de conception	138
Hors v1	138
babet time - horloges, sommeil, dates et durées	138
Table des matières du module	138

API	138
monotonic() et now()	139
sleep(amount [, unit])	139
babet.time.iso([ts])	140
babet.time.parse_iso(text)	140
babet.time.parse_duration(text)	141
babet.time.format_duration(seconds)	141
Contrat d'erreur récapitulatif	142
Limites et choix de conception	142
TLS — connexion chiffrée directe, STARTTLS, certificats et SNI	142
Table des matières du module	142
Conventions essentielles	143
Vérification activée par défaut	143
Même API de flux après le handshake	143
SNI n'est pas la vérification	143
Vue d'ensemble de l'API	143
Options TLS détaillées	144
verify	144
hostname	144
ca_cert et ca_path	144
min_version	145
timeout	145
babet.socket.connect_tls(host, port, opts?)	145
Hostname par défaut	145
Deadline globale	145
Timeout d'I/O après succès	146
sock:starttls(opts?)	146
Préconditions	146
Hostname obligatoire avec vérification	146
Consommer le plaintext avant l'upgrade	146
Déroulement applicatif	146
Vérification, hostname et SNI	147
Autorités de certification	147
CA supplémentaire, pas certificat serveur piné	147
Versions TLS	148
Timeouts	148
connect_tls	148
starttls	148
I/O après le handshake	148
État du socket après une erreur	148
connect_tls	148
starttls avant le handshake	148
starttls après le début du handshake	149
Exemples complets	149
Client HTTPS minimal sur socket brut	149
Connexion par IP avec nom de certificat	149
SMTP STARTTLS simplifié	149
Test local auto-signé	150
Contrat d'erreur	150
Erreurs Lua levées	150
Erreurs (nil, err)	150
Sécurité et limites	150
babet.toml - décodage TOML	151
Table des matières du module	151
API	151

Correspondance des types	152
Entiers et flottants	152
Chaînes, UTF-8 et clés	152
Arrays et tables	153
Ambiguïté des conteneurs vides	153
Dates et heures	153
Ordre et informations perdues	154
Contrat d'erreur	154
Limites	154
Non exposé en v1	154
USER - rechercher des comptes système par nom ou UID	155
Table des matières du module	155
Conventions générales	155
Sous-table dédiée	155
Nom ou UID, sans conversion implicite	155
Champs toujours présents	156
Vue d'ensemble de l'API	156
Table utilisateur renvoyée	156
Rechercher un compte avec get	157
Recherche par nom	157
Recherche par UID	157
Compte absent	157
Tester l'existence avec exists	158
Par nom	158
Par UID	158
Quand préférer get	158
Validation des arguments	159
NSS, workers et sécurité	159
Sources de comptes	159
Appels depuis des workers	160
Attention aux décisions de sécurité	160
Contrat d'erreur	160
Décisions et limites	160
WORKERS — threads OS, états Lua isolés et files de messages	161
Table des matières du module	161
Conventions essentielles	162
Chaque worker possède un état Lua distinct	162
Les résultats utilisent une convention proche de pcall	162
Le résultat final est consommable une seule fois	162
Vue d'ensemble de l'API	162
Côté parent	162
Côté worker	163
spawn(code, args?, opts?)	163
code	163
args et worker.args	164
Capacités des queues	164
Moment où setenv et chdir sont verrouillés	165
Valeurs transférables	165
Scalaires et chaînes	165
Tables listes et objets	165
Valeurs refusées	166
Copies, identité et profondeur	166
Résultat final du worker	166
join() — attendre et consommer	167
poll() — tester et consommer	167

Ne pas combiner <code>poll()</code> terminé puis <code>join()</code>	168
Messages parent vers worker	168
Messages worker vers parent	168
API worker. <code>send</code> et worker. <code>recv</code>	169
Timeouts et raisons d'échec	169
<code>close()</code> et cycle de vie des queues	170
Chargement des modules et environnement du worker	170
Exemples complets	171
Calcul simple avec <code>join</code>	171
Arguments structurés	171
Suivi non bloquant avec <code>poll</code>	171
Worker persistant requête/réponse	172
Envoyer et recevoir <code>nil</code>	172
Fermeture puis drainage de l'outbox	173
Contre-pression et timeout	173
Annulation coopérative	174
Pool borné corrigé	174
Interblocages à éviter	175
Worker en attente de l'inbox, parent dans <code>join()</code>	175
Worker bloqué sur une outbox pleine	175
Oublier que <code>poll()</code> consomme	176
GC potentiellement bloquant	176
Contrat d'erreur	176
Erreurs Lua levées par mauvais usage	176
Échecs renvoyés par <code>spawn</code>	176
Erreurs finales du worker	176
Sérialisation des messages	177
Garbage collector et destruction	177
Signaux, environnement et concurrence	177
Signaux	177
Environnement et <code>cwd</code>	177
Ressources externes	177
Coût	177
Fonctionnalités absentes	178
Cookbook	178
Surveiller un dossier et envoyer les nouveaux fichiers par email	178
S'assurer qu'un user système existe, puis basculer dessus	179
Récupérer beaucoup d'URLs en parallèle	179
Arrêt propre d'un script long-running	179
Lire une config TOML, valider, persister en SQLite	180
Cache avec TTL lisible, échéances en timestamps ISO	180
Pretty-print d'une valeur Lua	181
Hello there	181

Babet — Manuel utilisateur

Babet, n.m. — mot régional du sud-est de la France (Lyonnais, Forez, Dauphiné, Savoie, ainsi que la Suisse romande voisine) désignant une pomme de pin. Petit, léger, plein de graines, et capable d'allumer un feu — comme ce binaire.

Babet est un binaire Lua standalone pour le scripting et l'automatisation sous Linux. Ce manuel est organisé par besoin et par module : la table des matières ci-dessous indique non seulement le nom technique du module, mais aussi les fonctionnalités qu'il contient.

Documentation de la version stable et audité **2.6.0**.

Pour démarrer

- [Premiers pas](#) — installation, premier script, exécution d'un fichier ou d'un dossier, création d'un exécutable embarqué et utilisation via le PATH.
- [Sécurité](#) — modèle de menace, protections réellement fournies, limites et règles de moindre privilège.
- [Cookbook](#) — recettes complètes combinant plusieurs modules.
- [Journal des modifications](#) — notes complètes de la 2.6.0 et historique des versions précédentes, migration, validation et limites connues.

Trouver une fonctionnalité

Je cherche à...	Chapitre à consulter	Ce que j'y trouverai
créer, supprimer, lister, rechercher, copier ou déplacer des fichiers	FS – système de fichiers	fichiers, dossiers, chemins, symlinks, listFiles, find, copyTree, permissions et checksums
lire les arguments d'un script	Argparse – ligne de commande	flags, options, arguments positionnels, valeurs par défaut, choix et conversions
lancer un programme externe	Exec – processus	arguments sans shell, capture avec exec, streaming avec spawn, environnement, cwd et contrôle du processus
enchaîner plusieurs commandes	Pipelines de processus	capture complète ou streaming, stderr séparés, statuts individuels et nettoyage des groupes
créer ou inspecter/extraire des archives ZIP ou TAR	Archive – archives multi-formats sécurisées	création déterministe de ZIP et de TAR brut/gzip/xz/bzip2/zstd, listing, extraction, limites anti-bombe, refus des types spéciaux et publication atomique des fichiers
manipuler l'environnement, identifier le processus ou mesurer la VM Lua	SYS - processus et machine	version du runtime, PID, hostname, uname, PATH, env, setenv et mémoire de l'état Lua
envoyer une requête web	HTTP – client web	GET/POST et autres méthodes, query, headers validés, corps binaires, téléchargement atomique vers fichier, redirections, TLS, timeout et limites de taille
ouvrir une connexion TCP	Socket – TCP	client, serveur, acceptation, flux binaires, lignes, lecture jusqu'à EOF, timeouts, buffers et limites
chiffrer une connexion TCP ou faire STARTTLS	TLS – sockets sécurisées	TLS direct, STARTTLS, vérification, CA, hostname, SNI, versions, timeout et état après échec
stocker des données SQL localement	SQLite – base embarquée	ouverture, exécution, statements préparés, BLOB explicites et transactions assistées

Je cherche à...	Chapitre à consulter	Ce que j'y trouverai
encoder ou décoder du JSON	JSON – données structurées	scalaires, tableaux, objets, null, tableaux vides, pretty-print et erreurs
lire un fichier de configuration TOML	TOML – configuration	décodage, types TOML, tableaux, sections, dates et erreurs de parsing
surveiller un dossier	Inotify – événements fichiers	watchers, événements, lectures avec timeout, moves, cookies et fermeture
gérer les signaux Unix	SIGNAL – arrêt propre et rechargement	TERM/INT/HUP/USR1/USR2/PIPE, callbacks différés, ordre fixe, coalescence, appels
paralléliser du Lua	WORKERS – threads OS et messages	interruptibles et workers états Lua isolés, spawn, join, poll, inbox/outbox, timeouts, fermeture, sérialisation et interblocages
obtenir ou formater le temps	Time – horloges et durées	temps réel, monotone, sleep, ISO-8601, parsing et formatage de durées
rechercher un compte système par nom ou UID	USER - comptes système	NSS, get, exists, UID, GID principal, GECOS, home, shell et erreurs de résolution
découper ou transformer des chaînes	Strings – chaînes	split, séparateurs, limites et chaînes binaires
copier, fusionner ou marquer des tables	Tables – tables Lua	mergeTables, deepCopyTable, cycles et structures partagées
produire des logs	Logging – journalisation	niveaux, seuil, sortie, couleurs et comportement en cas d'erreur du sink

Modules de référence

Chaque module possède une page autonome sous modules/. Les intitulés développés ci-dessous permettent de comprendre leur périmètre sans avoir à ouvrir chaque fichier.

Module	Périmètre détaillé
Archive – opérations ZIP, TAR, TAR gzip, TAR xz, TAR bzip2 et TAR zstd sécurisées	Créer des ZIP, TAR, TAR gzip, TAR xz, TAR bzip2 ou TAR zstd déterministes et les inspecter/extraire avec ressources bornées, chemins confinés, refus explicite des types spéciaux et publication atomique fichier par fichier.
Argparse – arguments de ligne de commande	Déclarer des flags, options et positionnels ; générer l'aide ; valider les choix ; convertir les valeurs.
Exec – programmes externes et processus	Exécuter sans shell avec exec, ou piloter stdin/stdout/stderr progressivement avec spawn.
Pipelines de processus	Relier plusieurs commandes sans shell, en capture complète ou en streaming, avec statuts par étape.

Module	Périmètre détaillé
FS – fichiers, dossiers, chemins et attributs	Existence et types, création/suppression, chemins, listings, recherche, itérateur, copie/déplacement d'arbres, symlinks, modes Unix et checksums.
HTTP – requêtes web	URL, méthodes, query, headers validés, corps binaires, réponses, téléchargement atomique vers fichier, redirections, vérification TLS, timeout et limites de taille.
Inotify – surveillance du système de fichiers	Ajouter/retirer des watches, lire les événements, gérer timeouts, moves, cookies et fermeture.
JSON – encodage et décodage	Types Lua/JSON, null, tableaux vides, marquage de tableaux, indentation, UTF-8, cycles et limites.
Logging – journalisation	Niveaux, filtrage, destination, couleurs et appels variadiques.
SIGNAL – signaux POSIX et arrêt propre	Installer, remplacer ou retirer un callback ; ignorer/restaurer ; ordre de dispatch, coalescence, interruptions et restrictions multithread.
Socket – TCP client et serveur	Connexion, écoute, acceptation, flux binaires, lectures par bloc/ligne/EOF, buffer partagé, adresses, fermeture et timeouts.
SQLite – base de données embarquée	Connexions, WAL/timeout, SQL direct, statements préparés réutilisables, BLOB explicites, itération et transactions assistées.
Strings – manipulation de chaînes	Découpage, mode caractères, séparateurs, limite de splits et contenu binaire.
SYS - processus, machine, environnement et mémoire Lua	Constantes de version, PID, hostname, uname, recherche d'exécutables, lecture/modification de l'environnement, interaction avec les workers et mémoire de l'état Lua.
Tables – manipulation de tables Lua	Fusion déterministe, clés listes/maps, copie profonde, cycles et partage de sous-tables.
Time – horloges, ISO et durées	Temps réel et monotone, sommeil, formatage/parsing ISO-8601, parsing et rendu de durées.
TLS – connexions chiffrées	Connexion TLS directe, STARTTLS, vérification, CA, hostname, SNI, versions, deadlines et comportement fail-closed.
TOML – fichiers de configuration	Décodage TOML, scalaires, tableaux, tables, tableaux de tables, dates/heures et diagnostics.
USER - utilisateurs système via NSS	Recherche par nom ou UID, existence, distinction absence/erreur NSS, champs passwd, workers et limites de sécurité.
WORKERS – threads OS et files de messages	États Lua isolés, transport JSON, résultat consommable, poll/join, inbox/outbox, timeouts, fermeture, GC et pièges d'interblocage.

Organisation d'une page de module

Les pages détaillées sont progressivement alignées sur la structure suivante :

1. **Périmètre** — ce que couvre le module et ce qu'il ne couvre pas ;

2. **Table des matières interne** — accès direct à chaque groupe et fonction ;
3. **Vue d'ensemble de l'API** — signatures et résultats ;
4. **Comportement détaillé** — valeurs par défaut, symlinks, récursivité, limites et effets de bord ;
5. **Exemples** — un exemple par mode ou option importante ;
6. **Contrat d'erreur** — erreurs levées et erreurs renvoyées ;
7. **Décisions et limites** — choix d'API et éléments volontairement absents.

L'objectif est qu'une fonction puisse être utilisée sans lire son code source et sans devoir deviner ses valeurs par défaut.

Génération du PDF

Le manuel peut être exporté en un seul PDF :

```
cd docs
./build_doc.sh
```

L'ordre des chapitres français est défini dans `manual_order_fr.txt`. Le script de génération ajoute ensuite la table des matières du PDF à partir des titres et sous-titres.

Méthodologie de vérification

La documentation est vérifiée module par module selon trois sources :

- l'implémentation C/C++ réellement enregistrée dans `babet` ;
- les tests de non-régression de `examples/main.lua` et `run_tests.sh` ;
- les pages française et anglaise, qui doivent décrire le même contrat.

Une divergence découverte pendant cette vérification est traitée comme une incohérence à corriger, et non comme un détail rédactionnel à masquer.

Pour démarrer

Téléchargement

Des binaires précompilés pour `x86_64`, `aarch64` (RPi4) et `armv6l` (RPi0) sont disponibles sur la [page des releases](#).

Compilation depuis les sources

Babet embarque toutes ses dépendances (Lua, OpenSSL, SQLite, miniz, libarchive, zlib, XZ Utils/liblzma, bzip2/libbz2, Zstandard/libzstd, RE2, Abseil, nlohmann/json, cpp-httplib, tomlplusplus), donc les seuls prérequis sur ton système sont :

- un compilateur C++23 (GCC ou Clang récent)
- CMake (≥ 3.22)
- `wget`, `unzip` et `xz`

```
git clone https://github.com/Chipsterjulien/babet.git
cd babet
./build_local.sh           # télécharge les deps et compile
                           # (~5 min au premier lancement, plus vite après)
./run_tests.sh             # harness offline – doit terminer avec 0 FAIL
```

Le script de build télécharge chaque dépendance depuis une source amont épinglée, vérifie son SHA256, puis la compile statiquement. Plusieurs bibliothèques disposent d'un miroir ou d'une URL de secours explicite. Lorsqu'il n'existe aucun miroir, le script indique le nom exact de l'archive à déposer manuellement dans `downloads/` ; le même contrôle SHA256 reste appliqué avant l'extraction.

Le binaire produit est dans test/babet.

Trois manières de lancer un script Lua

Babet supporte trois modes de lancement :

1. Mode dossier

Pointe le binaire vers un dossier contenant main.lua :

```
echo 'print("hello from Babet")' > main.lua
./test/babet .
```

require("X") dans main.lua cherche X.lua dans le même dossier.

2. Mode embarqué (binaire autonome)

Empaquète un script et ses modules dans un binaire autonome :

```
./test/babet --create-exe . myapp
./myapp          # lance main.lua depuis le ZIP embarqué
```

Les dossiers de métadonnées de contrôle de version .git, .svn et .hg sont exclus automatiquement, à n'importe quelle profondeur.

En interne, Babet append un ZIP à son propre binaire et lit main.lua (plus tous les modules require) depuis ce ZIP au runtime. Le myapp résultant est totalement autonome — pas besoin d'avoir Babet installé sur la machine cible, juste glibc.

3. Embarqué via PATH (dossier + auto-détection)

Si Babet est sur \$PATH et qu'on fait chmod +x main.lua après avoir ajouté une ligne shebang :

```
#!/usr/bin/env babet
print("hello")

chmod +x main.lua
./main.lua
```

Babet utilise le dossier du script comme dossier de travail, donc require("helpers") trouvera helpers.lua à côté de main.lua.

Invocation en ligne de commande

En plus des trois modes de lancement ci-dessus, Babet accepte quelques flags standards :

Flag	Effet
-h, --help	Affiche l'aide et sort avec code 0.
-V, --version	Affiche babet <version> et sort avec code 0.
-c <dir> <out>, --create-exe <dir> <out>	Crée un exécutable autonome nommé <out> en embarquant <dir> (qui doit contenir main.lua).

Tout autre argument commençant par - est traité comme une **option inconnue** : Babet affiche Unknown option: ... + un indice pour utiliser --help, et sort en code 1 plutôt que d'essayer de l'interpréter comme un nom de dossier. Les dossiers dont le nom commence légitimement par - peuvent toujours être passés via ./-dirname (convention POSIX).

```
babet --version    # babet 2.6.0
babet --help      # usage complet
babet --bogus     # Unknown option: --bogus
                  # Try 'babet --help' for more information.
```

La même version est aussi exposée aux scripts via `babet.VERSION` (plus `babet.VERSION_MAJOR` / `VERSION_MINOR` / `VERSION_PATCH` en integer) — voir [sys](#).

Premier script

Une fois le binaire compilé, essaie ceci :

```
-- main.lua
print("Babet dit bonjour")
print("PID :", babet.pid())
print("Hôte :", babet.hostname())

local r, err = babet.http.get("https://example.com/")
if r then
    print("Status :", r.status)
else
    print("Échec HTTP :", err)
end
```

Lance-le avec `./test/babet .` (ou un autre mode au choix).

Pour aller plus loin

- Chaque module a sa propre page sous `modules/`.
- Le module `user` est un petit exemple complet du pattern de documentation utilisé partout — lis-le comme la référence canonique.
- Pour les utilitaires de dates et durées (timestamps ISO 8601, durées lisibles comme "5m" ou "2h30m"), voir `time` — la sous-table `babet.time.*` couvre le parsing et le formatage.
- Vois [security.md](#) avant d'exposer des scripts Babet à quoi que ce soit qui pourrait recevoir de l'input non fiable.

Modèle de sécurité

Babet exécute des scripts Lua de confiance. Ce n'est pas une sandbox.

Les scripts s'exécutent avec la bibliothèque standard Lua complète (`luaL_openlibs`), et Babet expose en plus des primitives système comme `exec`, `remove`, `rmdirAll` et `chdir`. Un script a donc les mêmes privilèges que le processus qui le lance : il peut lire, modifier ou supprimer des fichiers, et lancer des commandes arbitraires. C'est volontaire — comme `make`, un script shell, ou n'importe quel outil de build, Babet est conçu pour exécuter du code que tu contrôles.

Ne lance que des `main.lua` et des modules `require` auxquels tu fais confiance. **N'utilise pas** Babet pour exécuter du Lua venant de sources non fiables ; il n'offre aucune isolation contre du code hostile, et n'est pas pensé pour. Si tu dois exécuter des scripts non fiables, utilise plutôt une sandbox Lua dédiée à cet usage.

Ce contre quoi le durcissement *protège*

Le travail de durcissement dans Babet protège les usages *légitimes* contre les accidents et les attaques de supply chain :

- **Confinement de `copyTree` / `moveTree`** : la racine destination est ouverte une fois, puis chaque création, copie et déplacement est effectué relativement à ce descripteur avec

- openat/mkdirat/renameat et O_NOFOLLOW/AT_SYMLINK_NOFOLLOW. Un symlink préexistant ou remplacé pendant l'opération ne peut pas rediriger une écriture vers une cible extérieure. Les gardes `is_within()` empêchent en plus de choisir une destination résolue dans la source.
- **Création et extraction ZIP confinées** : `babel.archive` refuse les chemins de source de création et de destination de sortie/extraction lorsqu'ils sont symlinkés, les objets source non pris en charge, les noms source non UTF-8, une sortie dans l'arbre source, les chemins d'entrée absolus, les composants `./..`, les backslashes, les doublons, les symlinks ZIP et les types spéciaux. Les archives à lire sont ouvertes une fois et doivent se résoudre vers un fichier régulier, ce qui écarte notamment les FIFO bloquants. La destination est parcourue avec des descripteurs et O_NOFOLLOW; chaque fichier est décompressé dans un temporaire du même répertoire, soumis à des limites anti-bombe, puis publié atomiquement.
 - **Cleanup borné des groupes de processus** dans `babel.exec` couvre aussi la phase `chdir/exec`, les erreurs internes de polling et le cas où un enfant ferme ses pipes tout en continuant à tourner. Au timeout, TERM puis KILL visent le groupe entier ; Babet ne fait jamais de `waitpid` final illimité dans ces chemins d'erreur.
 - **Limites de sortie** sur `babel.exec` bornent la quantité de `stdout` et `stderr` capturés en mémoire (par défaut 10 MiB chacun, configurable), pour qu'un sous-processus emballé ne puisse pas OOM le processus Babet.
 - **Mises à jour locales épinglées** : `setAttributes` résout sa cible une seule fois et maintient `chown`, `chmod` et le `rollback` sur le même inode. `touch` crée avec O_EXCL, ne tronque jamais un fichier existant et épingle une cible existante avant de modifier son horodatage.
 - **Staging anonyme et publication épinglée des exécutables** : `--create-exe` supprime le nom de son ZIP temporaire immédiatement après `mkstemp`, puis laisse `miniz` et la fusion rouvrir l'inode encore vivant via `/proc/self/fd`, jamais par un chemin temporaire remplaçable. La fusion finale épingle le dossier de sortie et publie avec `renameat()` ; modifier un parent symlinké ne peut donc pas rediriger l'exécutable.
 - **Checksums des dépendances** : chaque dépendance embarquée est SHA256-pinnée dans `build_local.sh`. Un hash vide refuse la build. Un mismatch supprime le fichier téléchargé et sort en erreur. Ça protège contre un upstream compromis ou une archive Wayback Machine altérée (source de fallback).
 - **Vérification TLS** est activée par défaut (`verify=true`). La désactiver demande un `verify=false` explicite par appel — pas de fallback silencieux. Pour les sockets TLS, des CA supplémentaires peuvent être passées via `ca_cert` ou `ca_path`; HTTP expose `ca_cert`. Les variables OpenSSL `SSL_CERT_FILE` / `SSL_CERT_DIR` restent également disponibles. SNI est envoyé indépendamment de la vérification lorsqu'un hostname DNS est connu.
 - **Validation des lignes HTTP** : les URL et valeurs de headers refusent CR/LF, et les noms de headers doivent respecter la grammaire HTTP token. Une valeur contrôlée par un utilisateur ne peut donc pas injecter un header ou une seconde ligne de requête via ces champs.
 - **Limites réseau en mémoire** : `socket:recv_line`, `socket:recv_all` et `http.max_body_size` bornent les accumulations principales. Après un timeout de lecture socket, les octets déjà consommés restent dans un buffer commun afin de ne pas être perdus ou réordonnés si le script change de méthode.
 - **Filtrage borné des noms** : les champs `regex name`, `iname` et `path` de `babel.find` utilisent RE2 avec une limite de 4096 octets et un budget mémoire de 1 Mio par expression ; les globs sûrs disposent eux aussi d'une limite de 4096 octets et d'un automate non récursif. Les références arrière et assertions d'anticipation/rétrospection ne font volontairement pas partie de la syntaxe RE2.

Ce contre quoi il ne protège pas

- Un script malveillant. Babet n'a pas de sandbox. `babel.exec` n'invoque pas de shell, mais laisser une entrée non fiable choisir l'exécutable ou ses arguments peut tout de même permettre l'exécution d'un programme arbitraire ou une injection d'options. Transmettre du texte non fiable à `load` donne une exécution arbitraire de code Lua.
- Un attaquant déterminé qui a déjà obtenu une exécution de code sur la machine qui fait tourner Babet. Le durcissement rend les erreurs accidentelles visibles, pas les attaques adversariales

impossibles.

- Les problèmes OS-level (exploits kernel, évasions de conteneur, escalade de privilèges). Babet est un binaire userland ordinaire.

Pattern recommandé : moindre privilège

Quand tu fais tourner Babet en service, traite-le comme n'importe quel processus non privilégié :

```
# En root, crée un user système dédié, sans shell.
```

```
useradd --system \
        --home-dir /var/lib/myapp \
        --create-home \
        --shell /usr/sbin/nologin \
        --comment "myapp Babet service" \
        myapp
```

```
# Utilise babet.user.exists("myapp") dans ton script d'install
# pour vérifier que le user est provisionné avant de lancer le
# daemon.
```

Combine ça avec des options de unit systemd comme User=myapp, PrivateTmp=yes, ProtectSystem=strict, NoNewPrivileges=yes, et CapabilityBoundingSet= (vide sauf si tu as vraiment besoin d'une capability). Le kernel fait bien plus que Babet ne peut le faire pour contenir un script qui se comporte mal ; laisse-le faire.

Archive — opérations ZIP et TAR sécurisées avec gzip, xz, bzip2 ou zstd

Périmètre

Le sous-module `babet.archive` crée, inspecte et extrait des archives **ZIP**, **TAR** et **TAR compressées avec gzip, xz, bzip2 ou zstd**, sans lancer de commande externe :

```
babet.archive.create(source, archive [, opts])
babet.archive.list(archive [, opts])
babet.archive.extract(archive, destination [, opts])
babet.archive.extractFile(archive, entry, destination [, opts])
```

La création, l'inspection et l'extraction ZIP continuent d'utiliser miniz. Les opérations TAR utilisent le backend libarchive lié statiquement ; gzip s'appuie sur zlib, xz sur XZ Utils/liblzma, bzip2 sur libbz2 et zstd sur Zstandard/libzstd, également liés statiquement. Les lecteurs détectent le conteneur et la compression à partir du contenu. Pour `create()`, `.tar` sélectionne un TAR brut, `.tar.gz` et `.tgz` un TAR gzip, `.tar.xz` ou `.txz` un TAR xz, `.tar.bz2`, `.tbz2` ou `.tbz` un TAR bzip2, et `.tar.zst`, `.tar.zstd` ou `.tzst` un TAR zstd. `opts.format` peut remplacer l'extension. Les flux compressés simples ne sont pas encore activés.

Les quatre fonctions suivent le contrat habituel :

```
local result, err = babet.archive.list("backup.zip")
if not result then
    io.stderr:write(err, "\n")
end
```

Elles renvoient (résultat, nil) en cas de succès et (nil, message) en cas d'échec. Une mauvaise arité ou un argument obligatoire qui n'est pas une chaîne provoque une erreur Lua. Une table d'options invalide renvoie (nil, message).

Table des matières

- [babet.archive.create](#)
- [Options d'inspection/extraction et limites anti-bombe](#)
- [babet.archive.list](#)
- [babet.archive.extract](#)
- [babet.archive.extractFile](#)
- [Règles de chemins](#)
- [Protection de la destination](#)
- [Publication atomique et nettoyage](#)
- [Permissions](#)
- [Types d'entrées pris en charge](#)
- [Erreurs et limites connues](#)

babet.archive.create

```
local result, err = babet.archive.create(  
    source,  
    archive  
    [, opts]  
)
```

source doit être un répertoire existant. Babet archive son contenu, pas le nom du répertoire source lui-même. Le parcours utilise des descripteurs et ne suit aucun symlink. Un symlink dans le chemin ou l'arbre source, ainsi qu'un FIFO, socket, périphérique ou autre objet non pris en charge, fait échouer l'opération avant la publication de l'archive. Les noms d'entrées produits par create() doivent être en UTF-8 valide. Linux autorise des noms contenant des octets arbitraires, mais Babet refuse ceux qui ne peuvent pas être représentés de manière sûre et cohérente dans les deux formats de sortie.

Format de sortie

Sans opts.format, create() applique les règles suivantes :

- une destination terminée par .tar (sans distinction de casse) crée un TAR POSIX pax non compressé ;
- .tar.gz et .tgz (sans distinction de casse) créent un TAR POSIX pax compressé avec gzip ;
- .tar.xz et .txz (sans distinction de casse) créent un TAR POSIX pax compressé avec xz ;
- .tar.bz2, .tbz2 et .tbz (sans distinction de casse) créent un TAR POSIX pax compressé avec bzip2 ;
- .tar.zst, .tar.zstd et .tzst (sans distinction de casse) créent un TAR POSIX pax compressé avec zstd ;
- toute autre destination conserve le comportement historique antérieur à la 2.6 et produit un ZIP, quelle que soit son extension.

opts.format = "zip", "tar", "tar.gz", "tar.xz", "tar.bz2" ou "tar.zst" impose explicitement le backend. Le format explicite prime toujours sur le suffixe. Ainsi, format = "zip" peut créer un ZIP nommé backup.tar.gz, tandis que format = "tar" peut créer un TAR non compressé nommé backup.tgz, backup.txz, backup.tbz2 ou backup.tzst. Le format réel est déterminé par l'option explicite ou le contenu, jamais par une confiance aveugle dans l'extension.

Options de création :

Option	Défaut	Comportement
format	déduit	chaîne stricte : "zip", "tar", "tar.gz", "tar.xz", "tar.bz2" ou "tar.zst"

Option	Défaut	Comportement
<code>compression_level</code>	6	ZIP/gzip/xz : 0 à 9 ; bzip2 : 1 à 9 ; zstd : 0 à 19
<code>overwrite</code>	false	remplace atomiquement une archive régulière existante
<code>deterministic</code>	true	ordre stable et dates fixes propres au format
<code>include_directories</code>	true	émet les entrées de dossiers, y compris les dossiers vides
<code>max_entries</code>	10000	nombre maximal d'entrées ; plafond fixe 100000
<code>max_file_size</code>	256 * 1024 * 1024	taille maximale d'un fichier source ; plafond fixe 8 Gio
<code>max_total_size</code>	1024 * 1024 * 1024	somme maximale des octets source ; plafond fixe 64 Gio

Pour un TAR brut, fournir explicitement `compression_level` est une erreur. Pour ZIP, TAR gzip et TAR xz, le niveau 0 est valide et le niveau 9 est le maximum. TAR bzip2 accepte les niveaux 1 à 9 ; le niveau 0 est refusé car les tailles de bloc libbz2 commencent à 100 Kio. TAR zstd accepte les niveaux 0 à 19 ; Babet exclut volontairement les niveaux ultra 20 à 22.

Toutes les options entières exigent de vrais entiers Lua. Les options booléennes exigent de vrais booléens Lua. Les clés inconnues sont refusées. `format` est sensible à la casse et n'accepte aucun alias.

L'arbre source complet est analysé avant la création du fichier temporaire de sortie. Chaque nom est borné à 4096 octets et leur somme à 64 Mio. Le parcours possède aussi des plafonds fixes de 100000 objets du système de fichiers et 256 niveaux de répertoires, indépendamment de `include_directories`. Les fichiers sont ensuite rouverts depuis le descripteur du répertoire source avec `O_NOFOLLOW`. Le périphérique, l'inode, la taille, la date de modification et la date de changement sont vérifiés avant et après la lecture en flux ; un fichier modifié pendant la création provoque un échec et la suppression du temporaire.

Le parent de destination doit déjà exister et ne contenir aucun symlink ni composant ... La destination doit être absente sauf avec `overwrite = true`, et ne peut jamais être un symlink ou un répertoire. L'archive ne peut pas être créée dans l'arbre source, ce qui évite son auto-inclusion accidentelle.

Le résultat indique le format sélectionné :

```
{
  files = 12,
  directories = 3,
  bytes = 987654,
  path = "backup.tar",
  format = "tar",
  compression = "none",
  compression_level = nil,
  deterministic = true,
}
```

Pour ZIP et TAR gzip, xz, bzip2 ou zstd, `compression_level` contient le niveau entier effectif. Pour un TAR brut, il vaut nil. Le champ d'archive `compression` vaut "gzip", "xz", "bzip2" ou "zstd" pour un TAR compressé et "none" sinon. La compression des entrées ZIP reste indiquée par les métadonnées par entrée renvoyées par `list()`.

Avec le mode déterministe par défaut, des contenus, un format, des versions de libarchive/miniz/zlib/liblzma/libb et des options identiques produisent des archives strictement identiques octet par octet :

- les entrées ZIP utilisent la date DOS fixe du 1er janvier 1980 à 00:00:00 ;
- les entrées TAR utilisent l'époque Unix zéro, les UID/GID 0, des noms de propriétaire/groupe vides, le mode 0644 pour les fichiers et 0755 pour les répertoires ;
- un TAR gzip écrit en plus une date nulle dans l'en-tête gzip, afin que le flux externe n'introduise pas l'heure courante ;
- un flux xz ne contient aucune date courante et son preset LZMA2 est fixé par `compression_level` ;
- un flux bzip2 ne contient aucune date courante et sa taille de bloc est fixée par `compression_level` ;
- une frame zstd ne contient aucune date courante, active son checksum de frame et utilise le niveau fixé par `compression_level`.

Avec `deterministic = false`, l'ordre trié reste stable. ZIP utilise la date de modification source, qui doit être comprise entre 1980 et 2107 dans le fuseau local. TAR conserve la date de modification source avec sa précision en nanosecondes lorsqu'elle est représentable par la plateforme et n'est pas limité par la plage DOS du ZIP.

Les propriétaires, ACL, attributs étendus et bits de permission source ne sont pas copiés. TAR utilise le writer POSIX pax restreint de libarchive : les noms ordinaires utilisent des en-têtes ustar portables lorsque c'est possible, et les chemins longs ou métadonnées nécessitant une extension utilisent des en-têtes pax. ZIP64 est activé automatiquement lorsque la préanalyse indique que les champs ZIP classiques risquent d'être dépassés. Les petits ZIP restent des ZIP classiques.

La préanalyse et les contrôles empêchent les substitutions par symlink et détectent la modification de chaque fichier réellement archivé. La création n'est toutefois pas un instantané du système de fichiers : un fichier ajouté après l'analyse de son répertoire peut être absent du résultat ; un fichier déjà recensé puis supprimé ou renommé fait normalement échouer sa réouverture. Les renommages concurrents de répertoires ne sont pas signalés comme conflit transactionnel.

Avec `include_directories = false`, les répertoires vides ne peuvent pas être représentés et sont donc omis. Les parents nécessaires aux fichiers réguliers sont recréés implicitement pendant l'extraction.

Exemples :

```
local zip, err = babet.archive.create("projet", "projet.zip", {
    compression_level = 9,
    overwrite = true,
})
assert(zip, err)

local tar
tar, err = babet.archive.create("projet", "projet.tar", {
    deterministic = true,
})
assert(tar, err)

local gzip
gzip, err = babet.archive.create("projet", "projet.tar.gz", {
    compression_level = 9,
})
assert(gzip, err)

local xz
xz, err = babet.archive.create("projet", "projet.tar.xz", {
    compression_level = 9,
})
```

```

assert(xz, err)

local bzip2
bzip2, err = babet.archive.create("projet", "projet.tar.bz2", {
    compression_level = 9,
})
assert(bzip2, err)

local zstd
zstd, err = babet.archive.create("projet", "projet.tar.zst", {
    compression_level = 19,
})
assert(zstd, err)

local named
named, err = babet.archive.create("projet", "sauvegarde.data", {
    format = "tar.zst",
})
assert(named, err)

```

Options d'inspection/extraction et limites anti-bombe

Les options communes à `list()`, `extract()` et `extractFile()` sont :

Option	Défaut	Maximum accepté	Effet
<code>max_entries</code>	10000	100000	nombre maximal d'entrées dans l'archive
<code>max_entry_size</code>	256 * 1024 * 1024	8 * 1024 ³	taille décompressée maximale d'une entrée non répertoire
<code>max_total_size</code>	1024 * 1024 * 1024	64 * 1024 ³	somme maximale des tailles décompressées des entrées non répertoire
<code>max_compression_ratio</code>	1000	1000000000	rapport par entrée pour ZIP ; rapport global octets fichiers décompressés / taille archive pour TAR gzip, xz, bzip2 ou zstd ; sans effet pour TAR brut

Les trois limites entières doivent être des **entiers Lua strictement positifs**. `max_compression_ratio` doit être un nombre fini supérieur ou égal à 1. NaN, l'infini, les booléens et les chaînes numériques sont refusés.

Indépendamment des options, la somme des longueurs des noms d'entrées est bornée en interne à 64 Mio. Cette limite fixe borne la mémoire consacrée aux métadonnées, y compris lorsque `max_entries` est relevé explicitement. Les allocations réalisées par le lecteur ZIP miniz sont en

autre plafonnées à 128 Mio par archive. Le répertoire central et ses commentaires ne peuvent donc pas provoquer une allocation mémoire arbitraire avant l'application des limites entrée par entrée. L'inspection TAR fonctionne en flux : Babet lit les données de chaque entrée sans les conserver en mémoire, à la fois pour atteindre l'en-tête suivant et pour détecter les données tronquées. Pour TAR gzip, Babet effectue en plus une validation progressive et bornée avec zlib, car libarchive ne vérifie pas lui-même le CRC/ISIZE de la bande-annonce gzip ; les bandes-annonces corrompues et les données finales non gzip sont donc refusées. Le bourrage nul standard après un membre gzip reste accepté. Pour TAR zstd, Babet effectue une validation progressive et bornée indépendante avec libzstd : les frames complètes et les frames concaténées valides sont acceptées, tandis que corruption, troncature ou données finales étrangères sont refusées avant publication.

Pour une extraction complète, l'arbre de sortie est aussi borné en interne à 100000 répertoires uniques, parents implicites compris, et à 64 Mio de chemins de répertoires normalisés cumulés. Une poignée d'entrées extrêmement profondes ne peut donc pas provoquer une explosion non bornée du nombre de mkdir ou de la mémoire consacrée aux préfixes.

Ces limites sont appliquées pendant l'inspection de **toute l'archive**, y compris avec list() et extractFile(). Ainsi, extractFile() ne contourne pas la politique globale en sélectionnant une petite entrée au milieu d'une archive dont les métadonnées annoncent une expansion excessive.

Une entrée non vide qui annonce une taille compressée nulle est toujours refusée. Les répertoires ne comptent pas dans max_entry_size, max_total_size ni le rapport de compression, mais ils comptent dans max_entries.

Exemple :

```
local info, err = babet.archive.list("upload.zip", {
    max_entries = 2000,
    max_entry_size = 64 * 1024 * 1024,
    max_total_size = 512 * 1024 * 1024,
    max_compression_ratio = 200,
})
```

Les clés inconnues sont refusées. Les options propres à la création et à l'extraction ne sont pas acceptées par list().

babet.archive.list

```
local info, err = babet.archive.list(archive [, opts])
```

archive est le chemin d'un fichier ZIP, TAR brut, TAR gzip, TAR xz, TAR bzip2 ou TAR zstd. L'extension ne sert pas à la détection. Le chemin peut être un symlink vers un fichier régulier : Babet ouvre une seule fois sa cible et conserve ce même descripteur pendant toute l'analyse. Les répertoires, FIFO, sockets et périphériques sont refusés avant l'analyse du format ; l'ouverture non bloquante évite notamment qu'un FIFO sans écrivain suspende list().

Pour un ZIP, Babet essaie d'abord le lecteur miniz existant. Si le même fichier épinglé n'est pas un ZIP, Babet rembobine ce descripteur et n'active que le lecteur TAR de libarchive avec les filtres intégrés none, gzip, xz, bzip2 et zstd. Aucun décompresseur externe ne peut être lancé par ce chemin. Les archives TAR concaténées sont parcourues intégralement ; les données finales qui ne forment pas un TAR, les contenus tronqués et les données attachées à une entrée non régulière sont refusés. La fonction applique ensuite les limites précédentes et renvoie :

```
{
    format = "zip",           -- "zip" ou "tar"
    compression = "none",    -- "none", "gzip", "xz", "bzip2" ou "zstd" pour TAR
    entries = {
        {
            name = "docs/readme.txt", -- nom brut de l'entrée
            path = "docs/readme.txt", -- chemin normalisé pour l'extraction
            type = "file",             -- voir les types d'entrées pris en charge ci-dessous
        }
    }
}
```

```

        size = 1234,
        compressed_size = 530,
        crc32 = 305419896,          -- entier non signé sur 32 bits
        compression_method = 8,
        encrypted = false,
        supported = true,
        safe_path = true,
        extractable = true,
        reason = nil,
        unix_mode = 420,             -- 0644, ou nil sans mode Unix
        sparse = false,
        link_target = nil,          -- cible d'un symlink/hard link TAR
    },
    },
    count = 1,
    total_size = 1234,
    archive_size = 666,
    zip64 = false,                  -- nil pour TAR
}

```

path retire le / terminal d'une entrée répertoire. Il n'est renseigné de façon utile que si `safe_path == true`.

Pour un ZIP, `extractable` indique que l'entrée possède un chemin acceptable, un type pris en charge, une méthode de compression comprise par miniz et qu'elle n'est pas chiffrée. Pour un TAR, les fichiers réguliers et répertoires sûrs sont extractibles. Les fichiers sparse, symlinks, hard links, FIFO, périphériques et autres types spéciaux TAR sont inspectés mais refusés avec une raison explicite. `extractable` ne prédit jamais les conflits avec la destination ni les doublons entre entrées.

Les champs propres au ZIP conservent leurs valeurs exactes antérieures. Pour un TAR, `compressed_size`, `crc32` et `compression_method` valent nil, car TAR ne porte ni compression ni CRC par entrée. `encrypted` vaut false, `supported` indique que libarchive a pu analyser l'entrée, et `sparse` signale les métadonnées de fichier creux. `link_target` contient la cible brute annoncée pour un symlink ou hard link TAR lorsque libarchive l'expose, et vaut nil sinon. Les types TAR retournés par ce lot sont file, directory, symlink, hardlink, fifo, character_device, block_device, socket et unsupported.

`unix_mode` contient uniquement les bits 0000 à 7777 annoncés par une archive créée sur un système Unix. Il vaut nil lorsque cette information n'existe pas. La présence d'un mode ne signifie pas qu'il sera conservé : voir [preserve_permissions](#).

Les noms ZIP lus dans une archive existante sont des chaînes d'octets. Les chemins TAR sont les chaînes natives renvoyées par libarchive après traitement des en-têtes TAR, y compris les préfixes ustar, noms longs GNU et champs path pax. Les comparaisons sont exactes et sensibles à la casse, et Babet n'effectue aucune normalisation Unicode supplémentaire. Cette tolérance d'inspection n'affaiblit pas `create()`, qui n'émet que des noms UTF-8 valides en ZIP comme en TAR.

babet.archive.extract

```

local result, err = babet.archive.extract(
    archive,
    destination
    [, opts]
)

```

Options supplémentaires :

Option	Défaut	Comportement
overwrite	false	autorise le remplacement atomique d'un fichier régulier existant
preserve_permissions	false	conserve les bits Unix ordinaires rwx lorsqu'ils sont disponibles

Ces deux valeurs doivent être de vrais booléens Lua.

archive peut être un ZIP, un TAR brut, un TAR gzip, un TAR xz, un TAR bzip2 ou un TAR zstd ; format et compression sont détectés par le contenu, pas par l'extension. Avant toute écriture de fichier, Babet :

1. inspecte intégralement l'archive et applique les limites ;
2. refuse toute entrée non extractible ;
3. refuse les chemins de sortie dupliqués ;
4. refuse les conflits où un même chemin devrait être à la fois fichier et répertoire ;
5. vérifie les éléments déjà présents dans la destination.

Pour un TAR, Babet relit ensuite le **même fichier épinglé** depuis le début, compare chaque en-tête avec le plan issu de l'inspection, puis transmet les blocs des fichiers réguliers à la couche de destination sécurisée. Une entrée ajoutée, supprimée ou dont les métadonnées diffèrent entre les deux passes fait échouer l'opération. Cette vérification ne constitue pas un instantané : une modification concurrente de données de même taille peut encore être détectée par une erreur de lecture, mais n'est pas garantie d'être distinguée si les en-têtes restent identiques.

Le résultat est :

```
{
  files = 12,
  directories = 3, -- entrées répertoire explicites dans l'archive
  bytes = 987654, -- octets décompressés des fichiers
  path = "restauration",
}
```

directories ne compte pas les parents implicites créés pour atteindre un fichier.

Exemple :

```
local result, err = babet.archive.extract("backup.tar", "restore", {
  overwrite = false,
  preserve_permissions = true,
  max_total_size = 2 * 1024 * 1024 * 1024,
})

if not result then
  error(err)
end

print(result.files, result.bytes)
```

babet.archive.extractFile

```
local result, err = babet.archive.extractFile(
  archive,
  entry,
  destination
```

```
[, opts]
)
```

archive peut être un ZIP, un TAR brut, un TAR gzip, un TAR xz, un TAR bzip2 ou un TAR zstd ; la détection repose sur le contenu et non sur l'extension. Le ZIP reste traité par miniz. Le TAR utilise la même source épinglée et la même lecture libarchive en deux passes que l'extraction complète.

entry est le **nom brut exact** exposé par `list()`. La recherche est sensible à la casse et échoue si le nom est absent ou apparaît plusieurs fois. Pour TAR, les noms longs GNU et les enregistrements pax path sont recherchés après leur décodage par libarchive, exactement tels qu'ils figurent dans `list().entries[i].name`.

Seule une entrée de type fichier régulier peut être sélectionnée. Les fichiers TAR sparse, symlinks, hard links, répertoires et types spéciaux sont refusés lorsqu'ils sont sélectionnés. Le chemin de l'entrée n'est pas reproduit : le contenu est écrit exactement dans le chemin destination fourni par l'appelant.

```
local result, err = babet.archive.extractFile(
    "package.tar",
    "assets/logo.bin",
    "cache/current-logo.bin",
    { overwrite = true }
)
```

Résultat :

```
{
    bytes = 4096,
    path = "cache/current-logo.bin",
    entry = "assets/logo.bin",
}
```

Les limites anti-bombe restent calculées sur l'ensemble de l'archive. Pour TAR, la première passe consomme et valide tous les flux de données. La seconde compare chaque en-tête avec le plan inspecté et parcourt l'archive jusqu'à sa fin avant de publier le fichier temporaire sélectionné. Les données des autres fichiers réguliers sont ignorées ; un chemin dangereux, un type spécial ou un fichier sparse non sélectionné n'empêche donc pas de choisir un fichier régulier sûr. En revanche, toute donnée malformée ou tronquée ailleurs dans l'archive fait échouer l'opération sans publier la destination.

Règles de chemins

Pour une extraction complète, chaque nom d'entrée doit respecter toutes les règles suivantes :

- entre 1 et 4096 octets ;
- aucun octet NUL ;
- aucun \ ;
- pas de / initial ;
- pas de préfixe de lecteur comme C : dans le premier composant ;
- aucun composant vide (a//b) ;
- aucun composant . ou .. ;
- un / terminal uniquement pour une véritable entrée répertoire.

Les chemins restent relatifs à la destination. Babet ne transforme jamais un nom Windows en chemin Linux et ne « nettoie » jamais silencieusement une traversée : l'archive est refusée. Avec `extract-File()`, les mêmes règles s'appliquent au nom de l'entrée sélectionnée ; les noms dangereux non sélectionnés ne deviennent jamais des chemins de destination.

Les doublons sont détectés après normalisation du / terminal des répertoires. Une archive contenant deux sorties identiques, ou node comme fichier puis node/child, est refusée avant extraction.

Protection de la destination

La sécurité ne repose pas seulement sur une comparaison de chaînes. Babet parcourt la destination composant par composant avec des descripteurs de répertoire et refuse de suivre les symlinks :

- un symlink dans le chemin racine de destination est refusé ;
- un parent symlinké sous la destination est refusé ;
- une cible finale symlinkée est refusée, même avec `overwrite = true` ;
- un fichier ne peut remplacer qu'un fichier régulier ;
- une entrée répertoire ne peut utiliser qu'un répertoire existant ou nouveau.

Les répertoires déjà présents sont acceptés s'ils sont réels et restent inchangés. Babet ne modifie pas leurs permissions.

Le chemin de destination lui-même est fourni par le script et peut être absolu ou relatif. Les règles sur `..` décrites plus haut concernent les noms non fiables contenus dans l'archive.

Publication atomique et nettoyage

`create()` écrit le ZIP, TAR brut, TAR gzip ou TAR xz complet dans un temporaire unique de mode `0600`, placé dans le répertoire de destination. Son nom interne court ne reprend pas le nom de sortie, afin qu'une destination valide proche de la limite `NAME_MAX` reste créable. Après finalisation du répertoire central ZIP par `miniz` ou du flux TAR par `libarchive`, Babet vide les tampons, exécute `fsync`, passe le fichier en `0644`, le publie atomiquement, puis synchronise le répertoire parent. Sans écrasement, la publication par hard link refuse toute cible apparue entre-temps ; avec écrasement, un `rename` du même répertoire remplace atomiquement la cible. Toute erreur antérieure à la publication supprime le temporaire et laisse l'ancienne destination intacte. Si le `fsync` final du répertoire parent échoue, la nouvelle archive est déjà publiée atomiquement, mais l'appel renvoie une erreur car la persistance après panne ne peut pas être confirmée.

Chaque fichier extrait est écrit progressivement vers un fichier temporaire unique créé dans son répertoire final avec le mode `0600`. Les noms temporaires sont choisis de façon à ne jamais entrer en collision avec une sortie déclarée par l'archive. L'écriture est bornée par la taille annoncée. Pour ZIP, `miniz` vérifie la décompression et le CRC. Pour TAR, `libarchive` fournit les blocs en flux et Babet vérifie leur ordre, leurs bornes et la taille finale annoncée. Pour un TAR enveloppé dans gzip, Babet valide en plus le CRC et l'ISIZE de chaque membre gzip avec `zlib` avant toute publication. Le temporaire reçoit ensuite ses permissions finales.

Tous les fichiers d'une extraction complète sont préparés avant le début de la publication : une erreur de lecture, de décompression ou de CRC ne publie donc aucun fichier. `extractFile()` conserve lui aussi son fichier sélectionné en temporaire jusqu'à la fin de la lecture vérifiée de toute l'archive. Les temporaires sont supprimés et les nouveaux répertoires encore vides créés **sous** la destination sont retirés. Les composants du chemin racine de destination qui ont dû être créés peuvent rester présents.

La publication est atomique **fichier par fichier** :

- sans écrasement, Babet utilise une création qui échoue si la cible existe ;
- avec écrasement, le temporaire du même répertoire remplace atomiquement le fichier régulier cible.

L'opération entière n'est toutefois pas une transaction multi-fichiers. Une erreur système très tardive pendant la publication ou la mise en permissions des répertoires peut survenir après la publication de fichiers précédents. Ces fichiers peuvent alors rester présents, et un ancien fichier déjà remplacé n'est pas restauré.

Pour l'extraction, la publication atomique protège contre un fichier partiellement visible, mais ne garantit pas la persistance après une panne brutale : les fichiers et répertoires extraits ne sont pas tous synchronisés par `fsync`. `create()` applique la séquence plus forte de synchronisation du fichier et du parent décrite plus haut.

Permissions

Par défaut :

```
fichiers                0644
répertoires finaux      0755
temporaires              0600
répertoires d'archive pendant la préparation 0700
composants créés du chemin destination 0755 (soumis à l'umask)
```

Avec `preserve_permissions = true`, Babet conserve uniquement les neuf bits ordinaires propriétaire/groupe/autres (0777) des modes Unix annoncés par le ZIP ou le TAR. Les bits `setuid`, `setgid` et `sticky` sont systématiquement supprimés.

Les permissions d'un répertoire créé sont appliquées seulement après l'extraction de son contenu, afin qu'un mode final restrictif comme `0000` n'empêche pas l'opération en cours. Les répertoires préexistants ne sont jamais `rechmodés`.

Les UID, GID, ACL, attributs étendus et dates ne sont pas restaurés.

Types d'entrées pris en charge

Type	ZIP/TAR create()	ZIP/TAR list()	ZIP/TAR extract()	ZIP/TAR extractFile()
fichier régulier	pris en charge ; ZIP/gzip/xz acceptent les niveaux 0 à 9, bzip2 accepte 1 à 9, zstd accepte 0 à 19, TAR brut stocke les données	inspecté en ZIP et TAR brut/gzip/xz/bzip2/zstd	pris en charge en ZIP et TAR brut/gzip/xz/bzip2/zstd, sauf TAR sparse	pris en charge en ZIP et TAR brut/gzip/xz/bzip2/zstd, sauf TAR sparse
répertoire	pris en charge avec <code>include_directories = true</code>	inspecté en ZIP et TAR	pris en charge en ZIP et TAR	non sélectionnable
fichier TAR sparse	jamais créé	identifié en TAR	refusé	refusé s'il est sélectionné ; ignoré sinon refusé
symlink	toujours refusé	identifié en ZIP et TAR	toujours refusé	
hard link	non créé	identifié en TAR	refusé	refusé s'il est sélectionné refusé
FIFO, socket, périphérique ou type inconnu	toujours refusé	identifié lorsque le format l'expose	refusé	
entrée chiffrée	non créée	identifiée en ZIP	refusée	refusée
méthode de compression ZIP inconnue	non créée	identifiée	refusée	refusée

Une entrée ZIP qui ressemble à un hard link mais est encodée comme fichier régulier est traitée comme un fichier régulier indépendant ; aucun lien n'est créé.

Erreurs et limites connues

Les causes courantes d'échec sont notamment :

- source absente ou dangereuse, symlink source, nom source non UTF-8, type source non pris en charge, format de création invalide, option réservée au ZIP utilisée pour TAR ou date hors de la plage ZIP ;
- archive de sortie dans la source, parent de destination dangereux ou symlink cible ;
- fichier d'archive absent, illisible, non régulier, non pris en charge ou malformé ;
- données TAR tronquées, données finales qui ne constituent pas un TAR, contenu attaché à une entrée TAR non régulière ou changement détecté entre l'inspection et l'extraction ;
- métadonnées incohérentes ou CRC invalide ;
- limite anti-bombe dépassée ;
- chemin absolu, traversée, doublon ou conflit ;
- entrée chiffrée, symlink ou type non pris en charge ;
- destination existante sans `overwrite = true` ;
- symlink ou type dangereux dans la destination ;
- manque d'espace, permissions insuffisantes ou erreur d'E/S.

Le module est synchrone : l'appel ne revient qu'après la fin de la création, de l'inspection ou de l'extraction. Il n'expose pas encore de progression, d'annulation ou de timeout.

Le fichier d'archive ne doit pas être modifié simultanément. Une troncature ou modification ZIP provoque normalement une erreur de lecture, décompression ou CRC. Le TAR est parcouru en flux par `libarchive` ; les TAR concaténés sont lus jusqu'à leur fin et les données tronquées ou finales qui ne constituent pas un TAR sont refusées. Aucun des deux backends ne fournit d'instantané concurrent. La destination ne doit pas non plus être réorganisée simultanément par un autre processus. Les parcours refusent les symlinks et vérifient les types avant publication, mais un acteur qui renomme activement des répertoires peut faire échouer l'opération et rendre le nettoyage de ses temporaires seulement « meilleur effort ».

La création TAR brute, TAR gzip, TAR xz, TAR bzip2 et TAR zstd est prise en charge. Les flux compressés simples restent refusés jusqu'à la définition et l'audit séparés de leur API et de leur politique de sécurité.

Les archives multi-disques ne constituent pas une cible prise en charge. ZIP64 est accepté lorsque miniz peut le lire, tout en restant soumis aux limites configurées.

argparse — arguments de ligne de commande

`argparse` est un module Lua pur embarqué dans Babet. Il construit un parseur déclaratif pour les flags, les options à valeur et les arguments positionnels :

```
local argparse = require("argparse")
local parser = argparse("mon-script", "Description facultative")
```

Le module ne fait ni `print` ni `os.exit`. Il renvoie l'aide et les erreurs au script, qui décide comment les présenter et quel code de sortie utiliser.

Table des matières du module

- [Chargement et métadonnées](#)
- [API](#)
- [Déclarer des flags et des options](#)
- [Déclarer des positionnels](#)
- [Table opts](#)
 - [Champs communs](#)
 - [Champs réservés aux valeurs](#)
 - [Destination](#)

- Source des arguments
 - Table globale arg
 - Array explicite
- Formes reconnues
- Valeurs par défaut, choix et conversion
- Résultats de parse
 - Succès normal
 - Aide intégrée
 - Erreur utilisateur
 - Erreur de programmation
- Exemple complet
- Texte produit par `get_usage`
- Limites de la version actuelle

Chargement et métadonnées

```
local argparse = require("argparse")
```

```
print(argparse._VERSION)      -- "babet argparse 1.1.0"
print(argparse._DESCRIPTION)  -- description en anglais
```

`require("argparse")` renvoie une table callable. Le constructeur accepte zéro, un ou deux arguments :

```
argparse()                -- programme affiché : "prog"
argparse("outil")         -- nom du programme
argparse("outil", "Description") -- nom et description
```

`prog` et `description` doivent être des chaînes lorsqu'ils sont fournis. Un argument supplémentaire ou un mauvais type lève une erreur Lua : il s'agit d'une erreur de programmation, pas d'une erreur de ligne de commande.

API

Méthode	Résultat	Rôle
<code>parser:flag(spec [, opts])</code>	<code>parser</code>	Déclare un flag sans valeur.
<code>parser:option(spec [, opts])</code>	<code>parser</code>	Déclare une option qui consomme une valeur.
<code>parser:argument(name [, opts])</code>	<code>parser</code>	Déclare un argument positionnel.
<code>parser:parse([src])</code>	<code>res, err</code>	Parse <code>src</code> ou la table globale <code>arg</code> .
<code>parser:get_usage()</code>	chaîne	Génère le texte d'aide.

Les méthodes du builder sont chaînables :

```
local parser = argparse("copie")
  :flag("-v --verbose")
  :option("-o --output")
  :argument("source")
```

Les arités sont strictes. Par exemple, `parser:option("-o", {}, "extra")` et `parser:get_usage(true)` lèvent une erreur Lua.

Déclarer des flags et des options

`spec` est une seule chaîne contenant un ou plusieurs noms séparés par des espaces :

```

parser:flag("-v")
parser:flag("--verbose")
parser:flag("-v --verbose")
parser:option("-o --output")

```

Chaque nom doit :

- commencer par un ou deux tirets ;
- contenir au moins un caractère après les tirets ;
- ne pas contenir = ;
- ne pas être dupliqué dans la même spec ni dans le parseur.

Les noms suivants sont réservés et ne peuvent pas être déclarés :

- - : traité comme un positionnel littéral ;
- -- : termine le parsing des options ;
- -h et --help : aide intégrée.

Un nom à trois tirets comme ---verbose est également refusé.

Toutes les validations sont effectuées avant de modifier le parseur. Si une erreur du builder est interceptée par pcall, aucune option ni aucun alias partiel n'est conservé.

Déclarer des positionnels

```

parser:argument("input")
parser:argument("output", { required = false })

```

Le nom doit être une chaîne non vide, sans espace et sans tiret initial. Les positionnels sont attribués strictement de gauche à droite.

Un positionnel est requis par défaut. Il devient optionnel si :

- required = false est indiqué ; ou
- une valeur default est fournie sans préciser required.

Tous les positionnels requis doivent précéder les positionnels optionnels. Le builder refuse donc ce cas ambigu :

```

parser
  :argument("facultatif", { required = false })
  :argument("requis") -- erreur Lua

```

Table opts

opts doit être une table ou nil. Les champs inconnus et les champs dont le type est incorrect lèvent immédiatement une erreur Lua. Cela permet de détecter un nom mal orthographié comme hlep au lieu de help.

Champs communs

Champ	Type	Effet
help	chaîne	Texte ajouté par get_usage().
default	toute valeur Lua	Valeur utilisée lorsque l'élément est absent.
dest	chaîne non vide	Clé utilisée dans la table résultat.
required	booléen	Exige la présence de l'élément.

Ces quatre champs sont acceptés par flag, option et argument.

Champs réservés aux valeurs

Champ	Type	Effet
choices	array dense de chaînes	Autorise uniquement certaines valeurs brutes.
convert	fonction	Convertit la chaîne brute en une valeur Lua.

choices et convert sont acceptés par option et argument, mais pas par flag, puisqu'un flag ne consomme aucune valeur.

Le builder copie l'array choices. Une modification ultérieure de la table fournie ne change donc pas le parseur.

Destination

Sans dest, la destination est :

1. le premier nom long sans --, s'il existe ;
2. sinon le premier nom court sans son premier -.

```
parser:option("-o --output") -- destination : "output"  
parser:flag("-v")           -- destination : "v"
```

Une destination doit être unique dans tout le parseur. Les destinations help et usage sont réservées au résultat de l'aide intégrée.

```
parser:flag("-q", { dest = "quiet" })
```

Source des arguments

Table globale arg

Sans argument, parse() lit la table globale arg aux indices 1, 2, et ainsi de suite jusqu'au premier trou :

```
local result, err = parser:parse()
```

Les indices 0 et négatifs sont ignorés. parse(nil) est équivalent à parse().

Chaque valeur rencontrée doit être une chaîne. Si arg n'est pas une table, le parseur utilise une liste vide.

Array explicite

Pour les tests ou une source construite par le programme :

```
local result, err = parser:parse({ "-v", "input.txt" })
```

La table doit être un array dense indexé de 1 à n, sans trou, sans clé supplémentaire et composé uniquement de chaînes. Une source invalide est une erreur de parsing et renvoie (nil, err) ; elle ne lève pas d'exception.

Formes reconnues

Une option à valeur accepte :

```
--long valeur  
--long=valeur  
-s valeur  
-s=valeur
```

Une valeur inline vide est valide :

```
parser:parse({ "--output=" }) -- output == ""
```

Les options restent actives après un positionnel tant que -- n'a pas été rencontré :

```
input.txt --verbose
```

-- termine le parsing des options. Tous les tokens suivants deviennent des positionnels, même s'ils commencent par - :

```
parser:parse({ "--", "-5" })
```

Avant --, un token comme -5 est interprété comme un nom d'option. Pour un nombre négatif fourni comme valeur d'une option, aucun séparateur n'est nécessaire :

```
parser:parse({ "--count", "-5", "input.txt" })
```

Le token suivant une option à valeur est toujours consommé littéralement, même s'il ressemble à une option ou s'il vaut -- :

```
parser:parse({ "--output", "--verbose" })  
-- output == "--verbose" ; le flag verbose n'est pas activé
```

Le token isolé - est un positionnel. Les options répétées sont acceptées : la dernière valeur gagne. Un flag répété reste simplement à true.

Les formes suivantes ne sont pas prises en charge :

- options courtes agglomérées, par exemple -abc pour trois flags ;
- valeur courte collée, par exemple -ofile.txt ;
- positionnels variadiques.

Un nom déclaré littéralement -abc reste toutefois une option normale si la spec contient exactement ce nom ; il n'est jamais décomposé.

Valeurs par défaut, choix et conversion

L'ordre appliqué à une valeur fournie par l'utilisateur est :

1. contrôle de choices sur la chaîne brute ;
2. appel de convert avec cette chaîne.

```
parser:option("-n --count", {  
  choices = { "1", "2", "3" },  
  convert = tonumber,  
})
```

Les éléments de choices sont toujours des chaînes. Si la valeur brute ne figure pas dans la liste, parse() renvoie (nil, err) sans appeler le convertisseur.

Un convertisseur peut renvoyer :

- toute valeur non nil, y compris false, pour réussir ;
- nil pour échouer avec le message générique invalid value ;
- nil, "raison" pour fournir une raison ;
- lever une erreur, qui est interceptée et devient conversion error.

Les valeurs default sont des valeurs Lua déjà prêtes : elles ne sont ni contrôlées par choices ni passées à convert. Elles gardent leur type et, pour une table, leur identité. Une option required = true reste requise même si un default est présent.

Pour un flag :

- absent, il vaut default si ce champ est défini, sinon false ;
- présent, il vaut toujours true.

Une option ou un positionnel absent avec un défaut nil n'ajoute pas de clé observable dans la table résultat, conformément au comportement des tables Lua.

Résultats de parse

Succès normal

```
local result, err = parser:parse()
-- result : table
-- err    : nil
```

La table est indexée par les destinations déclarées.

Aide intégrée

Avant --, -h ou --help arrête immédiatement le parsing et renvoie :

```
{
  help = true,
  usage = "...",
}, nil
```

Les options requises, les positionnels manquants et les tokens qui suivent ne sont pas vérifiés. Après --, -h et --help sont de simples positionnels.

L'aide est un succès et n'est pas ajoutée à une table de résultat normale. Les destinations help et usage sont donc réservées pour éviter toute ambiguïté dans le code appelant.

Erreur utilisateur

Les erreurs de ligne de commande renvoient (nil, message) :

```
unknown option '--bad'
option '--output' requires a value
flag '-v' does not take a value
missing required option '--output'
missing required argument 'input'
argument 'mode': invalid choice 'other'
option '--count': invalid value
unexpected argument 'extra'
```

parse() intercepte les erreurs levées par convert. Une valeur utilisateur ne fait donc pas remonter l'exception du convertisseur.

Erreur de programmation

Le constructeur et le builder lèvent une erreur Lua pour :

- une mauvaise arité ;
- un type incorrect ;
- une spec vide ou inutilisable ;
- un nom ou une destination réservé ;
- un nom ou une destination dupliqué ;
- un champ opts inconnu ;
- un array choices invalide ;
- un positionnel requis placé après un positionnel optionnel.

La mauvaise arité de parse est également une erreur de programmation. En revanche, une table src mal formée renvoie (nil, err).

Exemple complet

```
local argparse = require("argparse")

local parser = argparse("convert", "Convertit un fichier.")
:flag("-v --verbose", { help = "Plus de détails" })
:option("-o --output", { default = "out.txt" })
:option("-n --count", {
    choices = { "1", "2", "3" },
    convert = tonumber,
    default = 1,
})
:argument("input")

local args, err = parser:parse()
if args and args.help then
    print(args.usage)
    return
end
if not args then
    io.stderr:write("erreur : ", err, "\n")
    io.stderr:write(parser:get_usage(), "\n")
    os.exit(2)
end

print(args.input, args.output, args.count, args.verbose)
```

Texte produit par get_usage

get_usage() renvoie une chaîne et n'écrit rien. Les titres et le message de l'aide intégrée sont fixes et en anglais : Usage, Arguments, Options et show this help. Il n'existe pas encore d'API de traduction, de largeur de colonne ou de groupes visuels.

Le texte affiche les positionnels et les aides déclarées, mais n'ajoute pas automatiquement les valeurs par défaut, les choix ni le caractère requis des options.

Limites de la version actuelle

Ne sont pas exposés :

- les sous-commandes ;
- les positionnels variadiques (nargs) ;
- les groupes mutuellement exclusifs ;
- les options répétables accumulées dans un array ;
- les options courtes agglomérées ;
- la personnalisation ou la localisation du texte d'aide ;
- une option permettant de désactiver l'aide intégrée.

EXEC - programmes externes, capture et streaming

Babet propose deux API complémentaires pour lancer un programme externe sans shell implicite :

- `babet.exec` envoie éventuellement une entrée complète, capture stdout et stderr en mémoire, puis attend la fin du programme ;
- `babet.spawn` renvoie immédiatement un objet processus pilotable, afin de lire et écrire progressivement sans accumuler automatiquement les flux en RAM.

Le module EXEC couvre :

- la recherche d'un programme dans PATH ;
- la construction sûre de argv, sans parsing shell ;
- un répertoire de travail spécifique au processus enfant ;
- la fusion d'un environnement personnalisé avec celui de Babet ;
- l'envoi d'une chaîne binaire sur l'entrée standard ;
- la capture binaire de stdout et stderr ;
- un timeout couvrant le lancement, les échanges et l'attente finale ;
- une limite de mémoire distincte pour chaque flux de sortie ;
- l'arrêt du groupe de processus enfant en cas de timeout.

Il ne fournit pas de redirection directe vers un fichier ni d'interpréteur de commandes. Pour relier plusieurs programmes sans shell, utilise `babet.pipeline()` ou `babet.spawnPipeline()`.

Table des matières du module

- Conventions essentielles
- Vue d'ensemble de l'API
- Signature et arguments
 - cmd et recherche du programme
 - args et absence de shell
 - Utiliser explicitement un shell
- Options
 - cwd
 - env
 - stdin
 - timeout
 - max_output
- Table de résultat
 - Code de sortie normal
 - Fin par signal
 - Timeout
 - Troncature
- Exemples complets
 - Capture simple
 - Arguments contenant des espaces
 - Traiter un code non nul
 - Capturer stderr
 - Envoyer du texte ou du binaire sur stdin
 - Modifier l'environnement
 - Changer de répertoire
 - Borner le temps
 - Borner la sortie
 - Créer un helper applicatif
- Processus en streaming avec `babet.spawn`
 - Signature et options
 - Méthodes du processus
 - Lire stdout et stderr
 - Écrire sur stdin
 - Attendre, terminer et fermer
 - Exemple de boucle de streaming
 - Limites et prévention des blocages
- Contrat d'erreur
- Processus, signaux et sécurité
- Décisions et limites

Conventions essentielles

Aucun shell implicite

Cette forme :

```
local result, err = babet.exec("printf", { "%s\n", "hello world" })
```

lance directement printf. Chaque élément de la table args devient un slot séparé de argv. Les espaces, \$, *, ;, >, les guillemets et les autres caractères spéciaux ne sont pas interprétés par un shell.

Cette propriété évite la plupart des problèmes de quoting et des injections de commandes.

Une sortie non nulle n'est pas une erreur de lancement

Un programme qui démarre correctement puis renvoie le code 1, 2 ou 127 produit normalement une table de résultat. L'appelant décide si ce code est acceptable.

```
local result, err = babet.exec("grep", { "needle", "data.txt" })
assert(result, err)
```

```
if result.code == 0 then
    print("motif trouvé")
elseif result.code == 1 then
    print("aucune correspondance")
else
    error(result.stderr)
end
```

assert(babet.exec(...)) vérifie seulement que Babet a pu lancer et suivre le programme. Il ne vérifie pas result.code.

Les chaînes de données sont binaires

opts.stdin, result.stdout et result.stderr sont des chaînes Lua binary-safe. Elles peuvent contenir des octets NUL et ne sont pas limitées à du texte UTF-8.

Les chaînes transmises à des API POSIX qui attendent une chaîne C - cmd, les éléments de args, cwd, les noms et valeurs de variables d'environnement - ne peuvent en revanche pas contenir d'octet NUL.

Vue d'ensemble de l'API

```
local result, err = babet.exec(cmd, args?, opts?)
```

Élément	Type	Défaut	Rôle
cmd	string stricte	obligatoire	programme ou chemin à exécuter
args	séquence de strings	aucun argument	éléments argv[1]..argv[n]
opts.cwd	string	répertoire courant hérité	répertoire de travail de l'enfant
opts.env	table string -> string	environnement hérité	variables ajoutées ou remplacées
opts.stdin	string binaire	stdin fermé immédiatement	données envoyées à l'enfant
opts.timeout	nombre fini > 0	aucune limite	budget global en secondes

Élément	Type	Défaut	Rôle
<code>opts.max_output</code>	integer 1..2 Gio	10 Mio	cap par flux capturé

Résultats principaux :

Situation	Résultat
programme lancé et terminé	(result, nil)
programme lancé puis timeout	(result, nil) avec <code>timed_out = true</code>
code de sortie non nul	(result, nil) avec <code>code ~= 0</code>
programme introuvable ou cwd invalide	(nil, err)
args ou opts invalides	(nil, err)
cmd absent ou non string	erreur Lua levée

Signature et arguments

cmd et recherche du programme

cmd doit être une véritable chaîne Lua.

```
local result, err = babet.exec("git", { "status", "--short" })
```

Si cmd ne contient pas de /, Babet demande à la libc de rechercher le programme dans le PATH du processus Babet.

```
local result = assert(babet.exec("python3", { "--version" }))
```

Si cmd contient un /, il est traité comme un chemin direct.

```
local result = assert(babet.exec("/usr/bin/id", { "-u" }))
```

Un chemin relatif contenant / est résolu dans le répertoire de l'enfant, après application éventuelle de `opts.cwd`.

```
local result = assert(babet.exec("./tool", {}, {
    cwd = "/opt/my-app/bin",
}))
```

Nuance sur PATH et opts.env L'environnement final du programme enfant peut contenir un PATH remplacé via `opts.env`. Toutefois, avec l'implémentation Linux actuelle basée sur `execvp`, la recherche initiale de cmd utilise le PATH du processus Babet, pas la valeur PATH fournie dans `opts.env`.

```
local result, err = babet.exec("my-private-tool", {}, {
    env = { PATH = "/opt/private/bin" },
})
```

Cette forme ne garantit donc pas que `/opt/private/bin/my-private-tool` sera trouvé. Utilise un chemin direct :

```
local result, err = babet.exec("/opt/private/bin/my-private-tool", {}, {
    env = { PATH = "/opt/private/bin" },
})
```

Le nouveau PATH sera bien visible **dans** le programme enfant et par les sous-commandes qu'il lancera ensuite.

args et absence de shell

args est facultatif. Il doit être une table représentant une séquence dense 1..n de chaînes.

```
local result = assert(babet.exec("cp", {
    "--",
    "fichier avec espaces.txt",
    "destination/",
}))
```

Babet lit uniquement la partie séquence déterminée par Lua, de 1 à #args. Pour un comportement prévisible :

- utilise des indices consécutifs à partir de 1 ;
- ne laisse aucun trou ;
- ne place pas de métadonnées dans la même table ;
- utilise uniquement des chaînes.

-- Correct

```
local args = { "-c", "print('ok') " }
```

-- Incorrect : valeur numérique

```
local result, err = babet.exec("lua", { "-e", 42 })
```

-- result == nil ; err indique que tous les arguments doivent être strings

argv[0] est automatiquement défini à cmd. Le premier élément de args devient argv[1].

Utiliser explicitement un shell

Les opérateurs |, >, &&, ;, les variables \$NAME et les globs ne sont interprétés que si tu lances volontairement un shell.

```
local result = assert(babet.exec("sh", {
    "-c",
    "printf '%s\\n' \"$HOME\" | sed 's/^#home=#'",
}))
```

Les données incorporées dans une commande shell doivent alors être quotées avec les règles du shell. Pour des valeurs non fiables, préfère les transmettre via args, stdin ou env plutôt que de les concaténer dans le script -c.

Options

Le troisième argument est facultatif. S'il est fourni, il doit être une table. Les champs inconnus sont actuellement ignorés ; une faute de frappe comme timeot = 5 ne déclenche donc pas d'erreur. Utilise exactement les noms ci-dessous.

opts.cwd

Définit le répertoire courant du programme enfant sans modifier celui du processus Babet.

```
local result, err = babet.exec("pwd", {}, {
    cwd = "/var/tmp",
})
assert(result, err)
print(result.stdout) -- /var/tmp
```

Le chdir est réalisé dans l'enfant avant le lancement du programme. Il affecte notamment :

- les chemins relatifs utilisés par le programme ;
- un cmd relatif contenant / ;
- les composantes relatives du PATH utilisé pour la recherche.

Si le dossier n'existe pas ou n'est pas accessible, Babet renvoie (nil, err). Le répertoire courant de Babet reste inchangé.

opts.env

env est une table dont chaque clé et chaque valeur doit être une chaîne. L'environnement est **fusionné** avec celui du processus Babet.

```
local result = assert(babet.exec("sh", {
    "-c", "printf '%s' \"$APP_MODE\"",
}, {
    env = { APP_MODE = "production" },
}))
```

```
assert(result.stdout == "production")
```

Règles :

- une clé fournie remplace la variable héritée de même nom ;
- une clé absente reste héritée ;
- une valeur "" crée ou conserve une variable définie mais vide ;
- il n'existe pas d'option pour supprimer complètement une variable ;
- une clé doit être non vide et ne pas contenir = ;
- clés et valeurs refusent les octets NUL ;
- les types autres que string sont refusés.

Définie mais vide n'est pas la même chose qu'absente :

```
local result = assert(babet.exec("sh", { "-c", [[
    if [ "${TOKEN+x}" = x ] && [ -z "$TOKEN" ]; then
        printf 'defined-empty'
    fi
]] }, {
    env = { TOKEN = "" },
}))
```

```
assert(result.stdout == "defined-empty")
```

L'environnement est préparé dans le parent avant fork, ce qui évite d'appeler setenv dans un enfant issu d'un processus multithreadé.

opts.stdin

stdin est une chaîne envoyée intégralement à l'entrée standard du programme.

```
local result = assert(babet.exec("sha256sum", { "-" }, {
    stdin = "hello world\n",
}))
print(result.stdout)
```

La chaîne est binary-safe :

```
local payload = "A\0B\0C"
local result = assert(babet.exec("cat", {}, { stdin = payload }))
assert(result.stdout == payload)
```

Si stdin est absent ou nil, Babet ferme immédiatement le pipe d'entrée. Le programme enfant observe donc un EOF. Une chaîne vide produit elle aussi un EOF, après l'ouverture normale du pipe.

Babet écrit et lit les trois pipes concurremment pour éviter les deadlocks avec un programme qui produit beaucoup de sortie avant de consommer toute son entrée.

opts.timeout

timeout est un nombre fini strictement positif exprimé en secondes.

```
local result = assert(babet.exec("sleep", { "30" }, {  
    timeout = 0.5,  
}))
```

```
assert(result.timed_out)
```

Le budget commence avant la préparation du lancement et couvre :

1. la préparation de l'environnement ;
2. fork ;
3. le chdir de l'enfant ;
4. l'appel exec ;
5. l'envoi de stdin ;
6. la capture de stdout et stderr ;
7. l'attente finale du processus.

La résolution interne est la milliseconde. Une valeur inférieure à 0.001 se comporte donc comme une échéance quasiment immédiate et ne doit pas être utilisée pour une temporisation précise.

La valeur doit rester inférieure ou égale à INT_MAX millisecondes, soit environ 24,8 jours. Les valeurs nulles, négatives, NaN, infinies ou trop grandes sont refusées avec (nil, err).

À l'expiration après le lancement :

1. Babet envoie SIGTERM au groupe de processus ;
2. il laisse jusqu'à 2 secondes pour une sortie propre ;
3. il envoie SIGKILL si nécessaire ;
4. il draine encore les sorties pendant une fenêtre bornée.

opts.max_output

max_output limite le nombre d'octets conservés **pour chaque flux**.

```
local result = assert(babet.exec("yes", {}, {  
    timeout = 0.2,  
    max_output = 4096,  
}))
```

```
assert(#result.stdout == 4096)  
assert(result.stdout_truncated)
```

Valeurs :

- défaut : 10 * 1024 * 1024, soit 10 Mio ;
- minimum : 1 octet ;
- maximum : 2 * 1024 * 1024 * 1024, soit 2 Gio ;
- type : nombre entier Lua.

La limite est indépendante : jusqu'à 10 Mio de stdout **et** 10 Mio de stderr par défaut.

Une fois la limite atteinte, Babet continue à lire et à jeter le surplus. Cette étape est essentielle : arrêter de lire remplirait le pipe et pourrait bloquer le programme enfant. Les premiers octets sont conservés ; les derniers sont perdus.

Table de résultat

Après un lancement réussi, le premier résultat est toujours une table avec les six champs suivants :

```

{
    stdout = "...",
    stderr = "...",
    code = 0,
    timed_out = false,
    stdout_truncated = false,
    stderr_truncated = false,
}

```

Champ	Type	Signification
stdout	string binaire	premiers octets capturés sur la sortie standard
stderr	string binaire	premiers octets capturés sur la sortie d'erreur
code	integer	code normal, convention signal, ou -1 si indisponible
timed_out	boolean	Babet a déclenché l'arrêt pour dépassement du budget
stdout_truncated	boolean	stdout a dépassé max_output
stderr_truncated	boolean	stderr a dépassé max_output

Code de sortie normal

Quand le programme appelle `exit(n)` ou retourne depuis `main`, `code` vaut le statut POSIX 0..255.

```

local result = assert(babet.exec("sh", { "-c", "exit 7" }))
assert(result.code == 7)
assert(not result.timed_out)

```

Fin par signal

Quand le processus est terminé par un signal, Babet utilise la convention :

`code = 128 + numéro_du_signal`

Par exemple, `SIGKILL` vaut généralement 9 sous Linux, donc `code == 137`. Cette convention est pratique mais ne remplace pas un champ séparé indiquant le signal exact ; aucun tel champ n'existe actuellement.

Timeout

`timed_out == true` signifie que Babet a atteint son échéance et a commencé la procédure d'arrêt. Le champ `code` dépend de la façon dont le programme s'est terminé :

- code volontaire si le programme intercepte `SIGTERM` et sort proprement ;
- souvent 143 pour `SIGTERM` ;
- souvent 137 après `SIGKILL` ;
- exceptionnellement -1 si le statut n'a pas pu être obtenu dans la fenêtre de nettoyage bornée.

Ne teste donc pas uniquement `code == 137` pour reconnaître un timeout : utilise `timed_out`.

La sortie produite avant l'échéance est conservée dans la limite de `max_output`.

Troncature

Les flags `stdout_truncated` et `stderr_truncated` indiquent uniquement que le flux correspondant a dépassé `max_output`.

```

if result.stdout_truncated then
    print("stdout incomplet : augmente max_output ou redirige vers un fichier")
end

```

Dans un cas noyau extrêmement pathologique où un descendant échappe au groupe et maintient un pipe ouvert après SIGKILL, Babet peut abandonner le drainage pour ne pas rester bloqué indéfiniment. Cette cause rare n'est pas représentée par les flags de troncature, qui restent réservés au cap mémoire.

Exemples complets

Capture simple

```

local result, err = babet.exec("git", { "rev-parse", "HEAD" })
assert(result, err)

if result.code ~= 0 then
    error("git a échoué : " .. result.stderr)
end

local commit = result.stdout:gsub("%s+$", "")
print(commit)

```

Arguments contenant des espaces

```

local filename = "rapport juillet 2026.txt"
local result = assert(babet.exec("printf", {
    "nom=<%s>\n",
    filename,
}))

assert(result.stdout == "nom=<rapport juillet 2026.txt>\n")

```

Aucun guillemet supplémentaire n'est nécessaire autour de filename : la frontière d'argument est déjà représentée par la table Lua.

Traiter un code non nul

```

local result, err = babet.exec("diff", {
    "--brief",
    "config.old",
    "config.new",
})
assert(result, err)

if result.code == 0 then
    print("fichiers identiques")
elseif result.code == 1 then
    print("fichiers différents")
else
    error("diff impossible : " .. result.stderr)
end

```

Capturer stderr

```

local result = assert(babet.exec("sh", {
    "-c", "printf out; printf err >&2",
}))

```

```
assert(result.stdout == "out")
assert(result.stderr == "err")
```

Envoyer du texte ou du binaire sur stdin

```
local csv = "name,score\nAlice,18\nBob,15\n"
local result = assert(babet.exec("sort", {}, { stdin = csv }))
print(result.stdout)

local bytes = string.char(0x00, 0x01, 0xFE, 0xFF)
local result = assert(babet.exec("cat", {}, { stdin = bytes }))
assert(result.stdout == bytes)
```

Modifier l'environnement

```
local result = assert(babet.exec("sh", { "-c", [[
    printf 'mode=%s lang=%s' "$APP_MODE" "$LANG"
]] }, {
    env = {
        APP_MODE = "test",
        LANG = "C",
    },
}))

print(result.stdout)
```

Seules les clés fournies sont remplacées. Les autres variables, notamment HOME, restent héritées.

Changer de répertoire

```
local result = assert(babet.exec("find", {
    ".", "-maxdepth", "1", "-type", "f",
}, {
    cwd = "/var/log",
}))

print(result.stdout)
```

Le CWD du script principal n'est pas modifié.

Borner le temps

```
local result, err = babet.exec("sh", {
    "-c", "printf started; sleep 30",
}, {
    timeout = 1.0,
})
assert(result, err)

if result.timed_out then
    print("commande interrompue après le budget")
    print("sortie disponible :", result.stdout)
end
```

Borner la sortie

```
local result = assert(babet.exec("sh", {
    "-c", "yes log-line | head -n 100000",
}, {
    max_output = 64 * 1024,
}))

if result.stdout_truncated then
    print("seuls les 64 premiers Kio ont été conservés")
end
```

Pour des gigaoctets, ne capture pas en mémoire. Lance explicitement un shell et redirige vers un fichier choisi avec soin :

```
local result = assert(babet.exec("sh", {
    "-c", "my-command > /var/tmp/my-command.log 2>&1",
}, {
    timeout = 600,
}))
```

Créer un helper applicatif

```
local function run_checked(cmd, args, opts)
    local result, err = babet.exec(cmd, args, opts)
    if not result then
        return nil, err
    end
    if result.timed_out then
        return nil, string.format("%s: timeout", cmd)
    end
    if result.code ~= 0 then
        local detail = result.stderr ~= "" and result.stderr or result.stdout
        return nil, string.format(
            "%s: code %d: %s",
            cmd,
            result.code,
            detail
        )
    end
    return result
end

local result, err = run_checked("git", { "status", "--short" }, {
    cwd = "/srv/project",
    timeout = 10,
})
assert(result, err)
print(result.stdout)
```

Processus en streaming avec babet.spawn

`babet.spawn` utilise le même moteur de lancement sécurisé que `babet.exec`, mais ne capture pas automatiquement les flux et n'attend pas la fin du programme. Il renvoie un userdata représentant le processus enfant :

```
local process, err = babet.spawn("yt-dlp", {
    "--newline",
})
```

```

    "https://example.invalid/video",
})
assert(process, err)

```

Le processus dispose de trois pipes non bloquants : stdin, stdout et stderr. Le script décide quand lire, écrire, attendre ou arrêter le groupe de processus. Cette API est adaptée aux programmes longs, aux sorties volumineuses et à l’affichage de progression en temps réel.

Signature et options

```
local process, err = babet.spawn(command, args?, opts?)
```

Élément	Type	Défaut	Rôle
command	string stricte	obligatoire	programme ou chemin à lancer
args	array dense de strings	aucun argument	argv[1]..argv[n]
opts.cwd	string	répertoire courant hérité	répertoire de l’enfant
opts.env	table string -> string	environnement hérité	variables ajoutées/remplacées
opts.launch_timeout	nombre fini > 0	attente illimitée	borne la phase chdir + exec

Contrairement à `babet.exec`, les options inconnues sont refusées. `spawn` n’accepte pas `stdin`, `timeout` ni `max_output` : `stdin` est écrit avec `process.write()`, la durée de vie est pilotée avec `wait()/terminate()`, et les sorties sont lues progressivement.

`launch_timeout` couvre uniquement la phase de lancement avant que le nouvel exécutable soit établi. Il ne limite pas la durée totale du programme.

La recherche dans `PATH`, le traitement des chemins contenant `/`, la fusion de `env` et l’absence de `shell` implicite suivent les mêmes règles que `exec`.

Méthodes du processus

Méthode	Retour	Rôle
<code>process:read_stdout([max_bytes(data, nil) ou (nil, reason) [, timeout]])</code>		lit au plus <code>max_bytes</code> sur stdout
<code>process:read_stderr([max_bytes(data, nil) ou (nil, reason) [, timeout]])</code>		lit au plus <code>max_bytes</code> sur stderr
<code>process:write(data [, timeout])</code>	(bytes_written, nil) ou (nil, reason)	écrit une partie de <code>data</code> sur stdin
<code>process:close_stdin()</code>	(true, nil)	ferme stdin, de façon idempotente
<code>process:is_running()</code>	boolean ou (nil, err)	vérifie l’état sans bloquer
<code>process:pid()</code>	integer	renvoie le PID attribué
<code>process:wait([timeout])</code>	(result, nil) ou (nil, reason)	attend la fin sans tuer au timeout
<code>process:terminate([grace_period])</code>	(result, nil) ou (nil, err)	envoie SIGTERM puis SIGKILL si nécessaire
<code>process:kill()</code>	(result, nil) ou (nil, err)	envoie SIGKILL au groupe
<code>process:close()</code>	(true, nil)	nettoie les pipes et un enfant encore actif

Les raisons courtes "timeout", "closed" et "interrupted" sont typées, comme pour les sockets. Les autres erreurs portent le préfixe process: ou spawn:.

La table renvoyée par wait, terminate et kill contient :

```
{
    code = 0,          -- code normal, ou 128 + signal
    exited = true,    -- fin normale via exit/_exit
    signaled = false,  -- fin causée par un signal
    signal = nil,      -- numéro du signal si signaled == true
}
```

Un second appel à wait() après la fin renvoie le même résultat : le statut est mis en cache après le waitpid initial.

Lire stdout et stderr

Les lectures sont binary-safe. Par défaut, elles lisent au plus 64 Kio et sont non bloquantes (timeout = 0). La taille maximale par appel est 16 Mio.

```
local chunk, err = process:read_stdout(64 * 1024, 0.25)
```

```
if chunk then
    io.write(chunk)
elseif err == "timeout" then
    -- rien de disponible pendant 250 ms
elseif err == "closed" then
    -- EOF définitif sur stdout
else
    error(err)
end
```

"closed" indique l'EOF du flux concerné, pas nécessairement la fin du processus. Un enfant peut fermer stdout puis continuer à travailler.

Il faut drainer stdout **et** stderr. Lire un seul flux tandis que l'autre se remplit peut bloquer le programme enfant lorsque le pipe noyau devient plein.

Écrire sur stdin

write accepte une chaîne Lua binaire, y compris des octets NUL. Le pipe est non bloquant : même après une attente réussie, l'appel peut n'écrire qu'une partie de la chaîne. La valeur renvoyée doit donc être utilisée pour reprendre à l'octet suivant.

```
local function write_all(process, data)
    local offset = 1
    while offset <= #data do
        local written, err = process:write(data:sub(offset), 1)
        assert(written, err)
        offset = offset + written
    end
end
```

```
write_all(process, payload)
assert(process:close_stdin())
```

Une chaîne vide renvoie (0, nil). Après close_stdin, write renvoie (nil, "closed"). Fermer stdin est souvent indispensable pour que des outils comme cat, sort ou ffmpeg sachent que l'entrée est terminée.

Attendre, terminer et fermer

`wait()` sans argument attend la fin du processus. `wait(timeout)` rend `(nil, "timeout")` si le délai expire, sans envoyer de signal et sans rendre l'objet inutilisable.

```
local result, err = process:wait(0)
if not result and err == "timeout" then
    -- le processus tourne encore
end
```

`terminate(grace_period)` envoie `SIGTERM` à tout le groupe. Si le groupe ne se termine pas avant la fin de la période de grâce, Babet envoie `SIGKILL`. La grâce par défaut est de deux secondes. `kill()` envoie directement `SIGKILL`.

`close()` est idempotent. Si l'enfant tourne encore, il est terminé et réapé de façon bornée, puis les trois pipes sont fermés. Le garbage collector et la métaméthode Lua `__close` effectuent le même nettoyage :

```
local process <close> = assert(babet.spawn("long-job"))
-- nettoyage automatique en quittant la portée
```

Une fermeture explicite reste recommandée, car le moment d'exécution du garbage collector n'est pas déterministe.

Exemple de boucle de streaming

```
local process, err = babet.spawn("yt-dlp", {
    "--newline",
    "-f", "bestvideo+bestaudio/best",
    url,
})
assert(process, err)

local stdout_open, stderr_open = true, true

while stdout_open or stderr_open do
    if stdout_open then
        local data, read_err = process:read_stdout(64 * 1024, 0.1)
        if data then
            io.write(data)
            io.flush()
        elseif read_err == "closed" then
            stdout_open = false
        elseif read_err ~= "timeout" then
            error(read_err)
        end
    end

    if stderr_open then
        local data, read_err = process:read_stderr(64 * 1024, 0)
        if data then
            io.stderr:write(data)
            io.stderr:flush()
        elseif read_err == "closed" then
            stderr_open = false
        elseif read_err ~= "timeout" then
            error(read_err)
        end
    end
end
```

```
end
```

```
local result, wait_err = process:wait(5)
assert(result, wait_err)
process:close()

if result.code ~= 0 then
    error("yt-dlp a quitté avec le code " .. result.code)
end
```

Limites et prévention des blocages

- `wait()` ne draine pas les sorties. Appeler `wait()` immédiatement sur un enfant très bavard peut bloquer si ses pipes deviennent pleins. Draine les deux flux pendant son exécution, ou utilise `babet.exec` pour une petite sortie capturée automatiquement.
- Écrire une grosse entrée à un programme qui réémet simultanément beaucoup de données exige d'alterner écriture et lecture. Un unique `write_all` avant toute lecture peut remplir les pipes dans les deux sens.
- Un objet issu de `babet.spawn()` ne peut pas être raccordé après coup à un autre objet processus. Pour un pipeline linéaire, utilise directement `babet.spawnPipeline()`.
- `stdout` et `stderr` sont toujours séparés. Babet ne les fusionne pas.
- `close()` et `terminate()` ciblent le groupe de processus créé au lancement. Un descendant qui change volontairement de groupe ou de session peut échapper à ce contrôle, comme avec `babet.exec`.

Contrat d'erreur

Erreurs Lua levées

Seule la signature du premier argument utilise une erreur Lua :

```
babet.exec()    -- lève : commande manquante
babet.exec(42)  -- lève : cmd doit être une string
```

Utilise `pcall` uniquement si une mauvaise signature peut provenir de données non maîtrisées.

Erreurs renvoyées (`nil`, `err`)

Les validations suivantes ne lèvent pas ; elles renvoient (`nil`, `err`) :

- `args` n'est ni une table ni `nil` ;
- un élément de `args` n'est pas une string ;
- `opts` n'est ni une table ni `nil` ;
- `cwd`, `stdin`, `timeout`, `max_output` ou `env` ont un mauvais type ;
- une clé d'environnement est vide, contient `=` ou un NUL ;
- une valeur d'environnement contient un NUL ;
- `timeout` nul, négatif, non fini ou trop grand ;
- `max_output` nul, négatif, fractionnaire ou supérieur à 2 Gio ;
- programme introuvable ou non exécutable ;
- `cwd` invalide ;
- échec de pipe, fork, poll, `waitpid` ou autre erreur interne système.

```
local result, err = babet.exec("echo", "hello")
assert(result == nil)
print(err) -- args must be a table
```

Situations renvoyant une table

Ces situations ne sont pas des erreurs de l'API :

- le programme renvoie un code non nul ;
- le programme écrit sur `stderr` ;
- le timeout expire après ou pendant le lancement ;
- la sortie dépasse `max_output`.

Processus, signaux et sécurité

Groupe de processus

L'enfant crée son propre groupe. En cas de timeout, Babet signale le groupe et pas seulement le PID direct. Cela couvre normalement les descendants lancés par un shell ou par le programme.

Un descendant qui appelle volontairement `setsid` ou change de groupe peut échapper à cette stratégie. `babet.exec` n'est pas un sandbox et ne remplace pas `systemd`, les `cgroups`, les namespaces ou les limites de ressources.

Interaction avec `babet.signal`

`babet.exec` n'est pas signal-aware. Un signal géré reçu pendant l'appel ne fait pas renvoyer (`nil`, "interrupted"). Le handler C marque le signal comme en attente ; le callback Lua ne pourra être dispatché qu'après le retour dans la VM Lua, généralement via le hook d'instructions du module `SIGNAL`.

Utilise `opts.timeout` pour borner une commande longue. Pour interrompre une commande à la demande, conçois un protocole externe ou lance-la via un service supervisé.

Mémoire

`stdin` est conservé en mémoire par le script, et `stdout/stderr` sont capturés en mémoire dans les limites choisies. Le défaut autorise environ 20 Mio de capture au total, plus les allocations du programme et des chaînes Lua.

Décisions et limites

- Linux/POSIX uniquement : l'implémentation utilise `fork`, `execvpe`, `poll`, les groupes de processus et les signaux.
- Aucun shell implicite.
- Un appel `exec` reste une commande unique ; les pipelines natifs utilisent `babet.pipeline()` ou `babet.spawnPipeline()`.
- Un seul bloc `stdin`, pas de producteur streaming.
- Capture complète retournée à la fin, pas de callback ligne par ligne.
- Pas de redirection native de `stdin/stdout/stderr` vers des fichiers ou `fd`.
- Pas de champ séparé `signal` dans la table de résultat.
- Pas de modification/unset complet de l'environnement : `env` fusionne et une chaîne vide reste une variable définie.
- Les champs inconnus de `opts` sont ignorés.
- La recherche initiale dans `PATH` suit actuellement l'environnement du processus Babet, même si `opts.env.PATH` est remplacé.
- Les callbacks de `babet.signal` ne sont pas dispatchés pendant l'appel.

Pour les règles de sécurité générales, consulte [Sécurité](#).

Pipelines de processus

Babet fournit deux API complémentaires pour relier plusieurs programmes par des pipes POSIX, sans passer par un shell :

- `babet.pipeline()` exécute le pipeline jusqu'à sa fin et capture les flux ;
- `babet.spawnPipeline()` rend immédiatement un objet pilotable en streaming.

Dans les deux cas, `stdout` d'une étape est connecté directement à `stdin` de la suivante. Le parent n'intercepte que `stdin` de la première étape, `stdout` de la dernière et `stderr` de chaque étape séparément.

Syntaxe commune des commandes

```
local commands = {
  { "printf", { "orange\npomme\norange\n" } },
  { "grep", { "orange" } },
  { "sort", { "-u" } },
}
```

`commands` doit être un tableau dense de 2 à 32 étapes. Chaque étape prend l'une des formes suivantes :

```
{ command }
{ command, args }
{ command, args, opts }
```

Pour fournir `opts` sans argument, utilisez explicitement `{ command, {}, opts }` ; les tableaux troués sont refusés.

- `command` est une chaîne non vide sans octet NUL ;
- `args` est un tableau dense de chaînes sans octet NUL ;
- `opts.cwd` définit le répertoire de travail de l'étape ;
- `opts.env` ajoute ou remplace des variables d'environnement pour l'étape.

Aucune chaîne n'est interprétée par un shell : `jokers`, redirections, variables, guillemets et opérateurs comme `|`, `&&` ou `>` n'ont aucun sens spécial.

Les options globales `cwd` et `env` servent de valeurs par défaut. Un `cwd` local remplace le `cwd` global. L'environnement local est fusionné après l'environnement global et gagne en cas de nom identique. Toutes les commandes sont validées avant le premier `fork()`.

Comme pour `exec()` et `spawn()`, lorsqu'une commande ne contient pas `/`, sa recherche initiale utilise le `PATH` du processus Babet, pas un `PATH` remplacé dans `opts.env` ou dans l'environnement local d'une étape. Pour lancer un outil présent uniquement dans ce nouveau `PATH`, utilisez son chemin explicite ou modifiez le `PATH` de Babet avant le lancement.

babet.pipeline() — exécution complète

Signature

```
result, err = babet.pipeline(commands [, opts])
```

Exemple :

```
local result, err = babet.pipeline({
  { "printf", { "orange\npomme\norange\n" } },
  { "grep", { "orange" } },
  { "sort", { "-u" } },
}, {
  timeout = 10,
  max_output = 16 * 1024 * 1024,
})
```

```
assert(result, err)
print(result.stdout)
```

Options

Option	Type	Description
cwd	chaîne	Répertoire de travail par défaut de toutes les étapes.
env	table chaîne vers chaîne	Variables fusionnées avec l'environnement de Babet.
stdin	chaîne binaire	Données écrites progressivement dans la première étape, puis stdin est fermé.
timeout	nombre fini strictement positif, au plus 10 ¹²	Délai global en secondes, lancement, E/S et attente finale compris.
max_output	entier de 1 à 2 Gio	Plafond distinct pour stdout final et chaque stderr ; défaut : 10 Mio.

Les clés inconnues sont refusées. stdin, stdout et stderr sont binaires et peuvent contenir des octets NUL. Le timeout a une résolution interne d'une milliseconde ; les valeurs supérieures à 10¹² secondes sont refusées.

Résultat

Après le lancement effectif de toutes les étapes, une terminaison non nulle ou un signal ne constitue pas une erreur d'API. Babet renvoie :

```
{
  stdout = "...",
  stderr = { "stderr étape 1", "stderr étape 2", ... },
  stages = {
    {
      launched = true,
      code = 0,
      exited = true,
      signaled = false,
      signal = nil,
    },
    -- ...
  },
  code = 0,
  all_succeeded = true,
  failed_index = nil,
  timed_out = false,
  stdout_truncated = false,
  stderr_truncated = { false, false, ... },
}
```

code est celui de la dernière étape, comme pour un pipeline Unix. Il vaut le code de sortie normal ou 128 + numéro_du_signal. Dans le cas pathologique où un statut reste indisponible après un nettoyage borné consécutif à un timeout, il vaut -1 pour l'étape concernée. all_succeeded n'est vrai que si toutes les étapes ont réussi. failed_index indique la première étape dont le code n'est pas zéro, même si la dernière réussit.

Une étape intermédiaire non nulle n'arrête pas artificiellement les autres. Les pipes conservent leur comportement POSIX normal ; un producteur amont peut notamment recevoir SIGPIPE si le consommateur ferme son entrée.

Timeout, gros volumes et troncature

Le timeout de `pipeline()` couvre le lancement, l'écriture de `stdin`, le drainage de `stdout` et `stderr` et l'attente finale. S'il expire avant que toutes les étapes aient terminé `chdir` et `exec`, l'appel renvoie `nil`, `err` avec le message "pipeline: launch timed out" après rollback des étapes déjà créées. Après un lancement complet, l'expiration produit une table de résultat avec `timed_out == true` : Babet ferme `stdin`, envoie `SIGTERM` aux groupes de toutes les étapes, puis `SIGKILL` aux survivants et récupère les enfants directs.

Les descripteurs sont non bloquants et pilotés avec `poll()`. `stdin`, `stdout` final et tous les `stderr` progressent simultanément, ce qui évite les deadlocks classiques avec de gros volumes. Après `max_output`, les octets supplémentaires sont jetés mais le flux continue d'être drainé ; le drapeau de troncature correspondant est alors vrai.

babet.spawnPipeline() — streaming

Signature

```
pipeline, err = babet.spawnPipeline(commands [, opts])
```

Options globales :

Option	Type	Description
<code>cwd</code>	chaîne	Répertoire de travail par défaut de toutes les étapes.
<code>env</code>	table chaîne vers chaîne	Variables fusionnées avec l'environnement de Babet.
<code>launch_timeout</code>	nombre fini strictement positif	Délai couvrant uniquement la création et le lancement complet des étapes, au plus <code>INT_MAX</code> millisecondes (environ 24,8 jours).

`spawnPipeline()` n'accepte ni `stdin`, ni `timeout`, ni `max_output` : le script écrit et lit progressivement et choisit lui-même ses limites. Un succès renvoie un `userdata` et `nil` ; une erreur de validation ou de lancement renvoie `nil`, `err` après nettoyage des étapes déjà créées.

```
local pipeline, err = babet.spawnPipeline({
  { "cat" },
  { "tr", { "a-z", "A-Z" } },
})
assert(pipeline, err)

local written, write_err = pipeline:write("bonjour\n")
assert(written, write_err)
assert(pipeline:close_stdin())

local chunks = {}
while true do
  local data, read_err = pipeline:read_stdout(64 * 1024, 1)
  if data then
    chunks[#chunks + 1] = data
  elseif read_err == "closed" then
    break
  elseif read_err ~= "timeout" then
    error(read_err)
```

```

    end
end

local status, wait_err = pipeline:wait(5)
assert(status, wait_err)
assert(status.code == 0)
print(table.concat(chunks))

```

Méthodes de lecture

```

data, err = pipeline:read_stdout([max_bytes [, timeout]])
data, err = pipeline:read_stderr(stage [, max_bytes [, timeout]])

```

- max_bytes vaut 64 Kio par défaut et doit être compris entre 1 et 16 Mio ;
- timeout est exprimé en secondes, vaut zéro par défaut, doit être fini et positif ou nul, et ne peut pas dépasser INT_MAX millisecondes, soit environ 24,8 jours ;
- stage est un indice de 1 au nombre d'étapes ;
- une arité invalide, un indice invalide ou un max_bytes invalide lève une erreur Lua, comme les méthodes de `babet.spawn()` ;
- un timeout invalide renvoie nil, err ;
- un succès renvoie une chaîne binaire non vide et nil ;
- aucune donnée immédiatement disponible renvoie nil, "timeout" ;
- la fin du flux, ou une lecture après sa fermeture, renvoie nil, "closed".

Chaque appel lit au plus un bloc. Il n'existe aucune capture cachée : le script est responsable de conserver, limiter ou jeter les données reçues. Les stderr restent indépendants ; il faut donc drainer ceux susceptibles de produire assez de données pour remplir leur pipe.

Écriture et fermeture de stdin

```

count, err = pipeline:write(data [, timeout])
ok, err = pipeline:close_stdin()

```

`write()` accepte une chaîne binaire. Il peut n'écrire qu'une partie de la chaîne ; count indique le nombre d'octets réellement transmis. Une chaîne vide renvoie 0, nil. Après timeout, l'objet reste réutilisable. Une entrée fermée par le script ou par le processus aval renvoie nil, "closed" sans livrer SIGPIPE au processus Babet.

`close_stdin()` ferme l'entrée de la première étape et est idempotente. Elle doit être appelée lorsque plus aucune donnée ne sera écrite, sinon un programme attendant EOF peut ne jamais terminer.

Exemple d'écriture complète :

```

local offset = 1
while offset <= #data do
    local count, err = pipeline:write(data:sub(offset), 1)
    if count then
        offset = offset + count
    elseif err ~= "timeout" then
        error(err)
    end
    -- Pendant un gros transfert, drainer aussi stdout et les stderr.
end
assert(pipeline:close_stdin())

```

État et identifiants

```

running = pipeline:is_running([stage])
pids = pipeline:pids()

```

Sans indice, `is_running()` est vrai tant qu'au moins une étape directe n'a pas été récupérée. Avec un indice, il décrit uniquement cette étape. L'appel interroge les statuts sans blocage et ne consomme pas le résultat final : `wait()` reste utilisable ensuite.

`pids()` renvoie les PID directs dans l'ordre des étapes. Ils sont fournis pour le diagnostic ; le script ne doit pas supposer qu'ils restent valides après la fin du pipeline.

Attente et résultat final

```
status, err = pipeline:wait([timeout])
```

Sans timeout, `wait()` attend tous les enfants directs. Avec un timeout fini et positif ou nul, `nil`, "timeout" laisse le pipeline intact et réutilisable. Une fois tous les statuts acquis, les appels suivants sont idempotents et renvoient le même résultat :

```
{
  stages = {
    { launched = true, code = 0, exited = true,
      signaled = false, signal = nil },
    -- ...
  },
  code = 0,
  all_succeeded = true,
  failed_index = nil,
}
```

`wait()` **ne draine pas** stdout ou stderr. Un pipeline qui remplit un pipe peut donc rester bloqué avant de terminer. Il faut lire les flux pendant l'exécution, exactement comme avec `babet.spawn()`.

Arrêt explicite

```
status, err = pipeline:terminate([grace_period])
status, err = pipeline:kill()
```

`terminate()` envoie SIGTERM à chaque groupe de processus, laisse `grace_period` secondes — 2 secondes par défaut — aux processus pour terminer, puis envoie SIGKILL aux survivants. La période de grâce doit tenir dans INT_MAX millisecondes, soit environ 24,8 jours. `kill()` envoie directement SIGKILL. Les deux méthodes ferment stdin et renvoient la même table de statuts que `wait()` lorsque tous les enfants directs ont été récupérés.

Chaque étape possède son propre groupe. Avant de récupérer le processus direct, Babet supprime les descendants encore présents dans ce groupe, y compris lors d'une terminaison normale où la commande a quitté sans attendre un enfant lancé en arrière-plan. Le PID du leader reste ainsi réservé jusqu'au signal et ne peut pas être recyclé vers un processus étranger.

Pour `terminate()`, les leaders terminés restent volontairement non récupérés pendant toute la période de grâce : les descendants conservent ce délai pour réagir à SIGTERM, puis le groupe reçoit SIGKILL avant la récupération finale. Une étape qui crée volontairement une nouvelle session ou un nouveau groupe de processus échappe à cette garantie POSIX.

close(), GC et Lua <close>

```
ok, err = pipeline:close()
```

`close()` est idempotente. Elle ferme les descripteurs, tente SIGTERM, utilise SIGKILL si nécessaire et récupère les enfants directs. Après fermeture, les lectures et écritures renvoient `nil`, "closed", et `is_running()` renvoie `false`. Un statut déjà entièrement acquis reste accessible par `wait()`.

Le même nettoyage est appelé par `__gc` et `__close`. L'usage recommandé est :

```
local pipeline <close> = assert(babet.spawnPipeline({
  { "producteur" },
  { "consommateur" },
}))
```

Le GC constitue un filet de sécurité, pas un mécanisme de synchronisation : utilisez `wait()`, `terminate()` ou `close()` explicitement lorsque le moment du nettoyage compte.

Erreurs de lancement et réutilisation

Les erreurs de validation, de pipe, de fork, de `chdir` ou d'`exec`, ainsi qu'un `launch_timeout`, renvoient `nil`, `err`. Si une étape échoue pendant le lancement, toutes celles déjà créées sont arrêtées et récupérées ; aucun objet partiellement valide n'est exposé.

Les erreurs non destructives "timeout" et "interrupted" autorisent un nouvel appel. "closed" signifie que le flux ou l'objet concerné ne peut plus servir pour cette opération. Une erreur système plus grave est renvoyée sous forme de texte et doit être traitée comme telle.

FS — fichiers, dossiers, chemins, recherche, copie, permissions et sommes de contrôle

Les fonctions de ce chapitre sont exposées directement dans la table globale `babet`. Il n'existe pas de sous-table `babet.fs` : ce namespace plat est historique et fait désormais partie de l'API stable.

Le module FS couvre :

- la vérification de l'existence et du type d'un chemin ;
- la création et la suppression de fichiers et de dossiers ;
- le changement de répertoire courant ;
- la manipulation lexicale des chemins ;
- le listing, l'itération et la recherche récursive ;
- la copie et le déplacement d'arborescences ;
- les liens symboliques ;
- les permissions, propriétaires et groupes Unix ;
- les sommes de contrôle CRC-32, MD5, SHA et BLAKE2.

Table des matières du module

- Conventions générales
- Vue d'ensemble de l'API
- Existence, type et taille
 - `fileExists`
 - `isFile / isfile`
 - `isDir / isdir`
 - `fileSize`
- Répertoire courant
 - `currentDir`
 - `chdir`
- Manipulation lexicale des chemins
 - `getBasename`
 - `getFilename`
 - `getExtension`
 - `getPath`
 - `joinPath`
- Créer, supprimer, renommer et lier
 - `touch`
 - `mkdir`

- remove
- rmdir
- rmdirAll
- rename
- link
- Lister, itérer et rechercher
 - listFiles
 - createFileIterator
 - find
- Copier et déplacer
 - copy
 - copyTree
 - moveTree
- Permissions et attributs Unix
 - getMode / setMode
 - getAttributes / setAttributes
 - symlinkAttr / symlinkattr
- Sommes de contrôle de fichiers
- CRC-32 en mémoire
- Contrat d'erreur
- Décisions et limites

Conventions générales

Chemins

Les chemins sont des chaînes Lua. Ils peuvent être relatifs ou absolus.

Babet ne réalise pas automatiquement :

- l'expansion de ~ ;
- l'expansion de variables comme \$HOME ;
- l'expansion des globs *.lua ;
- la normalisation générale de . et .. dans les fonctions lexicales.

Par exemple :

```
local home = assert(babet.env("HOME"))
local config = babet.joinPath(home, ".config", "mon-app")
```

Un octet NUL dans un chemin est toujours refusé. Selon la fonction, une mauvaise signature lève une erreur Lua, tandis qu'une erreur du système de fichiers est généralement renvoyée sous la forme (nil, err).

Liens symboliques

Il n'existe pas une règle unique valable pour toutes les fonctions :

- isFile, isDir, fileSize, getMode, getAttributes et les checksums suivent un lien valide vers sa cible ;
- remove supprime le lien lui-même, y compris s'il est cassé ;
- rmdir et rmdirAll refusent une racine symbolique ;
- symlinkAttr modifie le propriétaire et le groupe du lien lui-même ;
- listFiles, createFileIterator et find ne descendent pas dans un lien symbolique vers un dossier ;
- copyTree et moveTree refusent une racine source ou destination symbolique, mais gèrent les liens présents à l'intérieur de l'arbre.

Chaque fonction détaille ci-dessous son comportement exact.

Ordre des résultats

`listFiles`, `createFileIterator` et `find` suivent l'ordre fourni par le système de fichiers. Cet ordre n'est pas trié et ne doit pas être considéré comme stable.

Trie explicitement le résultat lorsque l'ordre compte :

```
local files = assert(babet.listFiles("documents", true))
table.sort(files)
```

Valeurs de retour

La convention la plus fréquente est :

```
local value, err = fonction(...)
if not value then
    print(err)
end
```

Une action réussie renvoie généralement `(true, nil)`. Quelques fonctions pures ou méthodes d'itérateur ont une forme différente ; ces exceptions sont signalées dans leur section.

Vue d'ensemble de l'API

Vérifier et inspecter

Fonction	Résultat en cas de succès
<code>babet.fileExists(path)</code>	<code>(boolean, nil)</code> — vrai uniquement pour un fichier régulier
<code>babet.isFile(path)</code>	<code>(boolean, nil)</code> — même prédicat fonctionnel que <code>fileExists</code>
<code>babet.isDir(path)</code>	<code>(boolean, nil)</code> — vrai pour un dossier
<code>babet.fileSize(path)</code>	<code>(integer, nil)</code> — taille d'un fichier régulier en octets
<code>babet.currentDir()</code>	<code>(string, nil)</code> — répertoire courant
<code>babet.getMode(path)</code>	<code>(integer, nil)</code> — mode Unix <code>0000..0777</code>
<code>babet.getAttributes(path)</code>	<code>(table, nil)</code> — <code>{mode, owner, group}</code>

Manipuler des chemins

Fonction	Résultat en cas de succès
<code>babet.getBasename(path)</code>	<code>(string, nil)</code>
<code>babet.getFilename(path)</code>	<code>(string, nil)</code>
<code>babet.getExtension(path)</code>	<code>(string, nil)</code>
<code>babet.getPath(path)</code>	<code>(string, nil)</code>
<code>babet.joinPath(a, b, ...)</code>	<code>string</code>
<code>babet.joinPath({a, b, ...})</code>	<code>string</code>

Agir sur le système de fichiers

Fonction	Résultat en cas de succès
<code>babet.touch(path)</code>	<code>(true, nil)</code>
<code>babet.mkdir(path)</code>	<code>(true, nil)</code> — récursif et idempotent
<code>babet.remove(path)</code>	<code>(true, nil)</code> — fichier régulier ou symlink

Fonction	Résultat en cas de succès
<code>babet.rmdir(path)</code>	<code>(true, nil)</code> — dossier réel et vide
<code>babet.rmdirAll(path)</code>	<code>(true, nil)</code> — dossier réel, suppression récursive
<code>babet.rename(source, destination)</code>	<code>(true, nil)</code>
<code>babet.link(target, linkpath)</code>	<code>(true, nil)</code> — crée un symlink
<code>babet.chdir(path)</code>	<code>(true, nil)</code>

Lister et rechercher

Fonction	Résultat en cas de succès
<code>babet.listFiles(path, recursive?)</code>	<code>(table, nil)</code>
<code>babet.createFileIterator(path, recursive?)</code>	<code>(iterator, nil)</code>
<code>babet.find(path, opts?)</code>	<code>(table, nil)</code>

Copier et déplacer

Fonction	Résultat en cas de succès
<code>babet.copy(source, destination)</code>	<code>(true, nil)</code> — un fichier
<code>babet.copyTree(source, destination, continue_on_error?)</code>	<code>(true, nil)</code> — un arbre
<code>babet.moveTree(source, destination)</code>	<code>(true, nil)</code> — un arbre

Modifier les attributs

Fonction	Résultat en cas de succès
<code>babet.setMode(path, mode)</code>	<code>(true, nil)</code>
<code>babet.setAttributes(path, uid, gid, mode?)</code>	<code>(true, nil)</code>
<code>babet.symlinkAttr(path, uid, gid)</code>	<code>(true, nil)</code>

Existence, type et taille

`babet.fileExists(path)`

Vérifie si `path` désigne un **fichier régulier**.

```
local exists, err = babet.fileExists("rapport.txt")
```

Résultats :

- fichier régulier : `(true, nil)` ;
- dossier, lien cassé ou chemin absent : `(false, nil)` ;
- lien valide vers un fichier régulier : `(true, nil)` ;
- véritable erreur d'inspection : `(nil, err)`.

Malgré son nom historique, `fileExists` ne répond pas « vrai » pour tous les types de chemins. Pour un dossier, utilise `isDir`.

```
local is_file = assert(babet.fileExists("archive.tar"))
local is_dir  = assert(babet.isDir("sauvegardes"))
```

babel.isFile(path) et babel.isfile(path)

isFile est le nom canonique. isfile est un alias déprécié conservé pour compatibilité.

La fonction teste le même type de chemin que fileExists : un fichier régulier, en suivant un éventuel symlink valide.

```
local regular, err = babel.isFile("programme.lua")
if err then
    print("Inspection impossible :", err)
elseif regular then
    print("C'est un fichier régulier")
end
```

Un chemin inexistant renvoie (false, nil) et non une erreur.

babel.isDir(path) et babel.isdir(path)

isDir est le nom canonique. isdir est un alias déprécié.

```
local directory, err = babel.isDir("assets")
```

Résultats :

- dossier réel : (true, nil) ;
- lien valide vers un dossier : (true, nil) ;
- fichier, lien cassé ou chemin absent : (false, nil) ;
- erreur d'accès ou d'inspection : (nil, err).

babel.fileSize(path)

Renvoie la taille d'un fichier régulier en octets.

```
local size, err = babel.fileSize("video.mp4")
if not size then
    print("Taille inaccessible :", err)
else
    print(size, "octets")
end
```

La fonction suit un symlink valide vers un fichier. Elle refuse les dossiers et les autres types de fichiers.

```
local size, err = babel.fileSize("documents")
-- size == nil ; err indique que le chemin n'est pas un fichier régulier
```

Répertoire courant

babel.currentDir()

Renvoie le répertoire de travail courant du processus.

```
local cwd, err = babel.currentDir()
assert(cwd, err)
print(cwd)
```

Le résultat est le chemin courant tel que fourni par le système de fichiers, généralement absolu.

babel.chdir(path)

Change le répertoire courant de tout le processus.

```

local previous = assert(babet.currentDir())
assert(babet.chdir("/tmp"))
print(assert(babet.currentDir()))
assert(babet.chdir(previous))

```

Le chemin est résolu vers sa forme canonique : les composants . et .. et les symlinks sont résolus avant le changement.

Le répertoire courant est partagé par tous les threads. Pour cette raison, `chdir` est définitivement refusé dès que le premier `babet.workers.spawn()` a été effectué, même si le worker est déjà terminé.

```

assert(babet.chdir("/srv/mon-app")) -- à faire avant tout spawn
local worker = assert(babet.workers.spawn("return true"))

local ok, err = babet.chdir("/tmp")
-- ok == nil ; err explique que chdir est interdit après workers.spawn

```

Manipulation lexicale des chemins

Ces fonctions traitent uniquement des chaînes. Elles ne vérifient pas que le chemin existe.

babet.getBaseName(path)

Renvoie la dernière composante, extension comprise.

```

local name = assert(babet.getBaseName("/var/log/app.log"))
print(name) -- app.log

```

Un chemin qui ne possède pas de dernière composante exploitable, comme /, renvoie (nil, err).

babet.getFilename(path)

Renvoie la dernière composante sans sa dernière extension.

```

print(assert(babet.getFilename("report.tar.gz"))) -- report.tar
print(assert(babet.getFilename("README")))        -- README

```

babet.getExtension(path)

Renvoie la dernière extension, point compris.

```

print(assert(babet.getExtension("report.tar.gz"))) -- .gz

```

Un nom sans extension renvoie (nil, err), pas une chaîne vide.

```

local ext, err = babet.getExtension("README")
-- ext == nil

```

babet.getPath(path)

Renvoie le dossier parent lexical.

```

print(assert(babet.getPath("/var/log/app.log"))) -- /var/log

```

Un nom simple sans composante parent, par exemple `app.log`, renvoie (nil, err) plutôt que ".".

babet.joinPath(...)

Deux formes sont acceptées :

```

local path = babet.joinPath("var", "lib", "mon-app", "data.db")
local path = babet.joinPath({ "var", "lib", "mon-app", "data.db" })

```

Il faut au moins deux segments non vides. En cas de succès, `joinPath` renvoie **une seule valeur**, la chaîne assemblée. En cas d'échec, elle renvoie `(nil, err)`.

La fonction ne fait qu'ajuster le séparateur / à la jonction des segments :

```
print(babet.joinPath("/var/", "/log", "app.log"))
-- /var/log/app.log
```

Elle ne normalise pas les composants . ou .. et ne traite pas un segment absolu ultérieur comme un nouveau point de départ :

```
print(babet.joinPath("base", "..", "autre")) -- base/./autre
print(babet.joinPath("base", "/autre"))      -- base/autre
```

Ne mélange pas la forme table et les arguments séparés ; lorsque le premier argument est une table, son contenu constitue la liste complète des segments.

Créer, supprimer, renommer et lier

`babet.touch(path)`

Crée un fichier vide lorsque le chemin n'existe pas, ou met à jour la date de dernière modification d'un chemin existant.

```
assert(babet.touch("cache/ready.flag"))
```

Le dossier parent doit déjà exister : `touch` ne le crée pas.

```
local ok, err = babet.touch("absent/sous/dossier/fichier.txt")
-- ok == nil si absent/sous/dossier n'existe pas
```

Pour créer d'abord les parents :

```
assert(babet.mkdir("cache/images"))
assert(babet.touch("cache/images/index.dat"))
```

Sur un chemin existant, la fonction met à jour son horodatage ; elle peut donc également toucher un dossier, comme la commande Unix `touch`. Un symlink final valide est suivi vers sa cible.

`touch` n'ouvre jamais un chemin existant dans un mode tronquant. Le parent et la cible existante sont épinglés avant l'opération : si un fichier apparaît pendant une course de création, il est repris sans perdre son contenu. Un symlink final pendant est refusé ; Babet ne crée pas sa cible absente à travers un chemin de lien encore mutable.

`babet.mkdir(path)`

Crée un dossier avec une sémantique équivalente à `mkdir -p` :

- la création est **toujours réursive** ;
- les parents manquants sont créés ;
- rappeler la fonction sur un dossier existant réussit ;
- un fichier ou un autre non-dossier bloquant le chemin produit `(nil, err)`.

```
assert(babet.mkdir("data/cache/images"))
-- crée data, data/cache et data/cache/images si nécessaire
assert(babet.mkdir("data/cache/images"))
-- succès également lorsque le dossier existe déjà
```

Il n'existe **aucune option pour désactiver la récursivité**. Pour exiger que le parent existe déjà, vérifie-le avant l'appel :

```
local parent = "data/cache"
assert(babet.isDir(parent), "le parent doit déjà exister")
assert(babet.mkdir(babet.joinPath(parent, "images")))
```

Les nouveaux dossiers utilisent les permissions 0777 filtrées par l'umask du processus. Le mode du dossier source n'entre pas en jeu.

babet.remove(path)

Supprime uniquement :

- un fichier régulier ;
- un lien symbolique, valide ou cassé.

```
assert(babet.remove("temp.txt"))
-- le lien est supprimé, pas sa cible
assert(babet.link("fichier-important.txt", "raccourci"))
assert(babet.remove("raccourci"))
```

La fonction refuse les dossiers, FIFO, sockets, devices et autres types spéciaux.

```
local ok, err = babet.remove("mon-dossier")
-- ok == nil ; utiliser rmdir ou rmdirAll
```

babet.rmdir(path)

Supprime un **vrai dossier vide**.

```
assert(babet.mkdir("cache-vide"))
assert(babet.rmdir("cache-vide"))
```

La fonction refuse :

- un dossier non vide ;
- un fichier ;
- un symlink, même s'il pointe vers un dossier ;
- un chemin absent.

```
local ok, err = babet.rmdir("dossier-non-vide")
```

babet.rmdirAll(path)

Supprime récursivement un **vrai dossier racine** et tout son contenu.

```
assert(babet.rmdirAll("build-temp"))
```

La racine doit exister et être un dossier réel. Un fichier ou un symlink vers un dossier est refusé.

Les symlinks situés à l'intérieur du dossier supprimé sont supprimés comme des entrées ; leur cible extérieure n'est pas parcourue.

```
local is_dir = babet.isDir("build-temp")
if is_dir then
    assert(babet.rmdirAll("build-temp"))
end
```

babet.rename(source, destination)

Renomme ou déplace une entrée sur le même système de fichiers.

```
assert(babet.rename("ancien.txt", "nouveau.txt"))
```

La fonction accepte les fichiers, dossiers et symlinks, y compris les liens cassés. Elle ne crée pas le dossier parent de destination.

Sous Linux, elle suit la sémantique de `rename(2)` : une destination existante peut être remplacée lorsque les types sont compatibles. Une traversée de systèmes de fichiers échoue ; `rename` n'effectue pas de fallback copie + suppression.

```
local ok, err = babet.rename("/tmp/a.txt", "/autre-montage/a.txt")
-- peut échouer avec cross-device link
```

Pour déplacer une arborescence avec fallback cross-filesystem, utilise moveTree.

babet.link(target, linkpath)

Crée toujours un **lien symbolique**, jamais un lien dur.

La cible est enregistrée exactement telle qu'elle est fournie :

```
assert(babet.link("../shared/config.toml", "app/config.toml"))
```

Une cible relative reste relative, et une cible inexistante est autorisée :

```
assert(babet.link("future-file.txt", "dangling-link"))
```

Contrairement à touch et copy, link crée récursivement les dossiers parents explicites de linkpath :

```
assert(babet.link(".././data.db", "a/b/c/database"))
-- crée a/, a/b/ et a/b/c/ si nécessaire
```

Le chemin du lien ne doit pas déjà être occupé.

Lister, itérer et rechercher

Les trois API ne renvoient pas les chemins sous la même forme :

Fonction	Forme des chemins renvoyés
listFiles(root, ...)	relatifs à root
createFileIterator(root, ...)	tels que produits depuis root : préfixés par le chemin fourni
find(root, ...)	tels que produits depuis root : préfixés par le chemin fourni

babet.listFiles(path, recursive?)

Renvoie une table séquentielle contenant uniquement les fichiers réguliers. Les dossiers ne sont jamais inclus.

Listing non récursif — comportement par défaut

```
local files, err = babet.listFiles("assets")
assert(files, err)

for _, relative_path in ipairs(files) do
    print(relative_path)
end
```

Seuls les fichiers directement contenus dans assets sont renvoyés.

Listing récursif

```
local files, err = babet.listFiles("assets", true)
assert(files, err)

table.sort(files)
for _, relative_path in ipairs(files) do
```

```
    print(relative_path)
end
```

Les chemins sont relatifs à la racine demandée :

```
logo.png
icons/open.png
icons/close.png
```

et non :

```
assets/logo.png
assets/icons/open.png
```

Symlinks

- un symlink valide vers un fichier régulier est inclus sous le nom du lien ;
- un lien cassé est ignoré ;
- un symlink vers un dossier n'est ni inclus comme fichier, ni parcouru ;
- une boucle de symlinks de dossiers ne provoque donc pas de récursion infinie.

```
local files = assert(babet.listFiles("collection", true))
-- un lien collection/externe -> /autre/dossier n'est pas suivi
```

Le second argument est facultatif et vaut false par défaut. Lorsqu'il est présent, il doit être un booléen Lua strict : nombres et chaînes ne sont pas convertis. La fonction accepte exactement un ou deux arguments.

babet.createFileIterator(path, recursive?)

Crée un userdata possédant deux méthodes :

- iterator:next() — renvoie le chemin suivant, ou nil à la fin ;
- iterator:close() — libère explicitement l'itérateur.

Non récursif

```
local iterator, err = babet.createFileIterator("assets")
assert(iterator, err)
```

```
while true do
    local path = iterator:next()
    if path == nil then
        break
    end
    print(path) -- assets/logo.png, etc.
end
```

```
iterator:close()
```

Récursif

```
local iterator = assert(babet.createFileIterator("assets", true))
while true do
    local path = iterator:next()
    if not path then break end
    print(path)
end
iterator:close()
```

Contrairement à son nom, l'objet construit actuellement la liste complète au moment de `create-FileIterator`. Une erreur d'accès ou de parcours est donc renvoyée lors de la création, avant le premier `next()`.

Comportement des symlinks :

- lien valide vers un fichier régulier : inclus sous le chemin du lien ;
- lien cassé, boucle ou cible inaccessible : ignoré ;
- lien vers un dossier : non parcouru ;
- véritable erreur d'inspection d'une entrée ou d'avancement du parcours : création en échec avec `(nil, err)`.

Après `close()`, appeler `next()` lève une erreur Lua. Oublier `close()` n'est pas fatal : le garbage collector finit par libérer l'objet. L'argument facultatif `recursive` doit être un booléen Lua strict et le constructeur accepte exactement un ou deux arguments.

`babef.find(path, opts?)`

Recherche récursivement les entrées d'un dossier et renvoie une table de chemins.

Les deux appels suivants sont équivalents :

```
local entries = assert(babef.find("src"))
local entries = assert(babef.find("src", nil))
```

Sans option, fichiers **et** dossiers sont inclus. La racine elle-même n'est pas incluse.

Options

Option	Type	Défaut	Effet
<code>type</code>	string	aucun filtre	"f" pour fichiers, "d" pour dossiers
<code>glob</code>	string	absent	glob borné sur le nom complet, sensible à la casse
<code>iglob</code>	string	absent	glob borné sur le nom complet, casse ASCII ignorée
<code>path_glob</code>	string	absent	glob borné sur le chemin complet renvoyé, sensible à la casse
<code>path_iglob</code>	string	absent	glob borné sur le chemin complet renvoyé, casse ASCII ignorée
<code>name</code>	string	absent	regex RE2 sur le nom complet, sensible à la casse
<code>iname</code>	string	absent	regex RE2 sur le nom complet, insensible à la casse
<code>path</code>	string	absent	regex RE2 recherchée dans le chemin complet
<code>mindepth</code>	integer	0	profondeur minimale incluse

Option	Type	Défaut	Effet
maxdepth	integer	sans limite pratique	profondeur maximale incluse

Toutes les options présentes sont combinées avec un **ET logique**. Un motif vide agit comme un filtre absent, conformément au comportement historique des champs regex.

Filtres glob sûrs Utilise les champs glob pour les recherches ordinaires sur les noms de fichiers, en particulier lorsque le motif peut provenir d'une personne non fiable :

```
local lua_files = assert(babet.find("src", {
    type = "f",
    glob = "*.lua",
}))
```

```
local images = assert(babet.find("assets", {
    type = "f",
    iglob = "*.jpg",
}))
```

Le langage glob est volontairement réduit :

- * : zéro ou plusieurs octets sauf / ;
- ** : zéro ou plusieurs octets, y compris / ;
- ? : exactement un octet sauf / ;
- \x : octet littéral x.

La correspondance est ancrée sur la totalité du nom ou du chemin. Les crochets, accolades et extensions glob du shell n'ont aucune signification particulière et sont comparés littéralement. Chaque motif est limité à 4096 octets. Le moteur n'utilise aucun backtracking récursif : son travail reste borné de façon polynomiale par la taille du motif et du candidat.

path_glob et path_iglob comparent le chemin complet renvoyé en utilisant / comme séparateur. Il faut employer ** lorsqu'un joker doit traverser des dossiers :

```
local tests = assert(babet.find(".", {
    type = "f",
    path_glob = "**/tests/**/*lua",
}))
```

Les variantes insensibles à la casse ne replient que les lettres ASCII. Les octets UTF-8 hors ASCII sont comparés exactement, ce qui rend le comportement indépendant de la locale du processus.

Expressions régulières RE2 bornées Les champs historiques name, iname et path utilisent désormais le moteur RE2 lié statiquement. name et iname correspondent au nom complet :

```
local images = assert(babet.find("assets", {
    type = "f",
    iname = ".*\\.(png|jpg|jpeg)$",
}))
```

path effectue une recherche regex partielle dans le chemin complet :

```
local tests = assert(babet.find(".", {
    type = "f",
    path = "/tests/",
}))
```

Chaque regex est limitée à 4096 octets et compilée avec un budget mémoire RE2 de 1 Mio. La correspondance reste linéaire : lorsque le cache DFA atteint son budget, RE2 utilise son NFA borné au lieu d'un backtracking exponentiel. Babet active le mode Latin-1 de RE2 afin que chaque octet d'un chemin Linux reste traitable, y compris pour un nom qui n'est pas un UTF-8 valide. `iname` désactive la sensibilité à la casse dans RE2, sans consulter la locale du processus.

RE2 refuse volontairement les constructions qui nécessitent du backtracking ou un état non régulier, notamment les références arrière et les assertions de regard avant/arrière. Un tel motif fait échouer tout l'appel avec `(nil, err)`. Utilise les globs sûrs pour les filtres simples et RE2 lorsqu'un regroupement, une alternative, une classe de caractères ou une répétition bornée est vraiment utile.

Limiter la profondeur La profondeur 0 correspond aux enfants directs de la racine :

```
local direct_children = assert(babet.find("src", {
    maxdepth = 0,
}))
```

Les enfants d'un sous-dossier sont à la profondeur 1 :

```
local one_level_below = assert(babet.find("src", {
    mindepth = 1,
    maxdepth = 1,
}))
```

`mindepth` filtre uniquement les résultats ; les dossiers moins profonds sont tout de même traversés pour atteindre les niveaux demandés.

Symlinks et erreurs Le parcours ne descend pas dans les symlinks de dossiers. Les tests de type suivent néanmoins une cible valide : un symlink vers un fichier peut donc correspondre à `type = "f"`, et un symlink vers un dossier peut apparaître avec `type = "d"` sans que son contenu soit parcouru.

Une regex invalide ou trop longue, un glob invalide ou trop long, ou une erreur de parcours fait échouer tout l'appel avec `(nil, err)`. Il n'existe pas de mode « continuer malgré les erreurs » pour `find`.

Copier et déplacer

`babet.copy(source, destination)`

Copie un seul fichier avec `std::filesystem::copy_file` et écrase une destination fichier existante.

```
assert(babet.copy("config/default.toml", "config/local.toml"))
```

Le dossier parent de destination doit déjà exister :

```
assert(babet.mkdir("backup/config"))
assert(babet.copy("config/app.toml", "backup/config/app.toml"))
```

Comportement important :

- un symlink source valide est suivi et donne un nouveau fichier régulier ;
- un symlink destination valide est suivi : la cible du lien est écrasée ;
- un dossier source ou destination n'est pas accepté ;
- les permissions ordinaires du fichier source sont copiées selon la sémantique du système ; `owner`, `group` et `horodatages` ne sont pas garantis ;
- l'opération ne bénéficie pas du confinement renforcé de `copyTree`.

Utilise `copyTree` pour une arborescence ou lorsque la destination doit être protégée contre les redirections par symlink.

babet.copyTree(source, destination, continue_on_error?)

Copie récursivement un dossier source dans un dossier destination.

```
assert(babet.copyTree("site", "backup/site"))
```

Valeur par défaut de continue_on_error Le troisième argument est facultatif et vaut **true par défaut**. Lorsqu'il est présent, il doit être un booléen Lua strict ; la fonction accepte exactement deux ou trois arguments.

Cela signifie qu'en cas d'erreur limitée à une entrée :

1. un avertissement est écrit sur stderr ;
2. la copie tente de poursuivre avec les autres entrées ;
3. l'appel final renvoie (nil, "completed with warnings ...").

Le succès partiel n'est donc jamais présenté comme un succès total.

```
local ok, err = babet.copyTree("source", "destination")
if not ok then
    print(err) -- peut signaler des warnings après une copie partielle
end
```

Mode strict Passe explicitement false pour arrêter la copie à la première erreur :

```
local ok, err = babet.copyTree("source", "destination", false)
assert(ok, err)
```

Passe explicitement true lorsque tu souhaites rendre le choix visible dans le code :

```
local ok, err = babet.copyTree("source", "destination", true)
```

Destination et fusion

- la destination est créée récursivement si elle n'existe pas ;
- si elle existe, le contenu est fusionné ;
- les fichiers réguliers existants sont remplacés ;
- un symlink déjà présent dans la destination, comme composant intermédiaire ou comme fichier final, est refusé ;
- la destination ne peut pas être située à l'intérieur de la source ;
- la racine source et la racine destination ne doivent pas être des symlinks.

Métadonnées Lorsqu'un nouveau fichier est créé :

- les bits ordinaires rwx du fichier source sont conservés ;
- setuid, setgid et sticky sont retirés ;
- le nouvel inode appartient à l'utilisateur et au groupe déterminés par le système pour le processus appelant ;
- les horodatages ne sont pas conservés.

Les dossiers créés utilisent 0777 filtré par l'umask. Le mode, owner, group et les dates des dossiers sources ne sont pas reproduits.

Symlinks internes Les liens symboliques présents dans l'arbre sont recréés :

- les cibles relatives restent identiques ;
- les liens cassés restent des liens cassés ;
- une cible absolue située à l'intérieur de la source est réécrite pour viser l'élément correspondant dans la destination ;
- une cible absolue extérieure reste inchangée.

Les symlinks de dossiers ne sont jamais parcourus comme des dossiers.

babet.moveTree(source, destination)

Déplace récursivement une arborescence.

```
assert(babet.moveTree("staging/site", "public/site"))
```

Chemin rapide Lorsque la destination n'existe pas, que le déplacement reste sur le même filesystem et qu'aucun lien absolu interne ne doit être réécrit, Babet renomme l'arbre complet. L'opération est alors rapide et conserve les inodes et leurs métadonnées.

Fusion ou traversée de filesystem Lorsque la destination existe, que le déplacement traverse un filesystem, ou qu'un lien interne doit être réécrit, Babet utilise un fallback entrée par entrée :

- scan complet de la source avant la première modification ;
- création des dossiers de destination ;
- création des symlinks de destination ;
- déplacement des autres entrées ;
- suppression de l'arbre source résiduel.

Dans ce fallback, un fichier copié vers un nouvel inode conserve ses bits rwx ordinaires, mais pas les bits spéciaux, l'owner, le group ou les dates. Les dossiers créés suivent l'umask.

Garde-fous

- la source doit être un vrai dossier, pas un symlink ;
- la destination racine ne doit pas être un symlink ;
- la destination ne peut pas se trouver dans la source ;
- un symlink préexistant dans la destination de fusion est refusé ;
- les liens internes utilisent les mêmes règles de réécriture que copyTree.

moveTree réduit les risques d'état partiel en scannant l'arbre avant de le modifier et en préparant les symlinks en premier. Il n'est toutefois pas une transaction générale : une erreur survenant après plusieurs déplacements de fichiers peut laisser une partie des entrées dans la source et une partie dans la destination. Le message d'erreur doit alors être traité comme nécessitant une vérification manuelle des deux arbres.

Permissions et attributs Unix

Les modes sont des entiers. Lua ne possède pas de littéral 0o755 : utilise une chaîne octale lorsque l'API l'accepte, ou tonumber("755", 8).

```
local mode_755 = tonumber("755", 8)
```

babet.getMode(path) et babet.setMode(path, mode)

getMode renvoie les permissions ordinaires et les bits spéciaux dans l'intervalle 0000..07777.

```
local mode, err = babet.getMode("run.sh")
assert(mode, err)
print(string.format("%04o", mode))
```

setMode accepte deux formes :

```
assert(babet.setMode("run.sh", "755"))
assert(babet.setMode("run.sh", tonumber("755", 8)))
```

Une chaîne est interprétée en base 8. Un nombre est pris comme valeur entière directe. Les valeurs non entières, négatives ou supérieures à 07777 sont refusées.

Les bits spéciaux peuvent être demandés explicitement :

```
assert(babet.setMode("outil", "4755"))
local mode = assert(babet.getMode("outil"))
assert(mode == tonumber("4755", 8))
```

Ces fonctions suivent un symlink vers sa cible.

babet.getAttributes(path)

Renvoie uniquement les trois champs suivants :

```
local attrs, err = babet.getAttributes("fichier.txt")
assert(attrs, err)
```

```
print(attrs.mode)  -- integer 0000..07777
print(attrs.owner) -- UID
print(attrs.group) -- GID
```

La table ne contient ni taille, ni mtime, ni type de fichier.

La fonction suit un symlink vers sa cible.

babet.setAttributes(path, uid, gid, mode?)

Modifie le propriétaire et le groupe, puis éventuellement le mode.

Sans mode :

```
local attrs = assert(babet.getAttributes("fichier.txt"))
assert(babet.setAttributes(
    "fichier.txt",
    attrs.owner,
    attrs.group
))
```

Avec mode :

```
assert(babet.setAttributes(
    "fichier.txt",
    1000,
    1000,
    tonumber("640", 8)
))
```

Contrairement à `setMode`, le quatrième argument doit être un **entier** ; une chaîne comme "640" n'est pas acceptée.

Validation avant modification :

- UID et GID doivent être des entiers non négatifs dans la plage de `uid_t` et `gid_t` ;
- le mode doit être compris entre 0 et 07777.

POSIX ne fournit pas d'opération atomique combinant `chown` et `chmod`. Babet résout le chemin une seule fois, épingle cette cible, puis effectue la lecture des métadonnées, le `chown`, le `chmod` éventuel et tout rollback sur le même inode. Un remplacement du chemin pendant l'appel ne peut donc pas rediriger la phase suivante vers une nouvelle cible. Si le `chmod` échoue après un `chown` réussi, Babet tente de restaurer l'owner, le group et le mode d'origine ; une restauration incomplète est signalée dans le message d'erreur.

L'appel suit un symlink final, mais la cible sélectionnée au début reste épinglée même si le lien est modifié concurremment. Les droits nécessaires dépendent du système et de l'identité du processus ; changer l'owner exige généralement les privilèges root.

`babef.symlinkAttr(path, uid, gid)` et `babef.symlinkattr(...)`

`symlinkAttr` est le nom canonique. `symlinkattr` est un alias déprécié.

La fonction applique `lchown` et ne suit donc pas un lien symbolique :

```
assert(babef.symlinkAttr("raccourci", 1000, 1000))
```

Elle modifie l'UID et le GID du lien lui-même, pas ceux de sa cible. Elle ne modifie pas le mode.

UID et GID sont validés comme dans `setAttributes`.

Sommes de contrôle de fichiers

Toutes les fonctions suivantes lisent le fichier en flux et renvoient une chaîne hexadécimale en minuscules :

Fonction	Algorithme	Longueur hex	Usage conseillé
<code>babef.crc32sum(path)</code>	CRC-32 IEEE	8	détection d'erreurs accidentelles uniquement
<code>babef.md5sum(path)</code>	MD5	32	compatibilité legacy, pas sécurité
<code>babef.sha1sum(path)</code>	SHA-1	40	compatibilité legacy, pas sécurité
<code>babef.sha256sum(path)</code>	SHA-256	64	intégrité cryptographique courante
<code>babef.sha384sum(path)</code>	SHA-384	96	intégrité cryptographique
<code>babef.sha512sum(path)</code>	SHA-512	128	intégrité cryptographique
<code>babef.sha3_256sum(path)</code>	SHA3-256	64	intégrité cryptographique
<code>babef.sha3_384sum(path)</code>	SHA3-384	96	intégrité cryptographique
<code>babef.sha3_512sum(path)</code>	SHA3-512	128	intégrité cryptographique
<code>babef.blake2b512sum(path)</code>	BLAKE2b-512	128	intégrité cryptographique
<code>babef.blake2s256sum(path)</code>	BLAKE2s-256	64	intégrité cryptographique

Exemple :

```
local digest, err = babef.sha256sum("release.tar.gz")
assert(digest, err)
print(digest)
```

Vérification avec une valeur attendue :

```
local expected = "...
local actual = assert(babef.sha256sum("release.tar.gz"))
assert(actual == expected, "checksum incorrect")
```

Contrat commun :

- seuls les fichiers réguliers sont acceptés ;
- un symlink valide vers un fichier régulier est suivi ;
- les dossiers, FIFO, sockets, devices et pseudo-fichiers non réguliers sont refusés ;

- une erreur de lecture renvoie (nil, err) et ne produit jamais le digest trompeur d'un contenu partiel ;
- le contenu n'est pas chargé entièrement en mémoire.

CRC-32, MD5 et SHA-1 ne doivent pas servir à authentifier des données face à un attaquant. Pour un nouveau contrôle d'intégrité, préfère SHA-256, SHA-3 ou BLAKE2.

CRC-32 en mémoire

`babet.crc32(data)`

Calcule le CRC-32 d'une chaîne Lua déjà en mémoire. Les chaînes Lua étant binary-safe, les octets NUL sont acceptés.

```
print(babet.crc32("abc")) -- 352441c2
print(babet.crc32(""))   -- 00000000

local bytes = "a\0b\0c"
local digest = babet.crc32(bytes)
```

En cas de succès, cette fonction renvoie une seule chaîne, sans seconde valeur nil. Un mauvais nombre ou type d'arguments lève une erreur Lua.

Contrat d'erreur

Erreurs Lua levées

Une mauvaise signature ou un type incompatible lève généralement une erreur Lua :

```
babet.fileSize({})           -- erreur Lua
babet.mkdir("a", true)      -- erreur Lua : mkdir accepte exactement un argument
```

Les chemins contenant un octet NUL sont refusés avant tout appel système.

Erreurs renvoyées

Les échecs du système de fichiers sont normalement renvoyés :

```
local ok, err = babet.remove("absent.txt")
if not ok then
    print(err)
end
```

Les prédicats `fileExists`, `isFile` et `isDir` traitent un chemin simplement absent comme une réponse normale (false, nil). Une vraie erreur d'inspection reste (nil, err).

Opérations partielles

- `copyTree` en mode continuation peut copier certaines entrées puis renvoyer une erreur de synthèse indiquant des warnings ;
- `moveTree` peut laisser un état partagé entre source et destination si une erreur survient tard dans le fallback entrée par entrée ;
- `setAttributes` tente un rollback si la phase `chmod` échoue après `chown`, mais signale explicitement si ce rollback n'est pas complet.

Décisions et limites

- **Namespace plat historique** : `babet.fileExists`, pas `babet.fs.fileExists`.
- **Noms canoniques** : `isFile`, `isDir` et `symlinkAttr`. Les formes `isfile`, `isdir` et `symlinkattr` sont dépréciées.

- **mkdir toujours récursif** : l'API ne possède pas de variante non récursive.
- **copy pour un fichier, copyTree pour un arbre** : aucune surcharge implicite selon le type de la source.
- **Le glob est un filtre de find, pas une fonction autonome** : utilise glob, iglob, path_glob ou path_iglob dans `babet.find`.
- **Pas de tri automatique** : appelle `table.sort` lorsqu'un ordre stable est nécessaire.
- **Pas de préservation complète des métadonnées dans les copies** : les permissions ordinaires des fichiers sont prises en charge, mais pas les propriétaires, groupes, dates ou modes des dossiers.
- **Pas d'I/O asynchrone** : utilise les [workers](#) pour paralléliser des traitements indépendants.

HTTP — requêtes web synchrones, headers, corps, TLS et limites

`babet.http` est un client HTTP/HTTPS synchrone construit sur `cpp-httplib` et `OpenSSL`. Il convient aux APIs web, webhooks, appels REST simples, réponses en mémoire bornées et téléchargements directs vers un fichier.

Le module couvre :

- HTTP et HTTPS ;
- GET, HEAD, OPTIONS, POST, PUT, PATCH et DELETE ;
- paramètres de query encodés et normalisés ;
- headers de requête ;
- corps texte ou binaire ;
- réponses binaires ;
- headers répétés ;
- redirections optionnelles ;
- vérification TLS activée par défaut ;
- timeout de connexion et budget global ;
- taille maximale de réponse en mémoire ;
- téléchargement GET en streaming avec remplacement atomique du fichier final.

Il ne fournit pas d'upload en streaming, de callbacks de réception, de session persistante exposée, de cookies, de multipart/form-data, de helper de formulaire URL-encoded, de proxy, de serveur HTTP, de WebSocket, ni d'HTTP/2 ou HTTP/3.

Table des matières du module

- [Conventions essentielles](#)
- [Vue d'ensemble de l'API](#)
- [request\(opts\)](#)
- [get\(url, opts?\)](#)
- [download\(url, destination, opts?\)](#)
- [post](#) et ses formes d'appel
- [URL, fragments et query](#)
- [Méthodes HTTP](#)
- [Headers de requête](#)
- [Corps de requête et Content-Type](#)
- [Timeouts](#)
- [TLS et CA](#)
- [Redirections](#)
- [Taille maximale de réponse](#)
- [Taille et sécurité du téléchargement](#)
- [Tables de réponse](#)
- [Exemples complets](#)

- [Contrat d'erreur](#)
- [Sécurité et limites](#)

Conventions essentielles

Client synchrone

L'appel bloque le thread courant jusqu'à la réponse ou l'erreur. Pour lancer plusieurs requêtes indépendantes en parallèle, utilise des [workers](#), chacun avec sa propre requête.

Statut HTTP et erreur de transport

Une réponse HTTP reçue est un succès de transport, même avec un statut 404 ou 500 :

```
local response, err = babet.http.get(url, { timeout = 10 })
assert(response, err)

if response.status >= 200 and response.status < 300 then
    print(response.body)
else
    io.stderr:write("HTTP ", response.status, "\n", response.body)
end
```

Un échec DNS, TCP, TLS, timeout ou limite de corps renvoie (nil, err).

Corps binaires

Le corps de requête et response.body sont binary-safe :

```
local response = assert(babet.http.post(url, "AB\0CD", {
    timeout = 10,
}))
assert(#response.body >= 0)
```

Les URL, méthodes, noms de headers, chemins de CA et valeurs destinées à une API C ne peuvent pas contenir de NUL. Les URL et headers refusent aussi CR/LF pour empêcher l'injection de lignes HTTP.

Options inconnues

Les champs inconnus de la table sont actuellement ignorés. Cette faute ne produit donc pas d'erreur :

```
local r = babet.http.get(url, { timeot = 5 }) -- typo : pas de timeout
```

Utilise exactement les noms documentés et centralise éventuellement la construction des options dans ton application.

Vue d'ensemble de l'API

```
local response, err = babet.http.request(opts)
local response, err = babet.http.get(url, opts?)
local result, err = babet.http.download(url, destination, opts?)
local response, err = babet.http.post(url)
local response, err = babet.http.post(url, body)
local response, err = babet.http.post(url, opts)
local response, err = babet.http.post(url, body, opts)
local response, err = babet.http.post(url, nil, opts)
```

Table d'options

Champ	Type	Défaut	Rôle
url	string	obligatoire pour request	URL absolue http:// ou https://
method	string	"GET"	méthode supportée, insensible à la casse
headers	table	{}	noms string, valeurs string ou number
body	string binaire	absent	corps, méthodes à corps uniquement
query	table	absente	clés string, valeurs string ou number
timeout	nombre fini > 0	défauts internes	secondes, connexion + budget global
verify	booléen strict	true	vérification du certificat HTTPS
ca_cert	string	trust store OpenSSL	fichier CA supplémentaire/configuré
follow_redirects	booléen strict	false	suit les redirections
max_body_size	entier 1..2 Gio	64 MiB	cap mémoire pour request/get/post
max_file_size	entier positif	8 Gio	cap du fichier reçu par download

Les wrappers get et post remplacent toujours url et method, même si la table fournie contient d'autres valeurs.

babet.http.request(opts)

Forme générale pour toutes les méthodes :

```
local response, err = babet.http.request({
  url = "https://api.example.com/items/42",
  method = "DELETE",
  headers = {
    ["Authorization"] = "Bearer " .. token,
  },
  timeout = 10,
})
```

Le premier argument doit être une table. opts.url doit être présent et être une string stricte. Les erreurs dans les champs renvoient (nil, err) plutôt que de lever, sauf mauvais type du premier argument.

babet.http.get(url, opts?)

Raccourci qui copie superficiellement opts, puis impose :

```
opts.url = url
opts.method = "GET"

local response, err = babet.http.get(
  "https://api.example.com/health",
  { timeout = 5 }
)
```

Le second argument doit être absent, nil ou une table.

Un corps est interdit pour GET, y compris via `opts.body` :

```
local response, err = babet.http.get(url, { body = "x" })
-- nil, "http: body not allowed for GET"
```

`babet.http.download(url, destination, opts?)`

Télécharge une réponse GET directement dans un fichier, sans accumuler le corps dans Lua ni dans une chaîne C++ :

```
local result, err = babet.http.download(
    "https://example.com/archive.tar.gz",
    "downloads/archive.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
        max_file_size = 4 * 1024 * 1024 * 1024,
    }
)
assert(result, err)
assert(result.saved, "HTTP " .. result.status)
print(result.path, result.bytes)
```

L'appel reste synchrone, mais les chunks reçus sont écrits progressivement dans un fichier temporaire placé dans le même dossier. La destination finale est remplacée par un unique renameat seulement après réception complète de la réponse finale, synchronisation et fermeture du temporaire.

Le wrapper impose toujours GET. Il accepte les options communes `headers`, `query`, `timeout`, `verify`, `ca_cert` et `follow_redirects`. `body` est refusé. `max_file_size` remplace `max_body_size` pour cet appel :

- entier Lua strictement positif ;
- défaut : 8 Gio ;
- configurable jusqu'à `math.maxinteger` ;
- appliqué aux octets réellement transmis au receiver fichier.

Seule une réponse finale 2xx est validée. Une réponse 1xx, 3xx non suivie, 4xx ou 5xx reçue n'est pas une erreur de transport : `download` renvoie une table avec `saved = false`, supprime le temporaire et laisse une éventuelle destination existante strictement inchangée.

Le contrat du chemin est volontairement strict :

- le dossier parent doit déjà exister ;
- Babet ne crée pas les dossiers parents ;
- les composants `..` sont refusés ;
- les composants parents qui sont des symlinks sont refusés ;
- un symlink exactement à la destination est remplacé comme inode, sans suivre ni modifier sa cible ;
- le temporaire est créé dans le même dossier avec `O_EXCL` et supprimé après toute erreur ;
- une destination existante reste intacte après erreur DNS, TCP, TLS, timeout, limite de taille, statut HTTP non-2xx ou écriture disque.

Le renommage atomique empêche les lecteurs d'observer un fichier final partiel. L'API ne fournit pas de reprise, callback de progression, durabilité garantie du dossier parent après crash, ni préservation des métadonnées de l'ancien fichier. Le nouveau fichier est créé avec le mode 0666 filtré par l'umask du processus.

babet.http.post et ses formes d'appel

Sans corps

```
local response, err = babet.http.post(url)
local response, err = babet.http.post(url, { timeout = 5 })
local response, err = babet.http.post(url, nil, { timeout = 5 })
```

Dans ces formes, aucun champ body n'est ajouté par le wrapper. La requête POST peut donc être envoyée sans corps explicite.

Corps positionnel

```
local response, err = babet.http.post(url, "payload", {
    timeout = 5,
})
```

Le corps doit être une string. Un nombre, un booléen ou une table à cette position lève une erreur Lua, sauf qu'une table au deuxième argument est interprétée comme opts.

Corps dans opts

```
local response, err = babet.http.post(url, {
    body = "payload",
    timeout = 5,
})
```

Priorité

Si un corps positionnel et opts.body sont fournis, le corps positionnel gagne :

```
local response = assert(babet.http.post(url, "used", {
    body = "ignored",
    timeout = 5,
}))
```

URL, fragments et query

URL absolue

Babet accepte uniquement les schemes http et https :

```
https://example.com:8443/path?existing=1#fragment
```

L'URL doit contenir un host non vide. Les littéraux IPv6 entre crochets sont pris en charge :

```
local r, err = babet.http.get("http://[::1]:8080/", { timeout = 2 })
```

Le fragment #... n'est jamais envoyé au serveur.

Babet n'effectue pas une validation RFC exhaustive du host avant de le passer à cpp-httplib. Les erreurs de résolution ou de syntaxe restantes apparaissent au transport.

Ajouter une query

```
local response = assert(babet.http.get("https://example.com/search", {
    query = {
        q = "lua socket",
        page = 2,
    },
})
```

```

    timeout = 10,
  )))

```

Les clés doivent être des strings. Les valeurs peuvent être des strings ou des numbers. Babet protège les délimiteurs et les octets non ASCII, puis cpp-httplib 0.45.0 normalise la query avant l'envoi. Le format effectivement envoyé suit donc les conventions suivantes :

- l'espace devient + ;
- + devient %2B ;
- / et ? restent littéraux dans une valeur ;
- les octets UTF-8 non ASCII sont percent-encodés en %HH ;
- &, = et % sont percent-encodés lorsqu'ils appartiennent à une clé ou une valeur.

Par exemple, q = "lua socket" devient q=lua+socket, tandis que path = "a/b" reste path=a/b.

Si l'URL contient déjà ?a=b, les nouveaux paramètres sont ajoutés avec &. La query déjà présente est elle aussi normalisée par cpp-httplib : %20 peut devenir + et %2F peut devenir /. Le fragment #... est supprimé avant l'envoi.

L'ordre d'itération d'une table Lua n'est pas garanti : ne signe pas une URL en supposant l'ordre produit par query. Pour une signature canonique, construis toi-même une URL déjà ordonnée et tiens compte de la normalisation décrite ci-dessus.

Les NUL présents dans une clé ou une valeur sont envoyés sous la forme %00, car la query est traitée comme des octets. Assure-toi que le serveur accepte ce contenu.

Méthodes HTTP

Méthodes supportées :

- GET ;
- HEAD ;
- OPTIONS ;
- POST ;
- PUT ;
- PATCH ;
- DELETE.

La casse est normalisée : method = "put" devient PUT.

```

local response = assert(babet.http.request({
  url = "https://api.example.com/items/42",
  method = "PATCH",
  body = '{"enabled":true}',
  headers = { ["Content-Type"] = "application/json" },
  timeout = 10,
}))

```

GET, HEAD et OPTIONS refusent un champ body. POST, PUT, PATCH et DELETE acceptent un corps absent, vide ou non vide.

Il n'existe pas de raccourci put, patch ou delete dans la version actuelle : utilise request.

Headers de requête

```

headers = {
  ["Accept"] = "application/json",
  ["Authorization"] = "Bearer " .. token,
  ["X-Retry"] = 3,
}

```

Noms

Une clé doit être une string non vide composée uniquement de caractères HTTP token : lettres, chiffres et `!#$%&'*+-.^_`|~`.

Sont notamment refusés :

- espace ou tabulation ;
- `:` ;
- CR/LF ;
- NUL ;
- clé non string.

Cette validation empêche l'injection d'un second header ou d'une ligne de requête.

Valeurs

Une valeur peut être une string ou un number. Les nombres sont convertis en texte avec la conversion Lua.

NUL, CR et LF sont refusés. Les booléens, tables et autres types sont refusés.

Doublons

Une table Lua ne peut contenir qu'une valeur par clé exacte. Le binding ne fournit pas de forme permettant d'émettre plusieurs headers de requête portant le même nom. Pour les listes combinables, construis une valeur séparée par des virgules lorsque le standard du header l'autorise. Ne combine pas ainsi Cookie ou d'autres champs sans vérifier leur syntaxe.

Casse

Les noms sont transmis avec la casse fournie. HTTP traite les noms de headers sans tenir compte de la casse. Pour détecter Content-Type, Babet compare en minuscules.

Corps de requête et Content-Type

Corps absent et corps vide

Ces deux cas sont distincts :

```
-- Aucun champ body.
babet.http.post(url, { timeout = 5 })

-- Corps explicitement présent, mais vide.
babet.http.post(url, "", { timeout = 5 })
```

Lorsque body existe sur une méthode à corps et qu'aucun Content-Type n'est fourni, Babet utilise : `application/octet-stream`

Avec un corps absent, aucun Content-Type par défaut n'est ajouté.

JSON

```
local payload = assert(babet.json.encode({
    name = "babet",
    enabled = true,
}))

local response = assert(babet.http.post(url, payload, {
    headers = {
```

```

        ["Content-Type"] = "application/json",
        ["Accept"] = "application/json",
    },
    timeout = 10,
}))

```

Formulaire URL-encoded

Babet ne fournit pas de helper form. Encode explicitement le corps et le header :

```

local body = "user=julien&active=1" -- encoder chaque champ si non fiable
local response = assert(babet.http.post(url, body, {
    headers = {
        ["Content-Type"] = "application/x-www-form-urlencoded",
    },
    timeout = 10,
}))

```

La table query ne doit pas être détournée pour produire automatiquement un corps de formulaire : elle modifie uniquement l'URL.

Multipart

Aucun constructeur multipart n'est fourni. Pour des uploads complexes ou du streaming de gros fichiers, utilise un outil spécialisé via `babet.exec`, par exemple `curl`, en transmettant les valeurs sensibles par arguments ou fichiers plutôt que par concaténation shell.

opts.timeout

```

local response, err = babet.http.get(url, { timeout = 10 })

```

Le timeout doit être un nombre fini strictement positif. Les valeurs positives inférieures à 1 ms sont arrondies à 1 ms. Le maximum représentable est environ `INT_MAX` millisecondes.

Babet configure deux protections `cpp-httpplib` :

- timeout de connexion ;
- timeout maximal global de la requête.

La première limite atteinte gagne.

Limite DNS

Comme les sockets bruts, la résolution DNS synchrone peut se produire hors du budget effectivement contrôlé par `cpp-httpplib`. Un résolveur bloqué peut donc faire dépasser la durée demandée.

Toujours en fournir un

Sans `opts.timeout`, Babet laisse les valeurs internes de `cpp-httpplib` et du système s'appliquer. Pour tout serveur distant, passe explicitement une limite.

HTTPS, vérification et CA

Vérification par défaut

```

local response = assert(babet.http.get("https://example.com/", {
    timeout = 10,
}))

```

`verify = true` par défaut. Le certificat et le hostname de l'URL sont vérifiés par `cpp-httpplib/OpenSSL`.

CA spécifique

```
local response = assert(babet.http.get("https://internal.example/", {
    ca_cert = "/etc/myapp/internal-ca.pem",
    timeout = 10,
}))
```

`ca_cert` configure le chemin de CA utilisé par le client HTTP. Contrairement à l'API socket TLS, HTTP n'expose pas `ca_path` et n'exécute pas le même probing manuel de plusieurs chemins de distributions. Il dépend du trust store par défaut de l'OpenSSL embarqué, des variables OpenSSL et du fichier fourni.

Désactiver la vérification

```
local response = assert(babet.http.get("https://127.0.0.1:8443/", {
    verify = false,
    timeout = 3,
}))
```

Réserve cette forme aux tests contrôlés. Il n'existe pas d'option hostname HTTP séparée : le host de l'URL est utilisé.

follow_redirects

Par défaut, une redirection reste visible :

```
local r = assert(babet.http.get(url, { timeout = 10 }))
if r.status == 301 or r.status == 302 or r.status == 307 or r.status == 308 then
    print(r.headers.location)
end
```

Pour suivre automatiquement :

```
local r = assert(babet.http.get(url, {
    follow_redirects = true,
    timeout = 10,
}))
```

La table de réponse décrit alors la réponse finale. Babet n'expose pas l'historique des hops ni une option `max_redirects`; la limite interne de `cpp-httpplib` s'applique.

Avant d'activer le suivi sur une URL non fiable, considère :

- redirection vers un autre domaine ;
- passage HTTPS vers HTTP ;
- accès à une adresse interne ou metadata cloud ;
- transmission de headers sensibles selon la politique de la bibliothèque.

max_body_size

La réponse est entièrement accumulée en mémoire avant d'être renvoyée.

```
local response, err = babet.http.get(url, {
    timeout = 30,
    max_body_size = 8 * 1024 * 1024,
})
```

- défaut : 64 MiB ;

- minimum : 1 octet ;
- maximum : 2 GiB ;
- type : entier Lua strictement positif.

Dès que le flux dépasserait la limite, Babet annule la réception et renvoie :

```
nil, "http: response body exceeds max_body_size"
```

Aucun corps partiel n'est exposé. Le header Content-Length n'est pas la seule protection : la limite s'applique aux chunks réellement reçus, y compris avec un transfert chunked ou une longueur absente/trompeuse.

Pour un gros payload GET, utilise `babet.http.download` au lieu d'augmenter cette limite en mémoire.

max_file_size et sécurité du fichier

`max_file_size` s'applique uniquement à `download`. La limite porte sur les octets du corps reçus, pas seulement sur le Content-Length annoncé. Si le flux dépasserait le plafond, Babet annule, supprime le temporaire, préserve la destination existante et renvoie :

```
nil, "http: response body exceeds max_file_size"
```

Le défaut de 8 Gio est un garde-fou, pas une recommandation d'accepter tout fichier de cette taille. Choisis la plus petite limite compatible avec l'artefact attendu. Un manque d'espace ou une erreur du système de fichiers renvoie aussi (`nil`, `err`) avec la même garantie de nettoyage.

Tables de réponse

Réponse en mémoire

```
{
  status = 200,
  body = "...",
  headers = {
    ["content-type"] = "application/json",
    ["set-cookie"] = "b=2",
  },
  headers_multi = {
    ["content-type"] = { "application/json" },
    ["set-cookie"] = { "a=1", "b=2" },
  },
}
```

status

Nombre entier du statut HTTP reçu.

body

Chaîne binaire complète, éventuellement vide. Pour HEAD, le corps est vide, même si Content-Length décrit la taille qu'aurait eue une réponse GET.

headers

Table avec noms en minuscules. Si un header apparaît plusieurs fois, la dernière occurrence rencontrée gagne. Cette vue est pratique pour les headers simples mais insuffisante pour Set-Cookie.

headers_multi

Chaque nom en minuscules pointe toujours vers une séquence de toutes les occurrences, même lorsqu'il n'y en a qu'une.

```
for _, cookie in ipairs(response.headers_multi["set-cookie"] or {}) do
    print(cookie)
end
```

L'ordre des occurrences suit celui fourni par cpp-httplib. L'ordre des noms dans pairs() n'est pas garanti.

Résultat d'un téléchargement

```
{
    status = 200,
    saved = true,
    bytes = 123456,
    path = "downloads/archive.tar.gz",
    headers = { ... },
    headers_multi = { ... },
}
```

- status, headers et headers_multi gardent le sens décrit ci-dessus ;
- saved vaut vrai uniquement si le statut final est 2xx et si le commit atomique a réussi ;
- bytes indique le nombre d'octets de corps reçus, y compris pour un non-2xx ;
- path existe uniquement lorsque saved vaut vrai ;
- aucun champ body n'est créé volontairement.

Exemples complets

GET JSON

```
local response, err = babet.http.get("https://api.example.com/v1/status", {
    headers = { ["Accept"] = "application/json" },
    query = { verbose = 1 },
    timeout = 10,
    max_body_size = 1024 * 1024,
})
assert(response, err)

if response.status ~= 200 then
    error("HTTP " .. response.status .. ": " .. response.body)
end

local data, decode_err = babet.json.decode(response.body)
assert(data, decode_err)
```

POST JSON authentifié

```
local body = assert(babet.json.encode({ title = "hello" }))
local response, err = babet.http.post(
    "https://api.example.com/v1/items",
    body,
    {
        headers = {
            ["Authorization"] = "Bearer " .. token,
            ["Content-Type"] = "application/json",
        }
    })
```

```

        ["Accept"] = "application/json",
    },
    timeout = 15,
    max_body_size = 2 * 1024 * 1024,
}
)
assert(response, err)

```

PUT binaire

```

local file = assert(io.open("image.bin", "rb"))
local payload = file:read("a")
file:close()

local response = assert(babet.http.request({
    url = "https://upload.example.com/blob/42",
    method = "PUT",
    body = payload,
    headers = { ["Content-Type"] = "application/octet-stream" },
    timeout = 30,
    max_body_size = 1024 * 1024,
}))

```

Le fichier est lu entièrement en mémoire avant l'appel. Cette recette ne convient pas à un très gros fichier.

Télécharger un artefact de release

```

assert(babet.mkdir("downloads"))
local result, err = babet.http.download(
    "https://example.com/releases/tool.tar.gz",
    "downloads/tool.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
        max_file_size = 512 * 1024 * 1024,
    }
)
assert(result, err)
if not result.saved then
    error("statut HTTP inattendu " .. result.status)
end
print(result.bytes, "octets enregistrés dans", result.path)

```

Gérer 404 séparément des erreurs réseau

```

local response, err = babet.http.get(url, { timeout = 5 })
if not response then
    io.stderr:write("transport impossible: ", err, "\n")
elseif response.status == 404 then
    print("ressource absente")
elseif response.status >= 400 then
    io.stderr:write("erreur HTTP ", response.status, "\n")
else
    print(response.body)
end

```

Requêtes parallèles avec WORKERS

```
local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = assert(babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, {
            timeout = 10,
            max_body_size = 1024 * 1024,
        })
        if not r then error(err) end
        return { status = r.status, size = #r.body }
    ]], { url = url })))
end

for i, job in ipairs(jobs) do
    local ok_result, value = job:join()
    if ok_result then
        print(urls[i], value.status, value.size)
    else
        io.stderr:write(urls[i], ": ", value, "\n")
    end
end
```

Contrat d'erreur

Erreurs Lua levées

Mauvais type des arguments positionnels :

```
babet.http.request("not a table")
babet.http.get(42)
babet.http.get(url, "not a table")
babet.http.post(url, 42)
babet.http.download(url, 42)
babet.http.download(url, "fichier", "pas une table")
```

Erreurs (nil, err)

- opts.url absent ou non string ;
- URL sans scheme/host, scheme non supporté, CR/LF ou NUL ;
- méthode non supportée ;
- corps sur GET/HEAD/OPTIONS ;
- champ d'option au mauvais type ou hors plage ;
- query invalide ;
- nom ou valeur de header invalide ;
- DNS, connexion, TLS, envoi, lecture ou timeout ;
- corps de réponse dépassant max_body_size ou max_file_size ;
- chemin de destination ou erreur du système de fichiers ;
- exception interne convertie en message http:

Pas une erreur Babet

Un statut 1xx, 3xx, 4xx ou 5xx reçu produit une table normale. Pour download, seul un 2xx est enregistré ; les autres statuts renvoient saved = false. L'application définit sa propre politique.

Sécurité et limites

- **SSRF.** Une URL contrôlée par un utilisateur peut viser loopback, le LAN, des sockets metadata cloud ou des services administratifs. Valide scheme, host, port et redirections selon ton modèle de menace.
- **Headers sensibles.** N'inclus pas un token dans les logs. Réfléchis à son comportement en cas de redirection inter-domaines.
- **TLS.** Garde `verify = true`. Une CA privée est préférable à `verify = false`.
- **Limites.** Réduis `max_body_size` ou `max_file_size` au minimum attendu.
- **Compression.** Selon les capacités compilées de `cpp-httplib` et les headers, le corps peut être traité par la bibliothèque ; la limite Babet porte sur les octets livrés au receiver.
- **Streaming ciblé.** `request`, `get` et `post` gardent la réponse en mémoire. `download` stream une réponse GET vers un fichier ; les uploads restent en mémoire.
- **Validation de l'artefact.** Le remplacement atomique évite un fichier final partiel, mais l'appelant doit encore valider statut, taille attendue et somme cryptographique.
- **Pas de session exposée.** Chaque appel construit un nouveau client ; aucun cookie jar ou pool de connexions n'est garanti à travers les appels.
- **Pas de formulaire/multipart automatique.** Encode explicitement ou utilise un outil spécialisé.
- **DNS synchrone.** Le resolver peut dépasser le timeout demandé.
- **API HTTP uniquement.** Pour un protocole custom, utilise `SOCKET/TLS`.

babet.inotify — surveillance du système de fichiers

`babet.inotify` expose une interface minimale au mécanisme Linux `inotify(7)`. Les notifications proviennent du noyau : il n'est pas nécessaire de parcourir périodiquement les dossiers surveillés.

Le module est **spécifique à Linux** et la surveillance n'est pas récursive.

Table des matières du module

- [API](#)
- [Créer un watcher](#)
- [Ajouter une surveillance](#)
 - [Option `onlydir`](#)
- [Lire les événements](#)
 - [Débordement de la file](#)
 - [Interruption par signal](#)
- [Retirer et fermer](#)
- [Exemple](#)
- [Contrat d'erreur](#)
- [Limites](#)

API

Fonction	Résultat
<code>babet.inotify.new()</code>	<code>watcher</code> <code>(nil, err)</code>
<code>w:add(path, events [, opts])</code>	<code>wd entier</code> <code>(nil, err)</code>
<code>w:read([timeout])</code>	<code>tableau d'événements</code> <code>(nil, "timeout")</code> <code>(nil, "interrupted")</code> <code>(nil, err)</code>
<code>w:remove(wd)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>w:close()</code>	<code>(true, nil)</code>

Les signatures sont strictes : les arguments supplémentaires lèvent une erreur Lua. `close()` est idempotente et le ramasse-miettes ferme aussi un watcher oublié.

Créer un watcher

```
local watcher, err = babet.inotify.new()
if not watcher then
    error(err)
end
```

Chaque appel crée une instance indépendante. Le descripteur inotify est ouvert avec `IN_NONBLOCK` et `IN_CLOEXEC` : `read()` pilote lui-même l'attente, et le descripteur n'est pas transmis aux programmes lancés par `exec`.

Ajouter une surveillance

```
local wd, err = watcher:add(path, events [, opts])
```

`path` doit être une chaîne sans octet NUL. Il peut désigner un dossier ou un fichier existant.

`events` est une liste stricte et non vide, indexée de 1 à N, sans trou ni clé supplémentaire. Chaque élément doit être l'un des noms suivants :

Nom	Événement demandé
"access"	lecture ou accès au fichier
"modify"	contenu modifié
"attrib"	attributs ou métadonnées modifiés
"close_write"	descripteur ouvert en écriture fermé
"close_nowrite"	descripteur non ouvert en écriture fermé
"close"	combinaison de <code>close_write</code> et <code>close_nowrite</code>
"open"	fichier ouvert
"moved_from"	entrée déplacée hors du dossier surveillé
"moved_to"	entrée déplacée dans le dossier surveillé
"move"	combinaison de <code>moved_from</code> et <code>moved_to</code>
"create"	entrée créée dans le dossier surveillé
"delete"	entrée supprimée du dossier surveillé
"delete_self"	cible surveillée supprimée
"move_self"	cible surveillée déplacée

Option `onlydir`

```
local wd = assert(watcher:add(path, { "create" }, {
    onlydir = true,
}))
```

`onlydir` doit être un booléen lorsqu'elle est présente :

- `true` ajoute le masque Linux `IN_ONLYDIR` et refuse donc une cible qui n'est pas un dossier ;
- `false` conserve le comportement normal ;
- les autres champs de `opts` sont actuellement ignorés.

Ajouter de nouveau un watch sur la même cible suit la sémantique native d'inotify : le masque existant est remplacé, car `IN_MASK_ADD` n'est pas utilisé.

Lire les événements

```
local events, err = watcher:read([timeout])
```

Le `timeout` est exprimé en secondes et accepte les valeurs décimales :

- argument omis ou `nil` : attente illimitée ;
- `0` : lecture non bloquante des événements déjà disponibles ;

- valeur positive : attente bornée.

Un timeout négatif, NaN, infini ou trop grand renvoie (`nil`, `err`). Un argument d'un mauvais type lève une erreur Lua.

Une lecture réussie renvoie une liste de tables :

```
{
  {
    wd = 1,
    name = "photo.jpg",
    events = { create = true, close_write = true },
    is_dir = false,
    cookie = 0,
  },
}
```

- `wd` est le watch descriptor renvoyé par `add` ;
- `name` est le nom de l'entrée concernée, ou "" pour un événement portant sur la cible surveillée elle-même ;
- `events` contient un booléen vrai pour chaque bit reçu ;
- `is_dir` reflète `IN_ISDIR` ;
- `cookie` permet d'apparier `moved_from` et `moved_to` ; hors déplacement, il vaut généralement 0.

En plus des événements demandables par `add`, le noyau peut produire :

- `events.ignored` lorsque le watch est retiré, automatiquement ou par `remove` ;
- `events.unmount` lorsque le système de fichiers surveillé est démonté.

Un appel peut renvoyer plusieurs événements, car le module lit et décode un lot complet de la file noyau.

Débordement de la file

Lorsque la file inotify déborde, certains événements sont perdus. Le module renvoie explicitement :

```
{
  wd = -1,
  name = "",
  events = { overflow = true },
  is_dir = false,
  cookie = 0,
}
```

Le programme doit alors rescanner les chemins surveillés pour reconstruire un état fiable.

Interruption par signal

Si un signal géré par `babet.signal` arrive pendant l'attente, son callback Lua est exécuté puis `read()` renvoie (`nil`, `"interrupted"`).

Retirer et fermer

```
assert(watcher:remove(wd))
assert(watcher:close())
```

Après `remove(wd)`, le noyau place normalement un événement `ignored` dans la file. `remove` sur un identifiant invalide renvoie (`nil`, `err`).

Après `close`, `add`, `read` et `remove` renvoient une erreur indiquant que le watcher est fermé. `close` peut être appelée plusieurs fois.

Exemple

```
local watcher = assert(babet.inotify.new())
local wd = assert(watcher:add("/srv/incoming", {
    "close_write",
    "moved_to",
}, {
    onlydir = true,
}))

while true do
    local events, err = watcher:read()
    if not events then
        if err == "interrupted" then
            break
        end
        error(err)
    end

    for _, event in ipairs(events) do
        if event.events.overflow then
            rescan_directory()
        elseif event.wd == wd then
            process_entry(event.name)
        end
    end
end

watcher:close()
```

Contrat d'erreur

- mauvais nombre ou mauvais type d'arguments : erreur Lua ;
- valeur invalide, erreur système ou watcher fermé : (nil, err) ;
- read sans événement avant l'échéance : (nil, "timeout") ;
- signal géré pendant read : (nil, "interrupted") ;
- remove et close réussis : (true, nil).

Les erreurs système sont préfixées par "inotify: " et conservent la description fournie par le système.

Limites

- Linux uniquement ;
- pas de surveillance récursive ;
- pas de IN_DONT_FOLLOW, IN_ONESHOT ni IN_MASK_ADD ;
- une instance peut contenir plusieurs watches et plusieurs instances indépendantes peuvent coexister dans le même processus.

babet.json — encodage et décodage JSON

Le module `babet.json` s'appuie sur **nlohmann/json 3.11.3**. Il convertit les valeurs Lua en texte JSON et inversement, avec deux sentinelles pour représenter sans ambiguïté `nil` et le tableau vide `[]`.

Table des matières du module

- [API](#)
- [Correspondance des types](#)
 - [Lua vers JSON](#)
 - [JSON vers Lua](#)
- [null](#)
- [Objet vide, tableau vide et as_array](#)
- [Options de encode](#)
- [Chaînes, UTF-8 et octets NUL](#)
- [Nombres](#)
- [Ordre des objets et clés dupliquées](#)
- [Contrat d'erreur](#)
- [Limites](#)

API

Élément	Contrat
<code>babet.json.encode(value [, opts])</code>	Renvoie texte, nil ou nil, err
<code>babet.json.decode(text)</code>	Renvoie valeur, nil ou nil, err
<code>babet.json.null</code>	Sentinelle représentant le null JSON
<code>babet.json.empty_array</code>	Sentinelle constante qui s'encode en []
<code>babet.json.as_array(t)</code>	Marque t comme tableau JSON et renvoie la même table

`encode` accepte exactement un ou deux arguments. `decode` exige exactement **une chaîne Lua** : un nombre n'est pas converti implicitement en texte. `as_array` exige exactement une table.

Correspondance des types

Lua vers JSON

Lua	JSON
nil passé directement à encode	null
<code>babet.json.null</code>	null
<code>babet.json.empty_array</code>	[]
booléen	booléen
entier Lua	nombre entier JSON
flottant Lua fini	nombre JSON flottant
chaîne UTF-8 valide	chaîne JSON
table vide non marquée	objet vide {}
clés entières exactement 1..n	tableau JSON
clés uniquement de type chaîne	objet JSON

Dans une table Lua, affecter nil supprime la clé avant même l'encodage. Utilise `babet.json.null` pour conserver explicitement une clé dont la valeur JSON doit être null.

Une table mixant clés chaînes et entières, un tableau à trous, ou une table contenant une clé non représentable (0, entier négatif, flottant, booléen, table, etc.) ne peut pas être convertie : `encode` renvoie alors nil, err.

JSON vers Lua

JSON	Lua
null	babet.json.null
booléen	booléen
entier signé dans la plage Lua	entier Lua
entier non signé supérieur à math.maxinteger	flottant Lua
nombre JSON flottant	flottant Lua
chaîne	chaîne Lua
tableau	table séquentielle 1..n
objet	table à clés chaînes

Un entier JSON trop grand pour `lua_Integer` est converti en flottant plutôt que de reboucler en valeur négative. Cette conversion peut perdre de la précision, comme toute conversion vers un nombre IEEE 754.

null

Lua ne peut pas conserver une valeur nil dans une table. La sentinelle partagée `babet.json.null` comble cette différence :

```
local J = babet.json

local value = assert(J.decode([[{"answer":null}])))
assert(value.answer == J.null)

local text = assert(J.encode({ answer = J.null }))
-- text représente {"answer":null}
```

Teste la sentinelle par identité avec `==`. C'est une table spéciale, mais elle ne doit pas être utilisée comme une table de données.

Objet vide, tableau vide et `as_array`

Une table Lua vide est ambiguë. Babet choisit l'objet JSON par défaut :

```
local J = babet.json

assert(J.encode({}) == "{}")
assert(J.encode(J.empty_array) == "[]")
assert(J.encode(J.as_array({})) == "[]")
```

`empty_array` est une sentinelle partagée destinée aux valeurs que l'on ne souhaite pas modifier. `as_array(t)` est préférable pour un tableau construit dynamiquement : il marque la table avec une métatable interne, la laisse mutable et renvoie exactement la même table.

```
local tags = {}
-- table.insert(tags, ...) peut ne jamais être appelé

local text = assert(J.encode({ tags = J.as_array(tags) }))
-- {"tags":[]}
```

Points importants :

- `as_array` est idempotent ;
- le marquage **remplace toute métatable préexistante** sur la table ;
- la forme n'est vérifiée qu'au moment de `encode` : une table marquée contenant des clés chaînes ou des trous produit nil, err ;

- `as_array(J.null)` et `as_array(J.empty_array)` lèvent une erreur Lua ;
- seul un tableau JSON **vide** décodé reçoit automatiquement ce marquage. Un tableau non vide décodé puis entièrement vidé devient une table vide ordinaire et se ré-encode donc en `{}` ; appelle `as_array(t)` pour conserver `[]` dans ce cas ;
- les écritures ordinaires sur les sentinelles sont refusées. Comme pour toute table Lua, `rawset` contourne les métaméthodes : ne modifie pas les sentinelles.

Options de encode

La table optionnelle ne contient qu'une option interprétée :

```
local compact = assert(babet.json.encode({ a = 1 }))
local pretty  = assert(babet.json.encode({ a = 1 }, { indent = 4 }))
```

- `indent = 0..256` active la mise en forme avec retours à la ligne ;
- `indent = 0` n'ajoute aucun espace d'indentation, mais conserve les retours à la ligne de la sortie formatée ;
- option absente, `nil` ou entier négatif : sortie compacte ;
- une valeur non entière ou supérieure à 256 lève une erreur Lua ;
- les autres champs de la table d'options sont ignorés. En particulier, l'ancienne option documentée par erreur `pretty` n'existe pas.

Chaînes, UTF-8 et octets NUL

Les chaînes JSON doivent être en UTF-8 valide. Les caractères Unicode sont conservés dans la sortie ; Babet ne force pas leur conversion en séquences `\uXXXX`.

Les chaînes Lua peuvent contenir des octets NUL. Ils sont binary-safe côté Lua et sont échappés conformément au JSON lors de l'encodage. Cela vaut aussi pour les clés d'objets :

```
local J = babet.json
local original = "a\0b"
local text = assert(J.encode({ [original] = original }))
local back = assert(J.decode(text))
assert(back[original] == original)
```

Une chaîne ou une clé contenant une séquence UTF-8 invalide fait échouer l'encodage avec `nil`, `err`. Un texte JSON contenant une chaîne UTF-8 invalide est également refusé par `decode`.

Nombres

Babet préserve le sous-type Lua lorsque le JSON le permet : 42 devient un entier Lua, tandis que 42.0 et 1e3 deviennent des flottants. À l'encodage, un flottant Lua comme 3.0 reste un nombre JSON flottant.

`NaN`, `+Inf` et `-Inf` ne font pas partie du standard JSON et sont refusés avec `nil`, `err`.

Ordre des objets et clés dupliquées

L'ordre des membres d'un objet JSON n'a pas de signification sémantique et ne doit pas être utilisé comme contrat. Le backend actuel les émet généralement dans l'ordre lexicographique des clés, mais un appelant doit comparer les données décodées, pas le texte brut, lorsque l'ordre n'est pas imposé par un format externe.

Lors du décodage d'un objet JSON contenant plusieurs fois la même clé, la dernière valeur remplace les précédentes.

Contrat d'erreur

Les erreurs de conversion ou de parsing sont renvoyées et ne lèvent pas d'exception Lua :

```
local value, err = babet.json.decode("{ invalide")
if not value then
    print(err) -- préfixé par "json:"
end
```

Cela comprend notamment :

- JSON invalide, données supplémentaires après le document ou UTF-8 invalide ;
- table mixte, à trous ou avec des clés non représentables ;
- fonction, userdata ou thread impossible à encoder ;
- NaN ou infinité ;
- cycle ou profondeur d'imbrication excessive.

Les erreurs d'utilisation de l'API lèvent en revanche une erreur Lua :

- mauvaise arité ;
- argument de decode qui n'est pas une chaîne ;
- second argument de encode qui n'est ni une table ni nil ;
- `opts.indent` non entier ou supérieur à 256 ;
- argument invalide pour `as_array`.

Les erreurs de parsing fournies par `nlohmann/json` indiquent la position de l'échec, généralement sous forme de ligne et de colonne.

Limites

La profondeur maximale de conversion est de **1000 niveaux** dans les deux sens. Cette limite sert également de garde-fou contre les tables cycliques. Le module construit le document complet en mémoire : il ne propose pas d'API de streaming.

logging — journalisation à niveaux

`logging` est un module Lua pur embarqué dans Babet. Il se charge avec `require("logging")` et ne fait pas partie de la table `babet`.

```
local log = require("logging")
```

Il expose cinq niveaux :

Niveau	Valeur
<code>log.TRACE</code>	10
<code>log.DEBUG</code>	20
<code>log.INFO</code>	30
<code>log.WARN</code>	40
<code>log.ERROR</code>	50

Il n'existe pas de niveau fatal.

Table des matières du module

- [Émettre un message](#)
 - [Garantie de non-propagation](#)
- [Format de sortie](#)
- [Seuil](#)

- [Destination](#)
- [Couleurs ANSI](#)
- [État du module](#)
- [Contrat d'erreur](#)
- [Limites](#)

Émettre un message

```
log.trace(...)
log.debug(...)
log.info(...)
log.warn(...)
log.error(...)
```

Chaque fonction accepte zéro ou plusieurs valeurs et ne renvoie aucune valeur. Un message est émis lorsque le niveau de la fonction est supérieur ou égal au seuil courant.

Le seuil initial est `log.INFO` : `trace` et `debug` sont donc filtrés par défaut.

Les arguments sont convertis avec `tostring` puis joints par un espace :

```
log.info("user=", 42, "active=", true)
-- ... [INFO ] user= 42 active= true
```

Un appel sans argument produit une ligne avec un message vide. Les octets NUL et les retours à la ligne contenus dans le message ne sont ni supprimés ni échappés. Un message multiligne produit donc plusieurs lignes physiques, mais seul le début de l'appel porte le préfixe du logger.

Garantie de non-propagation

Les cinq fonctions d'émission ne lèvent jamais d'erreur. Toute la chaîne est protégée :

- conversion des arguments par `tostring` ;
- création de l'horodatage par `os.date` ;
- appel de la méthode `write` du sink.

Si une de ces opérations échoue, le message est perdu silencieusement et l'exécution du script continue. Les arguments d'un message filtré ne sont pas convertis par `tostring`.

Format de sortie

Sans couleur, une ligne suit ce format :

```
YYYY-MM-DD HH:MM:SS [LEVEL] message\n
```

Exemple :

```
2026-07-13 21:45:02 [WARN ] tentative 3 sur 5
```

L'horodatage utilise **l'heure locale** de la machine. Il n'inclut ni fuseau, ni millisecondes. Les labels occupent cinq caractères : `[TRACE]`, `[DEBUG]`, `[INFO ]`, `[WARN ]`, `[ERROR]`.

Seuil

```
log.set_level(level)
local level = log.get_level()
```

`set_level` exige exactement un argument et ne renvoie aucune valeur. `get_level` n'accepte aucun argument et renvoie le seuil courant.

Le niveau peut être :

- un nom parmi `"trace"`, `"debug"`, `"info"`, `"warn"`, `"error"`, sans distinction de casse ;

- un nombre fini, ce qui permet un seuil intermédiaire comme 25.5.

```
log.set_level("DEBUG")
log.set_level(log.WARN)
log.set_level(25.5)
```

Les chaînes numériques ne sont pas converties et les espaces ne sont pas retirés : "30" et " info" sont refusés. NaN, +inf et -inf sont également refusés.

Destination

```
log.set_output(out)
local out = log.get_output()
```

La destination initiale est `io.stderr`.

`set_output` exige exactement une table ou un userdata exposant une méthode `write` callable. Une méthode fournie par `__index` est acceptée. La méthode est ensuite appelée avec la syntaxe `out:write(line)`.

```
local file, err = io.open("app.log", "a")
if not file then
    log.error("ouverture impossible :", err)
else
    log.set_output(file)
end
```

Le module :

- n'écrit rien pendant `set_output` ;
- ne ferme jamais la destination ;
- ne la vide jamais avec `flush` ;
- ignore la valeur renvoyée par `write` ;
- avale les erreurs d'écriture pendant l'émission.

Un fichier fermé possède encore une méthode `write` et peut donc être installé ; les écritures suivantes échoueront silencieusement. La gestion de la durée de vie du sink appartient au script.

`get_output` n'accepte aucun argument et renvoie exactement l'objet installé.

Couleurs ANSI

```
log.set_color(true)
local enabled = log.get_color()
```

Les couleurs sont désactivées par défaut. `set_color` exige exactement un booléen et ne renvoie aucune valeur. `get_color` n'accepte aucun argument et renvoie un booléen.

Quand les couleurs sont actives :

- `trace` : ANSI dim ;
- `debug` : cyan ;
- `info` : aucune couleur ;
- `warn` : jaune ;
- `error` : rouge.

Le reset ANSI est écrit avant le `\n`. Le module ne détecte pas si la sortie est un terminal : l'activation est toujours explicite.

État du module

`require("logging")` renvoie la même table tant que `package.loaded.logging` n'est pas supprimé. Le seuil, le sink et le réglage de couleur sont donc partagés par tous les utilisateurs du module dans **un même état Lua**.

Chaque worker Babet possède son propre état Lua : ses réglages de logging sont indépendants de ceux du thread principal et des autres workers.

Les valeurs initiales sont :

```
log.set_level(log.INFO)
log.set_output(io.stderr)
log.set_color(false)
```

Le module expose également :

```
log._VERSION      -- "babet logging 1.1.0"
log._DESCRIPTION  -- description textuelle
```

Contrat d'erreur

Les setters et getters lèvent une erreur Lua en cas de mauvaise arité ou de valeur invalide. Ils ne renvoient pas (`nil`, `err`).

Les fonctions `trace`, `debug`, `info`, `warn` et `error`, elles, ne lèvent jamais et ne renvoient aucune valeur.

Limites

Le module ne fournit pas :

- de logger nommé ou hiérarchique ;
- de seuil par module ;
- de destinations multiples ;
- de format personnalisé ou JSON ;
- de fuseau UTC ou de précision sub-seconde ;
- de rotation de fichiers ;
- de buffering ou d'écriture asynchrone ;
- de garde contre un sink récursif qui rappelle lui-même le logger ;
- de détection automatique de terminal.

Pour la rotation, utilisez un outil externe comme `logrotate`. Pour des besoins plus avancés, enveloppez le module ou remplacez ses fonctions dans le script.

SIGNAL — arrêt propre, rechargement et signaux POSIX

`babet.signal` permet au script principal d'installer un callback Lua pour un petit ensemble de signaux POSIX, d'ignorer un signal ou de restaurer son comportement système par défaut.

Le module couvre :

- l'arrêt propre sur `TERM` et `INT` ;
- le rechargement de configuration sur `HUP` ;
- deux signaux applicatifs, `USR1` et `USR2` ;
- le choix explicite d'ignorer ou de restaurer un signal ;
- l'interruption de certains appels bloquants Babet ;
- le dispatch différé et sûr des callbacks dans le thread Lua principal.

Il ne permet pas d'envoyer un signal, de consulter une file de signaux, de configurer un masque POSIX ou d'installer des handlers depuis un worker.

Table des matières du module

- Conventions essentielles
- Vue d'ensemble de l'API
- Signaux pris en charge
- `handle(name, fn)` — installer ou remplacer
- `handle(name, nil)` — retirer un callback
- `ignore(name)` — ignorer
- `default(name)` — restaurer le comportement système
- Exécution des callbacks
 - Thread principal et contexte sûr
 - Ordre de dispatch
 - Coalescence des occurrences
 - Erreurs du callback
- Interaction avec les appels bloquants
- Interaction avec `debug.sethook`
- Exemples complets
 - Arrêt propre sur `TERM` et `INT`
 - Recharger une configuration avec `HUP`
 - Interrompre une attente socket
 - Ignorer temporairement `PIPE`
 - Observer les erreurs du callback
- Contrat d'erreur
- Workers, concurrence et limites
- Fonctionnalités absentes

Conventions essentielles

Les succès renvoient une seule valeur

Les trois fonctions renvoient uniquement le booléen `true` en cas de succès. Elles ne renvoient pas un second `nil`.

```
local ok = babet.signal.ignore("PIPE")
assert(ok == true)
```

Une panne rare de `sigaction(2)` renvoie en revanche deux valeurs :

```
local ok, err = babet.signal.ignore("PIPE")
if not ok then
    io.stderr:write(err, "\n")
end
```

Les noms sont stricts

Un nom de signal doit être une véritable chaîne Lua, sans octet `NUL`, dans la casse indiquée par la documentation.

```
assert(babet.signal.handle("TERM", function() end))
```

Les formes suivantes sont invalides :

```
babet.signal.handle("term", function() end) -- casse incorrecte
babet.signal.handle(15, function() end)     -- nombre, pas string
babet.signal.handle("TERM\0X", function() end)
```

Un callback ne reçoit aucun argument

Le callback est appelé sans paramètre. Installe une fonction différente pour chaque signal lorsque le code doit savoir lequel est arrivé.

```
babet.signal.handle("USR1", function()
    print("USR1")
end)
```

Vue d'ensemble de l'API

```
local ok, err = babet.signal.handle(name, callback_or_nil)
local ok, err = babet.signal.ignore(name)
local ok, err = babet.signal.default(name)
```

Fonction	Succès	Échec système rare	Mauvais appel
handle(name, fn)	true	(nil, err)	erreur Lua levée
handle(name, nil)	true	(nil, err)	erreur Lua levée
ignore(name)	true	(nil, err)	erreur Lua levée
default(name)	true	(nil, err)	erreur Lua levée

handle(name) sans deuxième argument est volontairement refusé. Il faut écrire explicitement handle(name, nil) pour retirer un callback.

Signaux pris en charge

Nom Babet	Signal POSIX	Usage courant	Comportement système usuel
"TERM"	SIGTERM	demande d'arrêt propre, notamment par systemd	termine le processus
"INT"	SIGINT	interruption clavier, généralement Ctrl-C	termine le processus
"HUP"	SIGHUP	rechargement d'une configuration	termine souvent le processus
"USR1"	SIGUSR1	événement applicatif libre	termine le processus
"USR2"	SIGUSR2	second événement applicatif libre	termine le processus
"PIPE"	SIGPIPE	écriture sur un pipe ou socket fermé	termine le processus

Les noms KILL et STOP ne peuvent pas être exposés : POSIX interdit de les intercepter ou de les ignorer.

Les signaux dangereux ou réservés à d'autres mécanismes internes — par exemple SEGV, BUS, FPE, ILL, CHLD et ALRM — ne font pas partie de la liste blanche.

handle(name, fn) — installer ou remplacer

```
local ok, err = babet.signal.handle(name, fn)
```

- name est l'un des six noms supportés ;
- fn doit être une fonction Lua ;
- le handler POSIX précédent est remplacé ;
- un callback Lua déjà enregistré pour le même signal est remplacé ;
- le callback est conservé dans la registry Lua tant qu'il n'est pas retiré.

```

local stopping = false

assert(babet.signal.handle("TERM", function()
    stopping = true
end))

```

Remplacer le callback :

```

assert(babet.signal.handle("USR1", function()
    print("première version")
end))

assert(babet.signal.handle("USR1", function()
    print("nouvelle version")
end))

```

Après le second appel, seule la nouvelle fonction peut être exécutée.

handle(name, nil) — retirer un callback

```

local ok, err = babet.signal.handle(name, nil)

```

Cette forme :

1. efface le callback Lua associé ;
2. restaure SIG_DFL, le comportement système par défaut.

```

assert(babet.signal.handle("HUP", reload_config))

```

```

-- Plus tard : retrait explicite.
assert(babet.signal.handle("HUP", nil))

```

handle("TERM") sans second argument n'est **pas** un raccourci : il lève une erreur, afin d'éviter une désinstallation accidentelle due à un argument oublié.

ignore(name) — ignorer

```

local ok, err = babet.signal.ignore(name)

```

ignore installe SIG_IGN et efface tout callback Lua précédemment associé. Le noyau ignore ensuite le signal.

```

assert(babet.signal.ignore("PIPE"))

```

Cette forme est utile lorsqu'une application préfère recevoir les erreurs d'écriture normales plutôt que d'être terminée par SIGPIPE.

L'ignorance reste active jusqu'à un nouvel appel à handle, default ou handle(name, nil) pour ce même signal.

default(name) — restaurer le comportement système

```

local ok, err = babet.signal.default(name)

```

default installe SIG_DFL et efface tout callback Lua. Pour TERM, INT, HUP, USR1, USR2 et généralement PIPE, le prochain signal correspondant terminera le processus selon les règles du système.

```

assert(babet.signal.ignore("PIPE"))
-- ... section où SIGPIPE doit être ignoré ...
assert(babet.signal.default("PIPE"))

```

handle(name, nil) et default(name) ont le même effet système. default est plus explicite lorsqu'aucun callback n'est en jeu.

Exécution des callbacks

Le vrai handler POSIX n'appelle jamais Lua. Il se contente de poser un flag de type `sig_atomic_t`, opération compatible avec le contexte asynchrone d'un signal.

Le callback Lua est exécuté plus tard, depuis un point sûr :

1. un signal supporté arrive ;
2. le handler C marque ce signal comme en attente ;
3. le thread principal remarque le flag depuis le hook d'instructions ou la sortie d'un appel bloquant interrompu ;
4. Babet remet le flag à zéro ;
5. Babet appelle le callback Lua avec zéro argument.

Thread principal et contexte sûr

Le callback tourne toujours dans l'état Lua principal, jamais dans le handler POSIX et jamais dans un worker. Il peut donc utiliser normalement Lua et les modules Babet.

```
babet.signal.handle("USR1", function()  
    -- Ceci est exécuté dans un contexte Lua normal.  
    local now = babet.time.now()  
    print("USR1 à", now)  
end)
```

Cette liberté ne signifie pas qu'un callback doive être long. Une fonction courte qui positionne un booléen ou enfile une action reste plus simple à raisonner.

Ordre de dispatch

Lorsque plusieurs signaux différents sont en attente au même point de dispatch, Babet les traite dans l'ordre fixe de sa liste blanche :

1. TERM ;
2. INT ;
3. HUP ;
4. USR1 ;
5. USR2 ;
6. PIPE.

Ce n'est ni l'ordre d'enregistrement des callbacks ni une garantie de l'ordre exact d'arrivée au niveau du noyau.

Par exemple, si USR1 puis TERM arrivent avant le prochain dispatch, le callback TERM est exécuté avant le callback USR1.

Coalescence des occurrences

Babet maintient un flag par type de signal, pas un compteur ni une file. Plusieurs occurrences identiques reçues avant le prochain dispatch sont donc coalescées en un seul appel du callback.

USR1, USR1, USR1 avant le dispatch -> un appel du callback USR1

Un nouveau signal identique reçu **pendant** l'exécution du callback peut reposer le flag et être traité lors d'un dispatch ultérieur.

N'utilise donc pas ce module comme compteur fiable d'événements. Pour ne perdre aucune occurrence métier, utilise une queue, un pipe, une socket ou un autre mécanisme de messages.

Erreurs du callback

Babet appelle le callback avec `lua_pcall`. Une erreur non attrapée est retirée de la pile et silencieusement ignorée ; elle ne remonte pas au code qui était en cours d'exécution.

```
babet.signal.handle("USR1", function()
    error("invisible pour l'appelant")
end)
```

Pour journaliser une erreur, protège explicitement le corps :

```
babet.signal.handle("USR1", function()
    local ok, err = pcall(function()
        refresh_metrics()
    end)
    if not ok then
        io.stderr:write("USR1: ", tostring(err), "\n")
    end
end)
```

Interaction avec les appels bloquants

Babet installe les handlers sans `SA_RESTART`. Certains appels bloquants internes peuvent donc être interrompus par un signal géré. Lorsque le thread principal constate qu'un signal Babet est en attente, il dispatche les callbacks puis renvoie généralement une erreur typée "interrupted".

Les familles actuellement intégrées à ce mécanisme comprennent notamment :

- `babet.sleep(...)` ;
- `watcher:read(...)` du module `Inotify` ;
- les attentes TCP : connexion, acceptation et réception ;
- les handshakes et attentes TLS.

Exemple :

```
local data, err = peer:recv(4096, 30)
if not data then
    if err == "interrupted" then
        -- Un callback a déjà été dispatché : re-vérifier l'état du programme.
    elseif err == "timeout" then
        -- Aucun octet reçu avant l'échéance.
    else
        -- Fermeture ou erreur réseau.
    end
end
```

Toutes les fonctions Babet ne sont pas interruptibles par ce mécanisme. `babet.exec`, par exemple, applique son propre timeout et n'expose pas une sortie "interrupted" sur signal géré.

Un syscall peut aussi recevoir un `EINTR` sans qu'un signal géré par Babet soit en attente. Dans ce cas, les boucles internes reprennent normalement leur attente au lieu de produire un faux "interrupted".

Interaction avec `debug.sethook`

En dehors des fonctions bloquantes intégrées, Babet utilise un hook Lua de type `count`, déclenché environ toutes les 10 000 instructions, pour vérifier les flags en attente.

Lua ne permet qu'un hook actif par état. Par conséquent :

- le premier `signal.handle(...)` remplace un hook utilisateur déjà installé ;
- un appel ultérieur à `debug.sethook(...)` remplace le hook de Babet ;

- après ce remplacement, les callbacks ne seront plus dispatchés par le compte d'instructions, mais pourront encore l'être à la sortie des appels bloquants intégrés ;
- retirer tous les callbacks avec `handle(name, nil)`, `ignore(name)` ou `default(name)` ne désinstalle pas le hook de Babet et ne restaure pas un hook utilisateur précédemment remplacé. Le hook reste actif, mais ne fait rien tant qu'aucun callback Babet n'est enregistré.

Évite de combiner `babet.signal` et un hook de debug personnalisé dans le même état Lua, sauf si cette interaction est volontaire et maîtrisée.

Exemples complets

Arrêt propre sur TERM et INT

```
local running = true

local function request_stop()
    running = false
end

assert(babet.signal.handle("TERM", request_stop))
assert(babet.signal.handle("INT", request_stop))

while running do
    do_one_iteration()

    local slept, err = babet.sleep(250, "ms")
    if not slept and err ~= "interrupted" then
        io.stderr:write(tostring(err), "\n")
        break
    end
    -- En cas de "interrupted", le callback a déjà été exécuté et
    -- la boucle re-vérifie running.
end

close_resources()
```

Pour une boucle réseau, utilise directement le retour "interrupted" de la socket plutôt qu'un `pcall`.

Recharger une configuration avec HUP

Il est souvent préférable que le callback ne fasse que poser un drapeau. Le chargement complet s'effectue ensuite dans la boucle normale.

```
local running = true
local reload_requested = false

assert(babet.signal.handle("TERM", function()
    running = false
end))

assert(babet.signal.handle("HUP", function()
    reload_requested = true
end))

while running do
    if reload_requested then
        reload_requested = false
        local new_config, err = load_config()
```

```

        if new_config then
            config = new_config
        else
            io.stderr:write("reload: ", err, "\n")
        end
    end

    do_one_iteration(config)
end

```

Interrompre une attente socket

```

local running = true

assert(babet.signal.handle("TERM", function()
    running = false
end))

while running do
    local client, err = server:accept(60)
    if client then
        serve(client)
    elseif err == "interrupted" then
        -- Le callback TERM a pu mettre running à false.
    elseif err ~= "timeout" then
        io.stderr:write("accept: ", err, "\n")
    end
end

server:close()

```

Ignorer temporairement PIPE

```

assert(babet.signal.ignore("PIPE"))

local sent, err = socket:send(payload)
if not sent then
    io.stderr:write("send: ", err, "\n")
end

```

```

assert(babet.signal.default("PIPE"))

```

Attention : restaurer SIGPIPE à son comportement par défaut signifie qu'une écriture ultérieure sur un pipe fermé peut terminer le processus.

Observer les erreurs du callback

```

local function safe_handler(label, fn)
    return function()
        local ok, err = xpcall(fn, debug.traceback)
        if not ok then
            io.stderr:write(label, ": ", tostring(err), "\n")
        end
    end
end

assert(babet.signal.handle("USR2", safe_handler("USR2", function()

```

```

        rotate_logs()
end)))

```

Contrat d'erreur

Erreurs Lua levées

Les situations suivantes lèvent immédiatement une erreur Lua :

- appel depuis un worker ;
- name absent, non string ou contenant un NUL ;
- nom non supporté ou casse incorrecte ;
- handle sans deuxième argument ;
- callback différent d'une fonction ou de nil.

```

local ok, err = pcall(function()
    babet.signal.handle("TERM")
end)
assert(not ok)

```

Le nom non supporté lève également une erreur ; il ne produit pas (nil, err).

```

local ok, err = pcall(function()
    babet.signal.ignore("QUIT")
end)

```

Erreur système renvoyée

Si sigaction(2) échoue, la fonction renvoie :

```

nil, "signal: sigaction failed: ..."

```

Ce cas est rare avec les six signaux autorisés, mais l'appelant peut conserver la convention (ok, err) lorsqu'il souhaite le traiter.

Workers, concurrence et limites

Les dispositions de signal sont globales au processus, pas propres à un thread. Babet impose donc les règles suivantes :

- handle, ignore et default sont réservés au thread principal ;
- leur utilisation dans un worker lève une erreur explicite ;
- les workers bloquent les six signaux supportés afin qu'ils restent gérés par le thread principal ;
- les workers ne peuvent ni voir ni consommer les flags en attente du parent.

Un worker doit être contrôlé par ses queues de messages, pas par babet.signal :

```

local job = assert(babet.workers.spawn([
    while true do
        local ok, message = worker.recv()
        if not ok then
            return "closed"
        end
        if message == "stop" then
            return "stopped"
        end
    end
]))

```

-- Le handler principal transforme le signal en message applicatif.

```

babet.signal.handle("TERM", function()
    job:send("stop", 0)
end)

```

Ce callback reste volontairement bref. Une queue pleine peut faire échouer l’envoi non bloquant ; une application robuste conserve également un drapeau de secours.

Fonctionnalités absentes

Le module ne fournit pas actuellement :

- `kill(pid, name)` pour envoyer un signal ;
- `list()` pour obtenir les noms supportés ;
- `is_pending()` ou un compteur d’occurrences ;
- `sigprocmask` ou un masque par thread ;
- `signalfd` ;
- les signaux autres que les six noms listés ;
- un hook de debug chaînable avec celui de l’utilisateur.

Pour envoyer un signal depuis Lua, un programme externe reste possible :

```

local result, err = babet.exec("kill", {
    "-TERM",
    tostring(pid),
})
assert(result, err)
assert(result.code == 0, result.stderr)

```

La réception reste soumise aux règles POSIX normales et au modèle de dispatch décrit dans cette page.

SOCKET — client et serveur TCP, flux binaires, lignes et timeouts

`babet.socket` fournit des sockets TCP synchrones pour écrire des clients et serveurs de protocoles personnalisés : IRC, SMTP en clair avant STARTTLS, protocoles ligne par ligne, flux binaires ou petits services internes.

Le module couvre :

- la connexion TCP à un hostname ou une adresse IP ;
- l’écoute sur une interface précise ou sur toutes les interfaces ;
- l’acceptation de clients ;
- l’envoi intégral d’une chaîne binaire ;
- la lecture par blocs, par lignes ou jusqu’à la fermeture distante ;
- des timeouts globaux par socket ou par appel ;
- l’obtention des adresses locale et distante ;
- la fermeture explicite et le nettoyage automatique par le GC ;
- l’interaction avec les signaux gérés par Babet.

Il ne fournit pas UDP, les sockets Unix, l’I/O asynchrone, le multiplexage de milliers de connexions, ni un protocole applicatif prêt à l’emploi. Pour HTTP, utilise [babet.http](#). Pour TLS direct ou STARTTLS, consulte [TLS](#).

Table des matières du module

- [Conventions essentielles](#)
- [Vue d’ensemble de l’API](#)
- [Créer une connexion cliente](#)

- Créer un serveur
- Accepter un client
- Configurer le timeout par défaut
- Envoyer des données
- Lire au plus N octets
- Lire une ligne
- Lire jusqu'à EOF
- Mélanger les méthodes de lecture
- Adresses locale et distante
- Fermer et laisser le GC nettoyer
- Signaux et interruptions
- Exemples complets
- Contrat d'erreur
- Sécurité, performances et limites

Conventions essentielles

TCP synchrone et bloquant

Chaque appel s'exécute dans le thread courant. Sans timeout, connect, accept, send et les lectures peuvent attendre indéfiniment.

Pour une application qui gère plusieurs connexions indépendantes, utilise des [workers](#) ou conçois une boucle autour de plusieurs processus. Babet n'expose pas select, poll ou epoll directement à Lua.

Chaînes binaires

Les données envoyées et reçues sont des chaînes Lua binary-safe. Elles peuvent contenir des octets NUL :

```
local sent = assert(sock:send("AB\0CD"))
local data = assert(sock:recv(5, 2))
assert(#data == 5 and data:byte(3) == 0)
```

Les hostnames utilisent en revanche une chaîne C et ne peuvent pas contenir de NUL.

Timeouts

Les timeouts sont exprimés en **secondes** et peuvent être fractionnaires.

- nil ou argument omis : utilise le timeout par défaut du socket ;
- 0 : attente infinie ;
- nombre positif : budget global pour l'appel ;
- valeur positive inférieure à 1 ms : arrondie à 1 ms ;
- nombre négatif, NaN, infini ou trop grand : erreur.

Une deadline est créée une seule fois par appel. Elle n'est pas renouvelée à chaque octet ou à chaque tentative interne.

Valeurs de retour

Une création ou une lecture réussie renvoie la valeur utile et nil comme seconde valeur implicite :

```
local sock, err = babet.socket.connect("127.0.0.1", 9000, 5)
local data, err = sock:recv(4096, 2)
```

Les actions set_timeout, starttls et close renvoient exactement :

```
true, nil
```

Une erreur d'usage positionnelle lève une erreur Lua. Une erreur de valeur, de transport ou de timeout renvoie généralement (nil, err).

Vue d'ensemble de l'API

Constructeurs

```
local sock, err = babet.socket.connect(host, port, timeout?)
local server, err = babet.socket.listen(host, port, backlog?)
```

Argument	Type	Défaut	Rôle
host de connect	string stricte non vide	obligatoire	hostname ou adresse IP
host de listen	string stricte	obligatoire	interface ; "" signifie toutes les interfaces
port	entier Lua 0..65535	obligatoire	port TCP ; 0 est utile avec listen pour laisser le noyau choisir
timeout	nombre fini >= 0	0	timeout de connexion en secondes
backlog	entier Lua 1..INT_MAX	16	taille demandée de la file d'attente

Les chaînes numériques comme "443" et les floats comme 443.0 sont refusés pour port et backlog, même si leur valeur mathématique est entière.

Méthodes communes

```
local n, err = sock:send(data)
local data, err = sock:recv(count, timeout?)
local line, err, partial = sock:recv_line(timeout?)
local data, err = sock:recv_all(timeout?, max_bytes?)
local ok, err = sock:set_timeout(seconds)
local addr, err = sock:peer()
local addr, err = sock:sockname()
local ok, err = sock:close()
```

Méthode d'un socket d'écoute

```
local client, err = server:accept(timeout?)
```

Un socket d'écoute refuse send, recv, recv_line, recv_all, peer et starttls. Un socket connecté refuse accept.

babet.socket.connect(host, port, timeout?)

Ouvre une connexion TCP cliente.

```
local sock, err = babet.socket.connect("example.net", 9000, 5)
if not sock then
    error(err)
end
```

Résolution et adresses multiples

Babet appelle `getaddrinfo()` puis essaie les adresses retournées dans l'ordre. Le timeout est partagé entre toutes les tentatives IPv4/IPv6 : ce n'est pas un nouveau budget par adresse.

La résolution DNS elle-même est synchrone et se déroule avant cette deadline. Un résolveur lent peut donc faire durer l'appel plus longtemps que `timeout`. Pour éviter le DNS, utilise une adresse numérique :

```
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 2))
```

Timeout de connexion non conservé

Le troisième argument borne uniquement la connexion. Le socket créé possède un timeout d'I/O par défaut de 0, donc infini.

```
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 2))
assert(sock:set_timeout(5)) -- nécessaire pour les I/O suivantes
```

Erreurs typiques

- hostname vide ou contenant un NUL : (nil, err) ;
- port hors plage : (nil, err) ;
- mauvais type positionnel : erreur Lua ;
- refus de connexion, DNS, route ou autre erreur système : (nil, "socket: ...") ;
- expiration : (nil, "timeout") ;
- signal géré pendant l'attente : (nil, "interrupted").

`babet.socket.listen(host, port, backlog?)`

Crée un socket TCP d'écoute avec `SO_REUSEADDR`.

Écouter uniquement en loopback

```
local server = assert(babet.socket.listen("127.0.0.1", 9000))
```

Cette forme n'accepte pas les connexions provenant des autres interfaces.

Écouter sur toutes les interfaces

```
local server = assert(babet.socket.listen("", 9000))
```

Une chaîne vide active `AI_PASSIVE`. Selon l'adresse sélectionnée par le système, le socket peut être IPv4 ou IPv6. Pour imposer une famille, fournis explicitement `"0.0.0.0"` ou `:::`.

Écouter sur toutes les interfaces expose le service au réseau. Ajoute une authentification et des règles de pare-feu adaptées.

Demander un port libre au noyau

```
local server = assert(babet.socket.listen("127.0.0.1", 0))
local local_addr = assert(server:sockname())
print(local_addr.host, local_addr.port)
```

C'est la méthode recommandée pour les tests : elle évite les collisions de ports.

Backlog

```
local server = assert(babet.socket.listen("127.0.0.1", 9000, 128))
```

Le backlog est une demande faite au noyau. Le système peut le plafonner. Ce n'est ni un nombre maximal de clients simultanés ni une limite applicative.

server:accept(timeout?)

Attend et accepte une connexion cliente.

```
local client, err = server:accept(10)
if not client then
    if err == "timeout" then
        print("aucun client")
    else
        error(err)
    end
end
```

Le timeout positionnel remplace celui défini par `server:set_timeout()` pour cet appel uniquement.

Le socket accepté :

- est un socket connecté, pas un socket d'écoute ;
- possède un timeout d'I/O par défaut de 0 ;
- n'hérite donc pas du timeout du serveur ;
- est fermé automatiquement à l'exécution d'un processus enfant grâce à `CLOEXEC`.

```
local client = assert(server:accept(10))
assert(client:set_timeout(5))
```

sock:set_timeout(seconds)

Définit le timeout par défaut des opérations suivantes :

- send ;
- recv ;
- recv_line ;
- recv_all ;
- accept sur un serveur.

```
assert(sock:set_timeout(3.5))
```

0 rétablit les attentes infinies :

```
assert(sock:set_timeout(0))
```

Ce timeout ne s'applique pas rétroactivement à `connect`, ni au handshake de `starttls`. `starttls` utilise uniquement `opts.timeout`.

Une valeur positionnelle sur `recv`, `recv_line`, `recv_all` ou `accept` prime sur le défaut :

```
assert(sock:set_timeout(10))
local data, err = sock:recv(4096, 0.2) -- 200 ms pour cet appel
local data2, err2 = sock:recv(4096)    -- à nouveau 10 s
local data3, err3 = sock:recv(4096, 0) -- infini pour cet appel
```

sock:send(data)

Envoie la chaîne complète.

```

local count, err = sock:send("PING\r\n")
assert(count, err)
assert(count == 6)

```

Contrairement à l'appel POSIX `send(2)`, Babet boucle sur les écritures partielles. En succès, `count == #data`.

La chaîne peut être vide ou binaire :

```

assert(sock:send(""))
assert(sock:send("AB\0CD") == 5)

```

`send` n'accepte pas de timeout positionnel. Il utilise le timeout défini par `set_timeout()`. Sur TLS, les appels `SSL_write` sont découpés en blocs sûrs lorsque nécessaire.

Une erreur après l'envoi de certains octets ne fournit pas le nombre partiel à Lua. Pour un protocole nécessitant une reprise transactionnelle, ajoute un mécanisme applicatif d'identifiant ou d'acquittement.

sock:recv(count, timeout?)

Lit **au plus** `count` octets et renvoie dès qu'au moins un octet est disponible.

```

local chunk, err = sock:recv(4096, 5)

```

Ce n'est pas un équivalent de « lire exactement N octets ». Une lecture peut renvoyer moins que `count` sans que la connexion soit fermée.

`count` doit être un entier Lua strict compris entre 1 et 16 MiB.

Lire exactement N octets

```

local function recv_exact(sock, wanted, timeout)
    local chunks = {}
    local total = 0

    while total < wanted do
        local chunk, err = sock:recv(wanted - total, timeout)
        if not chunk then
            return nil, err
        end
        chunks[#chunks + 1] = chunk
        total = total + #chunk
    end

    return table.concat(chunks)
end

```

```

local header = assert(recv_exact(sock, 12, 5))

```

Attention : dans ce helper, `timeout` est appliqué séparément à chaque appel. Pour imposer un budget global applicatif, calcule une échéance avec `babet.monotonic()` et passe le temps restant.

EOF et timeout

- fermeture distante avant tout octet : `(nil, "closed")` ;
- expiration : `(nil, "timeout")` ;
- interruption par signal : `(nil, "interrupted")`.

sock:recv_line(timeout?)

Lit jusqu'au prochain LF (\n) et renvoie la ligne sans le LF final.

```
local line, err = sock:recv_line(10)
```

Si le LF est précédé d'un CR, le CR est aussi retiré :

```
hello\n    -> "hello"  
hello\r\n -> "hello"
```

Un CR situé ailleurs dans la ligne est conservé.

Limite de ligne

Une ligne est limitée à **8 MiB**. Une ligne plus longue renvoie une erreur `line too long`. Les octets de cette ligne surdimensionnée ne sont pas exposés comme résultat partiel ; ferme généralement la connexion, car le protocole est alors désynchronisé.

EOF avant LF

Si le pair ferme la connexion avant le LF, la méthode renvoie trois valeurs :

```
local line, err, partial = sock:recv_line(5)  
if not line and err == "closed" then  
    print("fragment final :", partial)  
end
```

`partial` est toujours une chaîne, éventuellement vide.

Timeout avant LF

Lors d'un timeout ou d'une interruption, les octets déjà lus sont conservés internement. Ils ne sont pas renvoyés comme troisième valeur, mais seront livrés en premier lors de la lecture suivante.

```
local line, err = sock:recv_line(0.2)  
if not line and err == "timeout" then  
    -- un prochain recv_line(), recv() ou recv_all() reprend sans perte  
end
```

sock:recv_all(timeout?, max_bytes?)

Lit jusqu'à la fermeture distante et renvoie tout le flux concaténé.

```
local body, err = sock:recv_all(10)
```

Ici, EOF est le terminateur normal et constitue un succès. Cette fonction est adaptée aux protocoles où la fermeture délimite le message. Elle est inadaptée à une connexion persistante qui reste ouverte après la réponse.

Limite mémoire

```
local body, err = sock:recv_all(10, 4 * 1024 * 1024)
```

- défaut : **64 MiB** ;
- valeur minimale : 1 ;
- maximum configurable : **2 GiB** ;
- type : entier Lua strict.

Au dépassement, le résultat est `(nil, err)` sans chaîne partielle. Les octets déjà consommés restent cependant dans le buffer interne. Un nouvel appel avec une limite supérieure peut reprendre le flux :

```

local body, err = sock:recv_all(2, 1024)
if not body and err:find("max_bytes", 1, true) then
    body = assert(sock:recv_all(2, 4096))
end

```

Le même principe s'applique après un timeout ou une interruption : les octets accumulés ne sont pas perdus.

Mélanger `recv`, `recv_line` et `recv_all`

Les trois méthodes partagent un buffer d'octets en attente. Cela garantit que changer de méthode après un timeout ne réordonne pas le flux.

```

-- Le pair a envoyé "abc" sans LF.
local line, err = sock:recv_line(0.1)
assert(line == nil and err == "timeout")

-- Les trois octets déjà consommés par recv_line restent disponibles.
local prefix = assert(sock:recv(3, 1))
assert(prefix == "abc")

```

La règle reste toutefois simple : utilise de préférence une seule stratégie de framing par phase de protocole. Mélanger volontairement les lectures peut être utile pour lire un header ligne par ligne puis un corps de taille connue, mais le code applicatif doit savoir exactement où se trouve la frontière.

Avant `starttls`, tout plaintext tamponné doit être consommé. Babet refuse le handshake si ce buffer n'est pas vide, afin de ne pas confondre des octets en clair avec des records TLS.

`sock:peer()` et `sock:sockname()`

Les deux méthodes renvoient une table :

```

{
    host = "127.0.0.1",
    port = 9000,
}

```

- `peer()` : endpoint distant d'un socket connecté ;
- `sockname()` : endpoint local du socket.

```

local remote = assert(sock:peer())
local local_addr = assert(sock:sockname())
print(remote.host, remote.port)
print(local_addr.host, local_addr.port)

```

Les adresses sont numériques : Babet ne réalise pas de reverse DNS. `peer()` est refusé sur un socket d'écoute. `sockname()` fonctionne aussi sur un serveur et permet de retrouver le port choisi après `listen(..., 0)`.

`sock:close()` et nettoyage automatique

`close()` est idempotent :

```

assert(sock:close())
assert(sock:close())

```

Après fermeture, les autres méthodes renvoient une erreur indiquant que le socket est fermé.

Le userdata possède le descripteur et l'éventuelle session TLS. Si le script oublie `close()`, le garbage collector ferme finalement les ressources. N'utilise pas ce mécanisme pour le contrôle normal du cycle de vie : le moment du GC n'est pas déterministe.

Les sockets créés et acceptés sont marqués close-on-exec. Un programme lancé par `babet.exec` n'hérite pas silencieusement des connexions ou du port d'écoute.

Signaux et interruptions

Lorsque l'attente noyau est interrompue par un signal enregistré avec `babet.signal`, les opérations concernées peuvent renvoyer :

```
nil, "interrupted"
```

Le callback Lua est dispatché avant le retour. Les octets déjà retirés du flux par `recv_line` ou `recv_all` restent tamponnés.

```
local stopping = false
babet.signal.handle("TERM", function()
    stopping = true
end)

while not stopping do
    local client, err = server:accept(1)
    if client then
        -- traiter le client
    elseif err ~= "timeout" and err ~= "interrupted" then
        error(err)
    end
end
```

Consulte la page [SIGNAL](#) pour l'ordre, la coalescence et les appels effectivement interruptibles.

Exemples complets

Petit serveur ligne par ligne

```
local S = babet.socket
local server = assert(S.listen("127.0.0.1", 9000, 64))
assert(server:set_timeout(1))

print("écoute sur 127.0.0.1:9000")

while true do
    local client, err = server:accept()
    if client then
        assert(client:set_timeout(30))

        local line, read_err = client:recv_line()
        if line then
            assert(client:send("echo: " .. line .. "\n"))
        elseif read_err ~= "closed" then
            io.stderr:write(read_err, "\n")
        end

        client:close()
    elseif err ~= "timeout" then
        error(err)
    end
end
```

Client avec longueur préfixée

```
local function recv_exact(sock, n, timeout)
    local out, size = {}, 0
    while size < n do
        local part, err = sock:recv(n - size, timeout)
        if not part then return nil, err end
        out[#out + 1] = part
        size = size + #part
    end
    return table.concat(out)
end

local sock = assert(babet.socket.connect("127.0.0.1", 9000, 3))
assert(sock:set_timeout(5))

assert(sock:send(string.pack(">I4", 5) .. "hello"))
local raw_len = assert(recv_exact(sock, 4, 5))
local len = string.unpack(">I4", raw_len)
local payload = assert(recv_exact(sock, len, 5))
print(payload)
sock:close()
```

Lire une réponse délimitée par EOF

```
local sock = assert(babet.socket.connect("127.0.0.1", 9001, 3))
assert(sock:send("request\n"))
local response = assert(sock:recv_all(10, 8 * 1024 * 1024))
print(#response)
```

Cette forme suppose que le serveur ferme sa moitié d'écriture pour signaler la fin.

Contrat d'erreur

Erreurs Lua levées

Les mauvais types positionnels lèvent une erreur Lua, par exemple :

```
babet.socket.connect("localhost", "9000") -- port string
sock:send(42)                             -- données non string
sock:recv(4096.0)                          -- float, pas entier Lua
sock:set_timeout("5")                     -- string, pas nombre
```

Utilise `pcall` uniquement lorsque l'appel reçoit des valeurs dont le type n'a pas déjà été validé.

Erreurs renvoyées

Les valeurs hors plage et les erreurs d'exécution renvoient `(nil, err)` :

- port négatif ou supérieur à 65535 ;
- backlog nul ou trop grand ;
- timeout invalide ;
- count de recv hors limite ;
- max_bytes invalide ;
- DNS, connexion, lecture, écriture ou fermeture ;
- méthode incompatible avec le type de socket ;
- socket déjà fermé ;
- "timeout", "closed" ou "interrupted" lorsque ces états sont spécifiquement identifiés.

Ne compare une erreur complète que pour les chaînes stables et typées comme "timeout", "closed" et "interrupted". Les messages système peuvent varier selon la libc et le noyau.

Sécurité, performances et limites

- **Pas d'authentification.** Un socket TCP brut ne prouve pas l'identité du pair et ne chiffre rien.
- **Pas de limite implicite de protocole.** `recv_line` et `recv_all` possèdent des caps, mais ton protocole doit aussi valider tailles, commandes et fréquences.
- **Timeouts recommandés.** Configure un timeout sur toute donnée contrôlée par un pair distant pour éviter les connexions lentes infinies.
- **Écoute réseau.** Préfère `loopback` lorsque le service n'a pas besoin d'être public.
- **Résolution DNS hors deadline.** La partie `getaddrinfo()` reste synchrone.
- **Un appel par thread.** N'utilise pas le même `userdata` simultanément depuis plusieurs états Lua. Les `userdata` ne sont de toute façon pas transportables par WORKERS.
- **Ordre TCP, pas messages.** TCP est un flux ; les frontières de `send` ne sont pas conservées à la réception.
- **Pas de half-close exposé.** Babet n'expose pas `shutdown(SHUT_WR)` ; seule la fermeture complète est disponible.
- **Pas de keepalive configuré.** Les options TCP avancées ne sont pas exposées.

babet.sqlite — base de données SQL embarquée

`babet.sqlite` embarque SQLite 3.53.1 et expose une API Lua complète mais volontairement compacte : connexions, exécution SQL directe, itérateurs paresseux, statements préparés réutilisables, transactions assistées et `bind BLOB` explicite.

Table des matières du module

- [API](#)
 - [Ouverture et fermeture](#)
 - [Exécution directe](#)
 - [Statements préparés réutilisables](#)
 - [Transactions assistées](#)
 - [Paramètres SQL](#)
 - [BLOB explicites](#)
 - [Texte SQL et octets NUL](#)
- [Lecture des lignes et mapping de types](#)
- [Cycle de vie](#)
- [Contrat d'erreur](#)
- [Exemples complets](#)
- [Fonctions non exposées](#)

API

Ouverture et fermeture

Fonction	Renvoie
<code>babet.sqlite.open(path, opts?)</code>	<code>db (userdata) (nil, err)</code>
<code>babet.sqlite.blob(data)</code>	wrapper BLOB opaque
<code>db:close()</code>	<code>(true, nil)</code> — idempotent
<code>db:in_transaction()</code>	<code>boolean (nil, err)</code>

`open` ouvre ou crée une base en lecture/écriture. Le chemin spécial `":memory:"` crée une base uniquement en mémoire, perdue à la fermeture. Le chemin doit être une chaîne sans octet NUL.

opts est une table facultative :

Champ	Type	Défaut
wal	boolean — demande PRAGMA journal_mode=WAL	false
busy_timeout	entier de 0 à 3600000 ms	0 — aucun délai d'attente

wal = true demande le mode WAL, mais SQLite peut conserver un autre mode lorsque WAL n'est pas applicable. C'est notamment le cas de ":memory:".

busy_timeout demande à SQLite de réessayer pendant la durée indiquée lorsque la base est verrouillée. La valeur 0 laisse remonter immédiatement SQLITE_BUSY.

db.in_transaction() renvoie true dès que la connexion se trouve dans une transaction, qu'elle ait été ouverte par db.transaction() ou par un BEGIN SQL manuel.

Exécution directe

Fonction	Renvoie
db.exec(sql)	(true, nil) (nil, err) — plusieurs instructions acceptées
db.exec(sql, params)	(true, nil) (nil, err) — une seule instruction
db.query(sql, params?)	stmt temporaire callable (nil, err)
stmt.close()	(true, nil) — idempotent

db:exec Sans table params, exec accepte plusieurs instructions séparées par des points-virgules. Elles sont préparées et exécutées dans l'ordre. L'exécution s'arrête à la première erreur ; les instructions précédentes restent acquises si elles ne se trouvaient pas dans une transaction explicite.

Avec une table params, une seule instruction est acceptée. Les espaces, points-virgules et commentaires SQL finaux (-- ... ou /* ... */) ne comptent pas comme une seconde instruction.

Un SELECT passé à exec est exécuté jusqu'au bout, mais ses lignes sont ignorées. Une chaîne SQL vide ou composée uniquement de séparateurs et de commentaires est un no-op réussi avec exec(sql) ou exec(sql, {}).

db:query query prépare immédiatement une instruction et renvoie un **itérateur callable à usage unique**. L'instruction n'est exécutée qu'au premier appel :

```
for row in db.query("SELECT id, name FROM users ORDER BY id") do
    print(row.id, row.name)
end
```

Chaque ligne est une table indexée par nom de colonne. L'itérateur renvoie nil une fois épuisé puis finalise automatiquement son statement. stmt.close() permet de le libérer plus tôt. Le ramasse-miettes finalise aussi un itérateur abandonné après un break.

query n'est pas limité aux SELECT. Une instruction DDL ou DML est exécutée au premier appel, puis l'itérateur se termine sans produire de ligne.

Un SQL multi-instructions est refusé. Un SQL vide ou composé seulement de commentaires produit un itérateur déjà épuisé.

Statements préparés réutilisables

Fonction	Renvoie
db:prepare(sql)	prepared (nil, err)
prepared:exec(params?)	(true, nil) (nil, err)
prepared:query(params?)	le même userdata prepared (nil, err)
prepared:reset()	(true, nil) (nil, err)
prepared:close()	(true, nil) — idempotent
prepared:finalize()	alias de close()

db:prepare() accepte exactement **une instruction SQL non vide**. La préparation est immédiate : une table ou une colonne inconnue est donc signalée avant la première exécution.

Le statement peut ensuite être réutilisé autant de fois que nécessaire :

```
local insert = assert(db:prepare(
    "INSERT INTO files(path, digest) VALUES(?, ?)"
))
```

```
for _, file in ipairs(files) do
    assert(insert:exec({ file.path, file.digest }))
end
```

```
assert(insert:finalize())
```

prepared:exec() réinitialise automatiquement le statement avant le bind, l'exécute jusqu'à SQLITE_DONE, puis efface les bindings. Un SELECT est entièrement parcouru mais ses lignes sont ignorées.

prepared:query() réinitialise et bind le statement, puis renvoie **le même userdata**, rendu appeable par sa métatable :

```
local select_by_age = assert(db:prepare([
    SELECT id, name FROM users
    WHERE age >= ?
    ORDER BY id
]))
```

```
for row in select_by_age:query({ 18 }) do
    print(row.id, row.name)
end
```

```
for row in select_by_age:query({ 65 }) do
    print("senior", row.name)
end
```

À l'épuisement naturel, le statement est automatiquement réinitialisé et ses bindings sont effacés. Après un break, trois choix sont possibles :

- appeler prepared:reset() ;
- démarrer un nouveau prepared:query(...) ;
- appeler prepared:exec(...).

Les deux dernières opérations réinitialisent aussi automatiquement l'itération précédente et abandonnent ses lignes restantes. Un statement préparé ne peut avoir qu'une seule itération active à la fois : réutiliser le même userdata dans une boucle imbriquée réinitialise donc la boucle extérieure.

Un appel direct prepared() sans prepared:query() actif renvoie simplement nil et n'exécute jamais le SQL avec des bindings effacés.

`prepared:reset()` interrompt l'exécution courante et efface les bindings. `close()` et `finalize()` finalisent définitivement le statement. Une fois fermé, son appel direct renvoie `nil`, tandis que ses méthodes d'exécution renvoient `(nil, "sqlite: statement closed")`.

Transactions assistées

```
local ok, result = db:transaction(function(tx)
    assert(tx:exec(
        "INSERT INTO accounts(name, balance) VALUES(?, ?)",
        { "alice", 100 }
    ))

    assert(tx:exec(
        "INSERT INTO audit(message) VALUES(?)",
        { "account created" }
    ))

    return "created"
end, "immediate")

assert(ok, result)
print(result) -- "created"
```

Signature :

```
ok, ...callback_results = db:transaction(callback [, mode])
-- ou
nil, err = db:transaction(callback [, mode])
```

Le callback reçoit la connexion `db` comme unique argument. `mode` vaut :

- "deferred" — défaut ;
- "immediate" ;
- "exclusive".

Si le callback se termine normalement, Babet exécute `COMMIT` et renvoie `true`, suivi de toutes les valeurs du callback. Un retour normal `nil` ou `false` **ne demande pas un rollback** : seule une erreur Lua déclenche le rollback automatique.

Les opérations SQLite renvoyant habituellement `(nil, err)`, utilise `assert` dans le callback pour transformer une erreur opérationnelle en erreur Lua :

```
local ok, err = db:transaction(function(tx)
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
end)
```

-- Le second INSERT échoue, assert lève, et les deux INSERT sont annulés.

Une erreur du callback est interceptée : Babet tente `ROLLBACK` puis renvoie `(nil, "sqlite: transaction callback failed: ...")`. Elle n'est pas relancée. Un échec de `COMMIT` renvoie également `(nil, err)` après une tentative de rollback.

Limitations volontaires :

- les appels imbriqués à `db:transaction()` sont refusés ;
- la fonction est refusée si une transaction SQL manuelle est déjà active ;
- `db:close()` est refusé pendant le callback ;
- le callback ne doit pas exécuter lui-même `BEGIN`, `COMMIT` ou `ROLLBACK`.

Les savepoints restent accessibles par SQL brut.

Paramètres SQL

Les formes suivantes sont prises en charge par `exec`, `query` et les statements préparés :

Placeholder	Valeur Lua
?	<code>params[1]</code> , <code>params[2]</code> , etc., selon l'ordre des ?
:name, @name, \$name	<code>params.name</code> — le préfixe est retiré

Les paramètres positionnels et nommés peuvent être mélangés :

```
assert(db:exec(
    "INSERT INTO events VALUES (?, :kind, ?)",
    { 42, 1700000000, kind = "start" }
))
```

La table doit correspondre exactement aux placeholders. Un paramètre manquant, en trop, un index numérique sparse, un index numérique non entier ou une valeur non supportée lève une erreur Lua. Passer explicitement `nil` comme table équivaut à ne pas fournir de paramètres.

Les placeholders numérotés ?NNN ne font pas partie du contrat public.

Types acceptés au bind :

Type Lua	Valeur SQLite
boolean	INTEGER 0 ou 1
integer	INTEGER
number non entier	REAL
string	TEXT, y compris avec des octets NUL
<code>babet.sqlite.blob(data)</code>	BLOB binary-safe

BLOB explicites

Une chaîne Lua ordinaire reste **toujours** bindée avec `sqlite3_bind_text`, même si la colonne possède une affinité BLOB. `babet.sqlite.blob(data)` marque explicitement les mêmes octets comme BLOB :

```
local insert = assert(db:prepare(
    "INSERT INTO assets(name, payload) VALUES(?, ?)"
))

assert(insert:exec({
    "icon",
    babet.sqlite.blob("\x89PNG\r\n\x1a\n...")
}))
```

Le constructeur exige exactement une chaîne Lua et renvoie un userdata opaque. Les octets NUL sont conservés. `babet.sqlite.blob("")` produit bien un BLOB de zéro octet, et non `NULL`.

Le wrapper ne sert qu'au bind ; la lecture d'une colonne BLOB renvoie toujours une chaîne Lua binary-safe.

Il n'existe pas encore de sentinelle pour binder explicitement `NULL`. Utilisez le littéral SQL `NULL` lorsque nécessaire.

Texte SQL et octets NUL

Le texte SQL doit être une chaîne sans octet NUL. SQLite considérerait ce NUL comme une fin de chaîne et pourrait n'exécuter qu'un préfixe ; Babet refuse donc la requête avant toute préparation. La taille du SQL est également limitée à `INT_MAX` octets, conformément à `sqlite3_prepare_v2`.

Cette restriction concerne seulement le **texte SQL**. Les chaînes et BLOB bindés sont binary-safe.

Lecture des lignes et mapping de types

Type SQLite lu	Type Lua
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	string binary-safe, y compris pour un BLOB vide
NULL	clé absente de la table (row.col == nil et absente de pairs)

Si plusieurs colonnes portent le même nom, la dernière colonne gagne. Utilise des alias pour conserver toutes les valeurs.

La fonction SQL `length()` compte les caractères d'un TEXT et peut s'arrêter à un octet NUL. Pour compter ses octets lors d'un diagnostic, utilise par exemple `length(CAST(col AS BLOB))`.

Cycle de vie

`db:close()` est idempotent. Après fermeture, toute nouvelle opération sur la connexion renvoie `(nil, "sqlite: connection closed")`.

Un itérateur ou un statement préparé créé **avant** `db:close()` reste cependant utilisable. Babet emploie `sqlite3_close_v2` : SQLite conserve une connexion « zombie » jusqu'à la finalisation du dernier statement actif.

```
local stmt = assert(db:prepare("SELECT id FROM users ORDER BY id"))
assert(db:close())
```

```
for row in stmt:query() do
    print(row.id)
end
```

```
stmt:finalize()
```

Les itérateurs temporaires de `db:query` et les statements préparés possèdent un `__gc` qui finalise leur handle. Une fermeture explicite reste préférable pour libérer immédiatement verrous et ressources.

Contrat d'erreur

Les erreurs se répartissent en deux catégories :

- les erreurs opérationnelles (`open`, `préparation`, `step`, `transaction`, `connexion fermée`) renvoient généralement `(nil, "sqlite: <description>")` ;
- les mauvais types, les arités invalides et les tables params incorrectes lèvent une erreur Lua.

Les erreurs survenant pendant l'appel d'un itérateur (`db:query` ou `prepared:query`) lèvent une erreur Lua, car le protocole d'itération ne permet pas de renvoyer proprement `(nil, err)` en plus de la fin de séquence.

Sans table params, la présence d'un placeholder est détectée avant l'exécution. Babet refuse de laisser SQLite binder silencieusement NULL.

`db:transaction()` constitue une exception contrôlée : il intercepte les erreurs Lua du callback, effectue le rollback et les convertit en `(nil, err)`.

Exemples complets

Inserts préparés et BLOB

```
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2000,
}))

assert(db:exec([[
    CREATE TABLE IF NOT EXISTS files (
        path TEXT PRIMARY KEY,
        digest BLOB NOT NULL
    )
]]))

local insert = assert(db:prepare([[
    INSERT INTO files(path, digest)
    VALUES(?, ?)
    ON CONFLICT(path) DO UPDATE SET digest = excluded.digest
]]))

for _, file in ipairs(files) do
    assert(insert:exec({
        file.path,
        babet.sqlite.blob(file.digest),
    }))
end

insert:finalize()
```

Requête réutilisable et transaction

```
local by_prefix = assert(db:prepare([[
    SELECT path, digest
    FROM files
    WHERE path LIKE ?
    ORDER BY path
]]))

local ok, count = db:transaction(function(tx)
    local n = 0
    for row in by_prefix:query({ "images/%" }) do
        assert(tx:exec(
            "INSERT INTO audit(path) VALUES(?)",
            { row.path }
        ))
        n = n + 1
    end
    return n
end, "immediate")

assert(ok, count)
print(count, "rows audited")

by_prefix:finalize()
assert(db:close())
```

Fonctions non exposées

Les éléments suivants ne sont pas implémentés :

- une sentinelle `sqlite.NULL` pour binder explicitement `NULL` ;
- `db:last_insert_rowid()` et `db:changes()` ;
- `opts.readonly` et `opts.foreign_keys` ;
- un helper de savepoint imbriqué ;
- l'API de streaming BLOB `sqlite3_blob_open` ;
- l'API de sauvegarde `sqlite3_backup_init`.

`last_insert_rowid`, `changes`, les clés étrangères, les savepoints et `VACUUM INTO` restent accessibles par SQL brut. FTS5 et R-Tree ne sont pas activés dans la compilation embarquée actuelle.

babet strings — découpage de chaînes

Babet expose une seule fonction de manipulation de chaînes : `babet.split`. Elle est enregistrée directement dans la table `babet`, sans sous-namespace, pour conserver l'API historique.

Table des matières du module

- [API](#)
- [Mode séparateur](#)
 - [Limiter le nombre de coupes](#)
- [Mode octets](#)
- [Chaînes binaires et octets NUL](#)
- [Chaîne vide](#)
- [Contrat d'erreur](#)
- [Limites et choix de conception](#)

API

```
local parts = babet.split(s [, sep [, max_splits]])
```

La fonction exige **entre un et trois arguments** et renvoie exactement une valeur : une table dense indexée de 1 à n.

Argument	Type et comportement
s	Chaîne Lua obligatoire. Les nombres ne sont pas convertis implicitement. Les octets NUL sont préservés.
sep	Chaîne Lua facultative. Omission ou chaîne vide : mode octets. Sinon, elle doit contenir exactement un octet , utilisé littéralement.
max_splits	Entier Lua facultatif, supérieur ou égal à -1. -1 signifie illimité, 0 interdit toute coupe. Utilisé uniquement lorsque sep contient un octet.

`nil` n'est pas une valeur de remplacement pour un argument facultatif : `babet.split("abc", nil)` lève une erreur. Pour fournir `max_splits` en mode octets, il faut passer explicitement `""` comme deuxième argument.

Mode séparateur

Lorsque `sep` contient exactement un octet, chaque occurrence de cet octet provoque une coupe. Le séparateur est **littéral** : il ne s'agit ni d'un pattern Lua ni d'une expression régulière.

```
babet.split("a,b,c", ",")
-- { "a", "b", "c" }
```

```
babet.split("a.b.c", ".")
-- { "a", "b", "c" } -- le point n'est pas un joker
```

Les entrées vides sont conservées, y compris en début, entre deux séparateurs et en fin de chaîne :

```
babet.split(",a", ",")
-- { "", "a" }
```

```
babet.split("a,,b", ",")
-- { "a", "", "b" }
```

```
babet.split("a,", ",")
-- { "a", "" }
```

Si le séparateur n'apparaît pas, le résultat contient la chaîne entière :

```
babet.split("hello", ",")
-- { "hello" }
```

Limiter le nombre de coupes

`max_splits` compte les **coupes**, pas le nombre d'éléments produits. Le reste non découpé est placé tel quel dans le dernier élément.

```
babet.split("a,b,c,d", ",", 2)
-- { "a", "b", "c,d" }
```

```
babet.split("a,b,c", ",", 0)
-- { "a,b,c" }
```

```
babet.split("a,b,c", ",", -1)
-- { "a", "b", "c" }
```

Toute valeur supérieure au nombre de séparateurs disponibles produit le même résultat que -1.

Mode octets

Si `sep` est omis ou vaut "", la chaîne est découpée octet par octet. Il n'existe donc **aucun séparateur espace par défaut**.

```
babet.split("ab c")
-- { "a", "b", " ", "c" }
```

```
babet.split("abc", "")
-- { "a", "b", "c" }
```

`max_splits`, lorsqu'il est fourni dans ce mode, est toujours validé mais n'est pas utilisé :

```
babet.split("abc", "", 0)
-- { "a", "b", "c" }
```

Ce mode travaille sur des **octets**, pas sur des points de code Unicode. Un caractère UTF-8 multi-octets est donc séparé en plusieurs chaînes d'un octet :

```
local bytes = babet.split("é")
-- #bytes == 2 en UTF-8
-- table.concat(bytes) == "é"
```

De même, un séparateur UTF-8 multi-octets comme "é" est refusé, car sep doit contenir zéro ou un octet. Pour découper sur une chaîne de plusieurs octets, il faut utiliser du Lua (string.find, string.gmatch, etc.) ou une fonction dédiée.

Chaînes binaires et octets NUL

Le sujet, le séparateur et les éléments produits sont manipulés avec leur longueur Lua exacte. Un octet NUL ne tronque donc rien :

```
local parts = babet.split("a\0b,c", ",")
-- { "a\0b", "c" }
```

```
local parts2 = babet.split("a\0b", "\0")
-- { "a", "b" }
```

Le contenu UTF-8 reste également intact lorsqu'il est séparé par un octet ASCII :

```
babet.split("été:ok", ":")
-- { "été", "ok" }
```

Chaîne vide

Deux comportements historiques différents sont conservés :

```
babet.split("", ",")
-- { "" }
```

```
babet.split("")
-- {}
```

```
babet.split("", "")
-- {}
```

En mode séparateur, zéro séparateur trouvé signifie un élément contenant la chaîne entière, même si elle est vide. En mode octets, une chaîne de zéro octet ne produit aucun élément.

Contrat d'erreur

Toutes les erreurs sont des erreurs Lua levées ; split ne possède pas de chemin de retour (nil, err).

La fonction lève notamment dans les cas suivants :

- moins d'un ou plus de trois arguments ;
- s n'est pas une vraie chaîne Lua ;
- sep est présent mais n'est pas une vraie chaîne Lua ;
- sep contient plus d'un octet ;
- max_splits n'est pas un entier Lua ;
- max_splits est inférieur à -1.

Les conversions implicites nombre-vers-chaîne ne sont pas acceptées :

```
babet.split(123, ",")      -- erreur
babet.split("123", 2)      -- erreur
babet.split("a,b", ",", "1") -- erreur
```

Limites et choix de conception

- Ce n'est pas un parseur CSV : aucune gestion des guillemets, échappements ou lignes n'est effectuée.

- Les séparateurs multi-octets et les patterns Lua ne sont pas pris en charge.
- Les entrées vides sont volontairement conservées.
- Un équivalent `join` n'est pas exposé, car `table.concat` remplit déjà ce rôle dans la bibliothèque standard Lua.

SYS - processus, machine, environnement, exécutables, version et mémoire Lua

Les fonctions de ce chapitre sont exposées directement dans la table globale `babet`. Il n'existe pas de sous-table `babet.sys` : ce namespace plat est historique et fait partie de l'API stable.

Le chapitre SYS couvre :

- l'identité du processus Babet ;
- le nom de la machine et les informations du noyau ;
- la lecture et la modification de variables d'environnement ;
- la recherche d'un programme exécutable ;
- les constantes de version du runtime ;
- la mémoire utilisée par l'état Lua courant.

Le répertoire de travail courant est lui aussi un état global du processus, mais ses fonctions `currentDir` et `chdir` sont documentées dans [FS - Répertoire courant](#), avec les autres fonctions de chemins.

Table des matières du module

- [Conventions générales](#)
- [Vue d'ensemble de l'API](#)
- [Constantes de version](#)
- [Identifier le processus et la machine](#)
 - [pid](#)
 - [hostname](#)
 - [uname](#)
- [Rechercher un exécutable](#)
 - [Recherche dans PATH](#)
 - [Chemin direct](#)
 - [Programme absent ou non exécutable](#)
- [Variables d'environnement](#)
 - [env](#)
 - [setenv](#)
 - [Valeur vide et variable absente](#)
 - [Interaction avec les workers](#)
- [Mémoire de la VM Lua](#)
 - [getMemoryUsage](#)
 - [getDetailedMemoryUsage](#)
 - [Ce qui est mesuré et ce qui ne l'est pas](#)
- [Contrat d'erreur](#)
- [Décisions et limites](#)

Conventions générales

Fonctions plates

Toutes les fonctions s'appellent directement depuis `babet` :

```
local pid = babet.pid()
local home = babet.env("HOME")
```

Les formes suivantes n'existent pas :

```
-- Incorrect : aucune sous-table babet.sys
-- babet.sys.pid()
-- babet.sys.env("HOME")
```

Chaînes strictes

env, setenv et which exigent de véritables chaînes Lua. Les nombres ne sont pas convertis automatiquement.

```
local ok, err = pcall(function()
    return babet.env(42)
end)
```

```
-- ok == false ; err décrit une mauvaise signature
```

Un octet NUL est toujours refusé. Cette règle évite qu'une chaîne Lua soit silencieusement tronquée lorsqu'elle est transmise à une API C.

État global du processus

L'environnement et le répertoire courant appartiennent au **processus entier**, pas à un script ou à un worker particulier.

- setenv modifie l'environnement vu par le runtime et les futurs programmes lancés avec babet.exec ;
- chdir modifie la résolution de tous les chemins relatifs ;
- ces deux mutations sont définitivement interdites après le premier babet.workers.spawn().

Cette restriction évite des courses de données entre threads.

Vue d'ensemble de l'API

Fonction ou constante	Résultat
babet.VERSION	chaîne major.minor.patch du runtime
babet.VERSION_MAJOR	composante majeure entière
babet.VERSION_MINOR	composante mineure entière
babet.VERSION_PATCH	composante patch entière
babet.pid()	PID du processus, sous forme d'integer
babet.hostname()	hostname, ou (nil, err)
babet.uname()	table système, ou (nil, err)
babet.which(name)	chemin d'un exécutable, ou (nil, err)
babet.env(name)	valeur de la variable, ou nil si elle est absente
babet.setenv(name, value)	(true, nil) ou (nil, err)
babet.getMemoryUsage()	mémoire de l'état Lua courant en octets
babet.getDetailedMemoryUsage()	deux integers actuellement identiques

Constantes de version

Les quatre constantes décrivent le binaire Babet qui exécute le script.

```
print(babet.VERSION)      -- par exemple "2.6.0"
print(babet.VERSION_MAJOR) -- par exemple 2
print(babet.VERSION_MINOR) -- par exemple 5
print(babet.VERSION_PATCH) -- par exemple 0
```

babet.VERSION correspond à la sortie de :

```
babet --version
```

Les composantes entières permettent de comparer des versions sans parser une chaîne.

```
local function version_at_least(major, minor, patch)
    local current = {
        babet.VERSION_MAJOR,
        babet.VERSION_MINOR,
        babet.VERSION_PATCH,
    }
    local wanted = { major, minor, patch }

    for i = 1, 3 do
        if current[i] ~= wanted[i] then
            return current[i] > wanted[i]
        end
    end
    return true
end

assert(version_at_least(2, 6, 0), "Babet >= 2.6.0 requis")
```

La cohérence suivante est garantie par les tests :

```
assert(babet.VERSION == string.format(
    "%d.%d.%d",
    babet.VERSION_MAJOR,
    babet.VERSION_MINOR,
    babet.VERSION_PATCH
))
```

Identifier le processus et la machine

babet.pid()

Renvoie le PID du processus Babet sous forme d'integer strictement positif. L'appel ne peut pas échouer sur un système POSIX supporté.

```
local pid = babet.pid()
print("PID Babet :", pid)
```

Les workers sont des threads du même processus. Ils voient donc le même PID que le thread principal ; pid() ne permet pas d'identifier un worker particulier.

```
local main_pid = babet.pid()
local worker = assert(babet.workers.spawn([
    return babet.pid()
]))

local joined, worker_pid = worker:join()
assert(joined and worker_pid == main_pid)
```

babet.hostname()

Renvoie le nom d'hôte configuré pour la machine.

```
local host, err = babet.hostname()
assert(host, err)
print("Machine :", host)
```

Signature :

```
local host, err = babet.hostname()
```

Résultats :

- succès : host est une chaîne non vide et err vaut nil ;
- erreur système : (nil, "hostname: ...").

Le hostname n'est pas nécessairement un nom DNS pleinement qualifié. Il peut être un nom court défini localement.

babet.uname()

Renvoie les cinq champs POSIX principaux de uname(2) dans une table.

```
local info, err = babet.uname()
assert(info, err)
```

```
print("Système :", info.sysname)
print("Noeud   :", info.nodename)
print("Noyau   :", info.release)
print("Version :", info.version)
print("Machine :", info.machine)
```

Table renvoyée :

```
{
  sysname = "Linux",
  nodename = "station",
  release = "6.12.0-arch1-1",
  version = "#1 SMP PREEMPT_DYNAMIC ...",
  machine = "x86_64",
}
```

Champ	Signification
sysname	nom du système d'exploitation ou du noyau
nodename	nom du noeud réseau, souvent proche du hostname
release	version de publication du noyau
version	chaîne de build détaillée du noyau
machine	architecture matérielle exposée par le noyau

En cas d'échec de uname(2), la fonction renvoie (nil, "uname: ..."). Cette erreur est extrêmement rare, mais elle fait partie du contrat réel.

Rechercher un exécutable

Signature :

```
local path, err = babet.which(name)
```

name doit être une chaîne sans octet NUL.

Recherche dans PATH

Lorsque name ne contient aucun /, Babet parcourt la variable PATH de gauche à droite et renvoie le premier candidat qui est :

- un fichier régulier ;
- exécutable par le processus courant.

```

local shell, err = babet.which("sh")
assert(shell, err)
print(shell) -- généralement /usr/bin/sh ou /bin/sh

```

Le chemin renvoyé est normalisé en chemin absolu lorsque le système le permet.

Pour vérifier uniquement la présence d'un programme :

```

if babet.which("ffmpeg") then
    print("ffmpeg est disponible")
end

```

PATH est lu au moment de l'appel. Une modification effectuée avec setenv avant les workers est donc prise en compte par les appels suivants.

```

local old_path = babet.env("PATH")
assert(babet.setenv("PATH", "/opt/mon-app/bin:/usr/bin"))
local tool = babet.which("mon-outil")

```

Une composante vide de PATH, comme dans `:/usr/bin` ou `/bin::/usr/bin`, représente le répertoire courant conformément à la convention Unix.

Chemin direct

Lorsque l'argument contient un `/`, PATH n'est pas consulté. Le chemin est testé directement.

```

local sh, err = babet.which("/bin/sh")
assert(sh, err)

```

Un chemin relatif contenant `/` est également accepté :

```

local tool, err = babet.which("./bin/mon-outil")
assert(tool, err)

```

Un symlink valide vers un fichier régulier exécutable est accepté, car l'inspection suit sa cible.

Programme absent ou non exécutable

Si aucun exécutable valide n'est trouvé :

```

local path, err = babet.which("programme-introuvable")
-- path == nil
-- err == "which: 'programme-introuvable' not found in PATH"

```

Un fichier présent mais non exécutable est traité comme introuvable :

```

local path, err = babet.which("./script-sans-bit-x")
assert(path == nil)
print(err)

```

Sont également refusés :

- les dossiers ;
- les symlinks cassés ;
- les fichiers non réguliers ;
- les fichiers auxquels le processus n'a pas le droit d'exécution.

Variables d'environnement

babet.env(name)

Lit une variable d'environnement du processus.

```

local home = babet.env("HOME")
if home then
    print("HOME :", home)
end

```

Signature :

```

local value = babet.env(name)

```

Résultats :

- variable définie : sa valeur sous forme de chaîne ;
- variable absente : nil seul, sans message d'erreur.

Cela permet l'idiome :

```

local port = babet.env("APP_PORT") or "8080"

```

name doit être une chaîne. Les formes suivantes lèvent une erreur Lua :

```

-- babet.env()           -- argument manquant
-- babet.env(42)         -- aucune conversion automatique
-- babet.env({})         -- mauvais type
-- babet.env("A\0B")     -- NUL embarqué

```

babet.setenv(name, value)

Crée une variable d'environnement ou remplace sa valeur existante.

```

local ok, err = babet.setenv("APP_MODE", "production")
assert(ok, err)
assert(babet.env("APP_MODE") == "production")

```

Un second appel écrase la valeur :

```

assert(babet.setenv("APP_MODE", "test"))
assert(babet.env("APP_MODE") == "test")

```

name et value doivent être de vraies chaînes Lua.

Erreurs d'appelant, qui lèvent une erreur Lua :

```

-- babet.setenv()
-- babet.setenv("APP_MODE")
-- babet.setenv(42, "test")
-- babet.setenv("APP_MODE", 42)

```

Erreurs renvoyées sous la forme (nil, err) :

```

local ok, err = babet.setenv("BAD=NAME", "x")
-- ok == nil ; err indique un nom invalide

```

Un nom vide, un nom contenant = ou un octet NUL est refusé. Un octet NUL dans la valeur est également refusé.

Valeur vide et variable absente

Une valeur vide reste une valeur définie :

```

assert(babet.setenv("APP_OPTION", ""))
assert(babet.env("APP_OPTION") == "")

```

Elle est donc différente d'une variable absente :

```

local value = babet.env("VARIABLE_ABSENTE")
assert(value == nil)

```

Babet n'expose pas encore `unsetenv`. Il n'est pas possible de retirer une variable avec `setenv(name, nil)` : `nil` est un mauvais type et l'appel lève une erreur Lua.

Interaction avec les workers

`setenv` est autorisé uniquement avant le premier `workers.spawn()`.

```
assert(babet.setenv("APP_MODE", "production"))
```

```
local worker = assert(babet.workers.spawn([  
    return babet.env("APP_MODE")  
]))
```

```
local joined, value = worker:join()  
assert(joined and value == "production")
```

Après le premier `spawn`, même si le worker est déjà terminé :

```
local ok, err = babet.setenv("APP_MODE", "test")  
-- ok == nil  
-- err explique que l'environnement partagé ne peut plus être modifié
```

L'interdiction est permanente pour la durée du processus, y compris si le premier `spawn` a échoué. Prépare donc `PATH`, les variables d'application et le répertoire courant avant de lancer les workers.

Mémoire de la VM Lua

Les deux fonctions effectuent d'abord un cycle complet du ramasse-miettes Lua. Le résultat reflète donc la mémoire encore utilisée après collecte, et l'appel peut provoquer une pause proportionnelle à la taille de l'état Lua.

`babet.getMemoryUsage()`

Renvoie le nombre d'octets actuellement comptabilisés par le gestionnaire de mémoire de l'état Lua courant.

```
local bytes = babet.getMemoryUsage()  
print(string.format("Lua utilise %d octets", bytes))
```

Le résultat est un integer positif.

Exemple de comparaison avant/après :

```
local before = babet.getMemoryUsage()  
  
local values = {}  
for i = 1, 10000 do  
    values[i] = string.rep("x", 100)  
end  
  
local after = babet.getMemoryUsage()  
print("Variation Lua :", after - before, "octets")
```

La mesure n'est pas un outil de profiling précis : l'allocateur, l'internement des chaînes et les optimisations du GC peuvent faire varier le résultat.

`babet.getDetailedMemoryUsage()`

Renvoie deux integers :

```
local used, total = babet.getDetailedMemoryUsage()
```

Dans la version actuelle, les deux valeurs sont **strictement identiques** :

```
assert(used == total)
```

La deuxième valeur est conservée uniquement pour la stabilité de l'API. Lua n'expose pas séparément une mesure fiable de fragmentation ou de mémoire réservée mais inutilisée.

Pour un nouveau code qui n'a besoin que d'une mesure, préfère :

```
local bytes = babet.getMemoryUsage()
```

Ce qui est mesuré et ce qui ne l'est pas

La mesure couvre la mémoire gérée par **l'état Lua qui exécute l'appel**.

Elle ne représente pas :

- le RSS total du processus ;
- la pile native ;
- les allocations C++ du runtime ;
- les buffers OpenSSL, SQLite, HTTP ou sockets ;
- le code de l'exécutable et les bibliothèques partagées ;
- les états Lua des autres workers.

Dans un worker, la fonction mesure uniquement l'état Lua isolé de ce worker. Pour mesurer la mémoire globale du processus, utilise un outil système comme /proc, ps, smem, Valgrind ou un profiler adapté.

Contrat d'erreur

Situation	Comportement
argument manquant ou mauvais type	erreur Lua, récupérable avec pcall
NUL dans env, setenv ou which	erreur Lua pour env, (nil, err) pour setenv et which
variable absente dans env	nil seul
exécutable introuvable	(nil, err)
erreur de hostname ou uname	(nil, err)
setenv invalide ou refusé après un worker	(nil, err)
pid	toujours un integer
fonctions mémoire	toujours un ou deux integers après un GC complet

Exemple de traitement d'un échec runtime :

```
local tool, err = babet.which("outil-optionnel")
if not tool then
    print("Fonction désactivée :", err)
end
```

Exemple de traitement d'une mauvaise signature :

```
local ok, err = pcall(function()
    return babet.env(42)
end)

if not ok then
    print("Erreur de programmation :", err)
end
```

Décisions et limites

- Les fonctions historiques restent directement sous `babot` pour ne pas casser les scripts existants.
- `which` renvoie le premier exécutable trouvé, pas la liste de tous les candidats.
- `which` respecte le `PATH` du processus et les droits du processus courant ; son résultat peut donc différer selon l'utilisateur ou l'environnement.
- `env` distingue une variable absente (`nil`) d'une variable définie à la chaîne vide (`""`).
- `setenv` écrase toujours une valeur existante.
- Aucune fonction `unsetenv` ni `dump` complet de l'environnement n'est exposé.
- `pid` est le PID du processus, commun à tous les workers.
- Les fonctions mémoire provoquent volontairement un GC complet et ne mesurent pas le processus entier.
- Pour les comptes système, consultez le module [User - utilisateurs système](#).

babot tables — fusion et copie de tables Lua

Deux helpers historiques vivent directement dans la table `babot` :

- `babot.mergeTables(...)` construit une nouvelle table à partir de plusieurs sources ;
- `babot.deepCopyTable(t)` copie récursivement les **valeurs** qui sont des tables.

Ils travaillent sur les entrées réellement stockées dans les tables Lua. Les métaméthodes `__pairs` et `__index` ne fournissent donc pas de champs virtuels à copier ou à fusionner.

Table des matières du module

- [API](#)
- [babot.mergeTables\(t1, t2, ...\)](#)
 - Clés entières positives : concaténation
 - Toutes les autres clés : dernier écrivain gagnant
 - Fusion superficielle
- [babot.deepCopyTable\(t\)](#)
 - Valeurs et graphe partagé
 - Les clés ne sont pas copiées
 - Métatables et accès brut
 - Profondeur maximale
- [Contrat d'erreur](#)
- [Limites et choix de conception](#)
- [Hors v1](#)

API

Fonction	Renvoie
<code>babot.mergeTables(t1, t2, ...)</code>	une nouvelle table
<code>babot.deepCopyTable(t)</code>	une nouvelle table

Les deux fonctions renvoient exactement une valeur en cas de succès. Elles ne renvoient jamais (`nil, err`) : une erreur d'appel ou de profondeur est levée.

babot.mergeTables(t1, t2, ...)

Au moins deux arguments sont obligatoires et chacun doit être une table. Il n'existe pas de limite d'arité propre à l'API au-delà des limites générales de Lua.

Les sources ne sont pas modifiées. Le résultat est une table ordinaire sans métatable, même si les sources en possèdent une.

Clés entières positives : concaténation

Toute clé que Lua représente comme un entier ≥ 1 est traitée comme une position de liste. Cela inclut une clé écrite 2.0, que Lua canonicalise en entier.

Pour chaque source, ces clés sont parcourues par ordre numérique croissant, puis leurs valeurs sont ajoutées à la suite du résultat. Les trous et les indices d'origine sont donc compactés :

```
local r = babet.mergeTables(
  { [4] = "d", [2] = "b" },
  { [7] = "g", [1] = "a" }
)

-- r == { "b", "d", "a", "g" }
```

Une même position présente dans plusieurs sources n'est pas écrasée : chaque valeur est ajoutée.

Toutes les autres clés : dernier écrivain gagnant

Les clés chaînes, booléennes, tables, fonctions, threads, userdata, light userdata ainsi que les clés numériques flottantes, nulles ou négatives conservent leur identité. Une source ultérieure écrase la valeur précédente pour la même clé :

```
local key = {}
local r = babet.mergeTables(
  { mode = "safe", [key] = 1, [0] = "a" },
  { mode = "fast", [key] = 2, [0] = "b" }
)

-- r.mode == "fast"
-- r[key]   == 2
-- r[0]     == "b"
```

Fusion superficielle

Les valeurs ne sont jamais copiées. Une sous-table placée dans le résultat est la même table que dans la source :

```
local nested = { enabled = false }
local r = babet.mergeTables({ nested = nested }, {})

r.nested.enabled = true
-- nested.enabled == true
```

Pour obtenir ensuite un graphe indépendant pour les valeurs-table, utilise :

```
local isolated = babet.deepCopyTable(
  babet.mergeTables(defaults, overrides)
)
```

babet.deepCopyTable(t)

La fonction exige exactement une table. Elle crée une nouvelle table pour la racine et pour chaque **valeur** qui est elle-même une table.

Valeurs et graphe partagé

Les cycles parcourus par les valeurs sont pris en charge et l'identité du graphe est préservée :

```
local shared = { value = 42 }
local source = { a = shared, b = shared }
source.self = source
```

```
local copy = babet.deepCopyTable(source)
```

```
-- copy ~= source
-- copy.a ~= shared
-- copy.a == copy.b
-- copy.self == copy
```

Les valeurs non-table — nombres, chaînes, booléens, fonctions, threads, userdata et light userdata — sont réutilisées telles quelles.

Les clés ne sont pas copiées

Toutes les clés conservent leur valeur et leur identité d'origine. En particulier, une table utilisée comme clé reste la table source :

```
local key = { id = 1 }
local source = {
    [key] = "value",
    key_as_value = key,
}
local copy = babet.deepCopyTable(source)

-- copy[key] == "value"           -- clé originale conservée
-- copy.key_as_value ~= key       -- valeur-table copiée
-- copy[copy.key_as_value] == nil
```

La fonction est donc une copie profonde des **valeurs-table**, pas une copie structurelle des clés-table.

Métatables et accès brut

La métatable réelle de chaque table copiée est attachée à la copie **par référence**. Elle n'est pas dupliquée. Modifier cette métatable affecte donc la source et la copie.

Seules les paires réellement stockées sont parcourues. Un champ obtenu par `__index` ou inventé par `__pairs` n'est pas matérialisé dans la copie. Une fois la métatable partagée attachée, ses comportements restent naturellement actifs sur la copie.

Profondeur maximale

La racine est à la profondeur 0. Jusqu'à **75 descentes vers de nouvelles valeurs-table** sont acceptées, soit au maximum 76 tables sur un chemin racine comprise. Une 76e descente vers une table encore inconnue lève :

```
Table is too deep to copy (max depth 75 exceeded)
```

Une référence vers une table déjà copiée ne crée pas un nouveau niveau : un cycle peut donc se refermer exactement à la limite.

Contrat d'erreur

Les erreurs sont levées via l'API Lua :

- mergeTables avec moins de deux arguments ;
- tout argument non-table de mergeTables ;
- nombre d'arguments différent de un pour deepCopyTable ;
- argument non-table de deepCopyTable ;
- profondeur maximale dépassée.

```
local ok, err = pcall(babet.deepCopyTable, 42)
-- ok == false ; err contient "Argument must be a table"
```

Limites et choix de conception

- mergeTables n'est pas un deep merge : une sous-table ultérieure remplace la référence précédente en bloc.
- Les métatables des sources sont ignorées par mergeTables ; son résultat n'en possède aucune.
- deepCopyTable partage les métatables et les clés-table avec la source.
- Une métatable faible (__mode) reste faible sur la copie puisqu'elle est partagée.
- L'ordre des clés de map n'est pas défini. Seul l'ordre des clés entières positives de mergeTables est garanti.
- La sélection ordonnée des clés-listes de mergeTables privilégie un code sûr face aux erreurs Lua ; son coût est quadratique par source en nombre de clés entières positives. Ce helper vise donc surtout des tables de taille modérée.

Hors v1

Des fonctions comme deepMergeTables ou une comparaison structurelle profonde pourraient être ajoutées séparément. Elles ne font pas partie de l'API actuelle.

babet time - horloges, sommeil, dates et durées

Babet expose deux horloges sub-seconde, un sommeil interruptible et des utilitaires simples pour les timestamps ISO 8601 et les durées compactes.

Les fonctions historiques restent disponibles à la racine :

- `babet.monotonic()` ;
- `babet.now()` ;
- `babet.sleep(amount [, unit])`.

Les mêmes fonctions existent sous `babet.time`, avec quatre utilitaires supplémentaires : `iso`, `parse_iso`, `parse_duration` et `format_duration`.

Table des matières du module

- [API](#)
- [monotonic\(\) et now\(\)](#)
- [sleep\(amount \[, unit\]\)](#)
- [babet.time.iso\(\[ts\]\)](#)
- [babet.time.parse_iso\(text\)](#)
- [babet.time.parse_duration\(text\)](#)
- [babet.time.format_duration\(seconds\)](#)
- [Contrat d'erreur récapitulatif](#)
- [Limites et choix de conception](#)

API

Fonction	Résultat
<code>babet.monotonic()</code>	number, ou exceptionnellement (nil, err) si <code>clock_gettime</code> échoue
<code>babet.now()</code>	number, ou exceptionnellement (nil, err) si <code>clock_gettime</code> échoue
<code>babet.sleep(amount [, unit])</code>	(true, nil) ou (nil, err)
<code>babet.time.iso([ts])</code>	string UTC
<code>babet.time.parse_iso(text)</code>	(integer, nil) ou (nil, "parse_iso: ...")
<code>babet.time.parse_duration(text)</code>	(integer, nil) ou (nil, "parse_duration: ...")
<code>babet.time.format_duration(seconds)</code>	string compacte, par exemple "1d2h3m4s"

Les alias `babet.time.now`, `babet.time.monotonic` et `babet.time.sleep` utilisent exactement les mêmes bindings et le même contrat que leurs noms à la racine.

monotonic() et now()

```
local started = babet.monotonic()
do_something()
local elapsed = babet.monotonic() - started
print(string.format("durée : %.3f s", elapsed))
```

```
local timestamp = babet.now()
print(timestamp)
```

`monotonic()` repose sur `CLOCK_MONOTONIC`. Sa valeur part d'une origine arbitraire, généralement liée au démarrage du système. Elle sert à mesurer une durée, car elle ne subit pas les sauts de l'horloge civile.

`now()` repose sur `CLOCK_REALTIME`. Il renvoie le temps POSIX en secondes depuis le 1er janvier 1970 UTC, avec une partie fractionnaire. Il convient aux timestamps, mais pas à la mesure d'une durée : l'horloge civile peut être corrigée.

Les deux fonctions :

- n'acceptent aucun argument ;
- renvoient une seule valeur number en fonctionnement normal ;
- peuvent théoriquement renvoyer (nil, err) si l'appel système `clock_gettime` échoue.

sleep(amount [, unit])

```
assert(babet.sleep(250, "ms"))
assert(babet.time.sleep(0.5))      -- 0,5 seconde
assert(babet.sleep(1000, "us"))
```

`amount` doit être un véritable number Lua, fini et positif ou nul. Les floats sont acceptés. Une chaîne numérique comme "100" est refusée.

`unit` doit être une véritable string Lua :

Unité	Signification
"s"	secondes, valeur par défaut
"ms"	millisecondes
"us"	microsecondes

Une durée nulle réussit immédiatement. Une unité textuelle inconnue conserve le comportement historique suivant :

```
local ok, err = babet.sleep(1, "minutes")
-- ok == nil
-- err == "Invalid time unit"
```

Si un signal g  r   par `babet.signal` arrive pendant l'attente, son callback Lua est dispatch   puis `sleep` renvoie (`nil`, `"interrupted"`). Un signal non g  r   provoquant `EINTR` ne raccourcit pas l'attente : Babet reprend avec le temps restant.

Une autre erreur de `nanosleep` renvoie (`nil`, `"sleep: <description>"`).

babet.time.iso([ts])

`iso` formate une seconde Unix en UTC :

```
print(babet.time.iso(0))
-- 1970-01-01T00:00:00Z
```

```
print(babet.time.iso())
-- heure courante,    la seconde enti  re
```

Sans argument, `iso()` utilise l'heure courante. Avec un argument :

- `ts` doit   tre un v  ritable number Lua ;
- un entier ou un float fini est accept   ;
- une fraction est arrondie vers moins l'infini ;
- la valeur doit tenir dans un entier sign   64 bits.

L'arrondi est important avant l'  poque Unix :

```
babet.time.iso(0.9)  -- 1970-01-01T00:00:00Z
babet.time.iso(-0.5) -- 1969-12-31T23:59:59Z
```

Pour les ann  es usuelles, le r  sultat poss  de la forme `YYYY-MM-DDTHH:MM:SSZ`. Les timestamps extr  mes valides en `int64` peuvent produire une ann  e sign  e de plus de quatre chiffres. Ce r  sultat   tendu n'est pas forc  ment accept   par `parse_iso`, dont la grammaire impose une ann  e sur exactement quatre chiffres.

`iso` n'applique ni locale ni fuseau local et ne conserve pas les fractions de seconde.

babet.time.parse_iso(text)

Le parseur accepte volontairement un sous-ensemble strict et d  terministe :

```
YYYY-MM-DDTHH:MM:SSZ
YYYY-MM-DD HH:MM:SSZ
YYYY-MM-DDTHH:MM:SS.fraction+HH:MM
YYYY-MM-DDTHH:MM:SS.fraction-HH:MM
```

R  gles exactes :

- `text` doit   tre une v  ritable string Lua ;
- l'ann  e comporte exactement quatre chiffres et va de 0001    9999 ;
- le calendrier gr  gorien est valid  , y compris les ann  es bissextiles ;
- le s  parateur est T majuscule ou un espace ;
- heures 00..23, minutes 00..59, secondes 00..59 ;
- les secondes intercalaires :60 sont refus  es ;
- la fraction est facultative, mais le point doit   tre suivi d'au moins un chiffre ; tous ses chiffres sont ignor  s, sans arrondi ;
- la timezone est obligatoire : Z majuscule, +HH:MM ou -HH:MM ;
- les offsets accept  s vont de 00:00    23:59 ;
- +0200, +02, z, les espaces en d  but ou en fin et les donn  es suppl  mentaires sont refus  s ;
- un octet NUL embarqu   n'est jamais tronqu   : il rend la cha  ne invalide.

Exemple :

```
local ts = assert(babet.time.parse_iso(
    "2026-06-17T10:00:00+02:00"))
```

```
assert(ts == babet.time.parse_iso("2026-06-17T08:00:00Z"))
```

Une chaîne syntaxiquement ou calendriquement invalide renvoie (nil, "parse_iso: ..."). Un mauvais type ou un mauvais nombre d'arguments lève une erreur Lua.

babet.time.parse_duration(text)

Le format est une suite sans espace de couples nombre + unité :

```
assert(babet.time.parse_duration("45s") == 45)
assert(babet.time.parse_duration("1h30m") == 5400)
assert(babet.time.parse_duration("2d12h") == 216000)
```

Unités disponibles :

Unité	Secondes
d	86400
h	3600
m	60
s	1

Les unités doivent apparaître au plus une fois et dans l'ordre strict d > h > m > s. Les signes, décimales, espaces et unités w, ms, us ou ns sont refusés. Un octet NUL embarqué n'est pas tronqué et provoque également une erreur de parsing.

Les composantes ne sont pas normalisées : "1h90m" est valide et vaut 9000 secondes. Les zéros de tête et les composantes nulles ordonnées sont également acceptés, par exemple "0001s" et "0d0h0m0s".

Le total doit rester compris entre 0 et math.maxinteger sur la configuration 64 bits de Babet. Les dépassements renvoient une erreur de parsing.

Une chaîne invalide renvoie (nil, "parse_duration: ..."). Un mauvais type ou un mauvais nombre d'arguments lève une erreur Lua.

babet.time.format_duration(seconds)

```
babet.time.format_duration(0)      -- "0s"
babet.time.format_duration(90)    -- "1m30s"
babet.time.format_duration(90061) -- "1d1h1m1s"
```

seconds doit être un véritable number Lua, entier et positif ou nul. Un float de valeur entière comme 3.0 est accepté ; 3.7, "3", NaN, Inf et les valeurs négatives lèvent une erreur Lua.

La sortie est la représentation canonique utilisée par Babet :

- unités dans l'ordre d, h, m, s ;
- composantes nulles omises ;
- zéro représenté par "0s".

Pour tout entier valide $n \geq 0$:

```
assert(babet.time.parse_duration(
    babet.time.format_duration(n)) == n)
```

L'inverse n'est pas textuellement garanti, car le parseur accepte aussi des formes non normalisées comme "90m" ou "0h".

Contrat d'erreur récapitulatif

Situation	Comportement
Mauvais type ou mauvaise arité	erreur Lua
NaN, Inf, durée négative ou valeur numérique hors plage	erreur Lua
Unité textuelle inconnue dans sleep	(nil, "Invalid time unit")
Signal géré pendant sleep	(nil, "interrupted") après dispatch du callback
Texte ISO ou durée invalide	(nil, "parse_iso: ...") ou (nil, "parse_duration: ...")
Échec système d'horloge ou de sommeil	(nil, err)

Limites et choix de conception

- `monotonic()` sert aux durées ; `now()` sert aux timestamps.
- `iso` est un formateur UTC fixe, pas un remplaçant général de `os.date`.
- Aucun fuseau nommé, changement d'heure, locale ou calcul calendaire n'est fourni.
- Les fractions ISO sont ignorées et ne sont pas renvoyées par `iso`.
- `parse_duration` manipule des secondes entières uniquement.
- `CLOCK_BOOTTIME`, les durées négatives et les unités semaine/mois/année ne sont pas exposés.

TLS — connexion chiffrée directe, STARTTLS, certificats et SNI

Le support TLS de Babet repose sur les mêmes userdata que [babet.socket](#). Après le handshake, les méthodes `send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `peer`, `sockname` et `close` restent identiques.

Deux modes sont disponibles :

- `babet.socket.connect_tls()` pour une connexion TLS dès le premier octet ;
- `sock:starttls()` pour transformer en place une connexion TCP après une négociation applicative en clair.

Le module couvre :

- TLS 1.2 minimum, avec TLS 1.3 négocié lorsqu'il est disponible ;
- la vérification du certificat activée par défaut ;
- la vérification du hostname ;
- SNI, indépendamment de l'activation de la vérification ;
- les autorités système et des CA supplémentaires par appel ;
- un timeout global connexion TCP + handshake pour `connect_tls` ;
- un comportement fail-closed après le début d'un STARTTLS raté.

Il ne fournit pas de serveur TLS, de certificat client, d'ALPN, de pinning de clé publique, d'OCSP applicatif ni de réglage fin des suites cryptographiques.

Table des matières du module

- [Conventions essentielles](#)
- [Vue d'ensemble de l'API](#)
- [Options TLS](#)
- [Connexion TLS directe](#)
- [Transformer une connexion avec STARTTLS](#)
- [Vérification, hostname et SNI](#)
- [Autorités de certification](#)
- [Versions TLS](#)

- [Timeouts](#)
- [État du socket après une erreur](#)
- [Exemples complets](#)
- [Contrat d'erreur](#)
- [Sécurité et limites](#)

Conventions essentielles

Vérification activée par défaut

Cette forme vérifie la chaîne de certification et l'identité du serveur :

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
```

`verify = false` est un opt-out explicite destiné aux tests ou à un protocole qui applique une autre authentification forte. Il désactive **à la fois** la validation de chaîne et la vérification du hostname :

```
local sock = assert(babet.socket.connect_tls("127.0.0.1", 8443, {
    verify = false,
    timeout = 5,
}))
```

Cette option chiffre le transport, mais ne protège pas contre un attaquant actif de type homme-du-milieu.

Même API de flux après le handshake

```
local sock = assert(babet.socket.connect_tls("irc.example.net", 6697, {
    timeout = 10,
}))
assert(sock:set_timeout(120))
assert(sock:send("PING :hello\r\n"))
local line = assert(sock:recv_line())
```

Le timeout du handshake et le timeout d'I/O sont distincts.

SNI n'est pas la vérification

SNI indique au serveur quel virtual host sélectionner. La vérification décide si le certificat reçu est digne de confiance et correspond au nom attendu.

Babet envoie SNI lorsqu'un hostname DNS est disponible, même avec `verify = false`. Désactiver la vérification ne doit pas empêcher un serveur multi-sites de choisir le bon certificat ou le bon endpoint.

Vue d'ensemble de l'API

```
local sock, err = babet.socket.connect_tls(host, port, opts?)
local ok, err = sock:starttls(opts?)
```

Les deux formes utilisent la même table d'options :

```
{
    verify = true,
    hostname = "example.com",
    ca_cert = "/path/ca-bundle.pem",
    ca_path = "/path/certs",
    min_version = "1.2",
```

```
    timeout = 10,
}
```

Champ	Type	Défaut	Rôle
verify	booléen strict	true	vérifie la chaîne et le hostname
hostname	string	host de connect_tls	nom attendu et nom SNI s'il est DNS
ca_cert	string	aucune CA supplémentaire	fichier PEM ajouté au trust store
ca_path	string	aucun dossier supplémentaire	dossier OpenSSL de CA hashées
min_version	"1.2" ou "1.3"	"1.2"	version TLS minimale
timeout	nombre fini >= 0	0	budget du handshake, en secondes

Les champs inconnus sont actuellement ignorés. Une faute comme `hostnme = "example.com"` ne produit donc pas d'erreur et ne remplace pas `hostname`.

Options TLS détaillées

verify

Doit être un vrai booléen Lua :

```
verify = true
verify = false
```

`verify = 1` et `verify = "yes"` sont refusés.

Avec `true`, Babet configure `SSL_VERIFY_PEER` et une vérification d'identité avec `SSL_set1_host`.

hostname

Sert à deux choses :

1. identité de référence pour le certificat lorsque `verify = true` ;
2. valeur SNI lorsqu'il s'agit d'un nom DNS.

```
local sock = assert(babet.socket.connect_tls("203.0.113.10", 443, {
    hostname = "api.example.com",
    timeout = 10,
}))
```

Cette forme connecte l'adresse IP fournie mais vérifie le certificat pour `api.example.com` et envoie ce nom en SNI.

Un `hostname` qui est lui-même une adresse IP est utilisé pour la vérification OpenSSL, mais n'est pas envoyé comme SNI, conformément au rôle DNS du champ `server_name`.

ca_cert et ca_path

Ces options ajoutent des autorités à celles déjà chargées pour cet appel. Elles ne remplacent pas le trust store système et ne constituent pas du pinning.

```
local sock = assert(babet.socket.connect_tls("service.internal", 443, {
    ca_cert = "/etc/myapp/internal-root.pem",
    timeout = 10,
}))
```

ca_path doit désigner un répertoire préparé selon les conventions OpenSSL, avec les liens hashés attendus. Un simple dossier rempli de PEM arbitraires ne suffit pas nécessairement.

min_version

"1.2" et "1.3" sont les seules valeurs acceptées. Il s'agit d'une version **minimale**, pas d'une version maximale.

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    min_version = "1.3",
    timeout = 10,
}))
```

timeout

0 signifie aucun timeout de handshake. Une valeur positive est arrondie à au moins 1 ms et doit rester représentable en millisecondes système.

babet.socket.connect_tls(host, port, opts?)

Établit une connexion TCP puis réalise immédiatement un handshake TLS client.

```
local sock, err = babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
})
if not sock then
    error(err)
end
```

Hostname par défaut

Lorsque opts.hostname est absent, le premier argument host sert de nom de référence et de SNI s'il s'agit d'un nom DNS.

```
-- Vérifie et envoie example.com automatiquement.
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
```

Avec une connexion par IP vers un certificat DNS, passe explicitement le nom :

```
local sock = assert(babet.socket.connect_tls("192.0.2.15", 443, {
    hostname = "www.example.com",
    timeout = 10,
}))
```

Deadline globale

Après la résolution DNS, le même budget couvre :

- les tentatives de connexion TCP sur toutes les adresses ;
- le handshake TLS.

Le handshake ne reçoit pas un nouveau budget complet après la connexion. Comme pour socket.connect, la résolution DNS synchrone reste hors deadline.

Timeout d'I/O après succès

`opts.timeout` n'est pas conservé dans le socket. Après succès, le timeout d'I/O par défaut vaut 0 :

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
assert(sock:set_timeout(15))
```

sock:starttls(opts?)

Transforme en place un socket TCP connecté après la négociation STARTTLS du protocole applicatif.

```
local ok, err = sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
})
```

En succès, la méthode renvoie exactement `(true, nil)` et toutes les lectures et écritures suivantes passent par TLS.

Préconditions

Le socket doit être :

- ouvert ;
- connecté, pas en écoute ;
- encore en TCP clair ;
- sans octets plaintext conservés dans le buffer de lecture de Babet.

Hostname obligatoire avec vérification

Contrairement à `connect_tls`, `starttls` ne mémorise pas le hostname utilisé lors de la connexion TCP. Avec `verify = true`, `opts.hostname` est donc obligatoire :

```
assert(sock:starttls({
    hostname = "smtp.example.com",
    timeout = 10,
}))
```

Cette omission est refusée **avant** le début du handshake, et le socket TCP reste utilisable en clair.

Consommer le plaintext avant l'upgrade

Si `recv_line()` ou `recv_all()` a déjà retiré des octets du noyau puis a expiré, ces octets restent tamponnés. Babet refuse STARTTLS tant qu'ils ne sont pas consommés :

```
local line, err = sock:recv_line(0.2)
if not line and err == "timeout" then
    -- Finir de consommer/résoudre la réponse plaintext avant starttls.
end
```

Cela empêche de présenter des octets applicatifs en clair au décodeur TLS.

Déroulement applicatif

Babet n'envoie pas la commande STARTTLS à ta place. Le script doit suivre le protocole :

1. connexion TCP ;
2. dialogue initial ;

3. commande STARTTLS ;
4. vérification de la réponse positive ;
5. appel à `sock:starttls()` ;
6. reprise du protocole chiffré, souvent avec une nouvelle salutation.

Vérification, hostname et SNI

Les combinaisons importantes sont les suivantes :

Connexion	verify	hostname	Vérification	SNI
host DNS example.com	true	absent	example.com	example.com
IP 192.0.2.5	true	api.example.com	api.example.com	api.example.com
IP 192.0.2.5	true	absent	vérification de l'IP	aucun SNI
host DNS	false	absent	aucune	host DNS
IP	false	api.example.com	aucune	api.example.com
IP	false	absent	aucune	aucun SNI

Avec `verify = false`, fournir `hostname` reste utile pour un virtual host TLS, même si le certificat n'est pas vérifié.

SNI est envoyé avant que le serveur choisisse son certificat. La vérification du `hostname` intervient ensuite sur le certificat présenté. Les deux mécanismes sont donc liés dans l'usage courant, mais conceptuellement indépendants.

Autorités de certification

Babet utilise un OpenSSL embarqué statiquement. Le contexte TLS socket tente :

1. les chemins par défaut d'OpenSSL ;
2. les variables `SSL_CERT_FILE` et `SSL_CERT_DIR` prises en compte par OpenSSL ;
3. plusieurs emplacements de CA connus sous Linux et BSD ;
4. les CA supplémentaires `ca_cert` et `ca_path` de l'appel.

Les CA personnalisées sont isolées par connexion. Charger une CA pour un appel ne la rend pas fiable pour les connexions suivantes.

```
local a = assert(babet.socket.connect_tls("internal.example", 443, {
    ca_cert = "/tmp/test-root.pem",
    timeout = 5,
}))
a:close()

-- Cet appel ne réutilise pas /tmp/test-root.pem.
local b, err = babet.socket.connect_tls("internal.example", 443, {
    timeout = 5,
})
```

CA supplémentaire, pas certificat serveur piné

Si `ca_cert` contient une autorité capable de signer plusieurs certificats, tous les certificats valides pour le `hostname` et cette autorité peuvent être acceptés. Pour du pinning strict, ajoute une vérification applicative externe ; Babet n'expose pas actuellement le certificat pair ni son empreinte.

Versions TLS

Le contexte refuse les protocoles antérieurs à TLS 1.2.

- défaut : minimum TLS 1.2 ;
- `min_version = "1.3"` : minimum TLS 1.3 ;
- TLS 1.3 est négocié automatiquement avec le défaut si le serveur le propose.

Babet ne permet pas de forcer un maximum. La version finale dépend d'OpenSSL et du serveur.

Timeouts

`connect_tls`

```
local sock, err = babet.socket.connect_tls(host, port, {  
    timeout = 5,  
})
```

Le budget couvre TCP + handshake après DNS.

`starttls`

```
local ok, err = sock:starttls({  
    hostname = host,  
    timeout = 5,  
})
```

Le budget couvre uniquement le handshake STARTTLS. Il ne reprend pas le timeout défini par `sock:set_timeout()`.

I/O après le handshake

```
assert(sock:set_timeout(30))  
local line, err = sock:recv_line()
```

Les opérations TLS utilisent un descripteur non bloquant en interne, piloté par OpenSSL et `poll`, afin de respecter la deadline globale même lorsque `SSL_read` ou `SSL_write` réclame une autre direction d'I/O.

État du socket après une erreur

`connect_tls`

Une erreur ne renvoie aucun socket. Les ressources TCP/TLS intermédiaires sont fermées.

`starttls` avant le handshake

Les erreurs suivantes laissent le socket TCP inchangé :

- options invalides ;
- hostname manquant avec `verify = true` ;
- plaintext tamponné ;
- échec de préparation du contexte TLS ou de configuration avant `SSL_connect` ;
- échec de passage non bloquant avant le handshake.

Le script peut corriger le problème et continuer en clair si le protocole le permet.

starttls après le début du handshake

Dès que SSL_connect a commencé, tout échec ferme définitivement le socket :

- timeout ;
- interruption ;
- certificat invalide ;
- alerte TLS ;
- erreur de protocole ou d'I/O.

Le flux peut déjà contenir un ClientHello ou avoir consommé des octets du pair. Le réutiliser en clair serait ambigu et dangereux. Babet adopte donc un comportement **fail-closed**.

```
local ok, err = sock:starttls(opts)
if not ok then
    -- Si le handshake a commencé, les appels suivants signaleront "closed".
    sock:close() -- reste idempotent
end
```

Exemples complets

Client HTTPS minimal sur socket brut

Pour une requête HTTP classique, préfère [babet.http](#). Cet exemple montre seulement l'API de flux :

```
local S = babet.socket
local sock = assert(S.connect_tls("example.com", 443, { timeout = 10 }))
assert(sock:set_timeout(10))

assert(sock:send(
    "GET / HTTP/1.1\r\n" ..
    "Host: example.com\r\n" ..
    "Connection: close\r\n\r\n"
))

local response = assert(sock:recv_all(nil, 8 * 1024 * 1024))
print(response)
sock:close()
```

Connexion par IP avec nom de certificat

```
local sock = assert(babet.socket.connect_tls("203.0.113.20", 443, {
    hostname = "api.example.com",
    ca_cert = "/etc/myapp/ca.pem",
    timeout = 5,
}))
```

SMTP STARTTLS simplifié

```
local S = babet.socket
local sock = assert(S.connect("mail.example.com", 587, 5))
assert(sock:set_timeout(10))

local banner = assert(sock:recv_line())
assert(banner:match("^220"), banner)

assert(sock:send("EHLO client.example\r\n"))
repeat
```

```

    local line = assert(sock:recv_line())
    if line:match("^250[ -]STARTTLS") then
        -- capability vue
    end
until line:match("^250 ")

assert(sock:send("STARTTLS\r\n"))
local reply = assert(sock:recv_line())
assert(reply:match("^220"), reply)

assert(sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
}))

-- SMTP demande généralement un nouvel EHLO après STARTTLS.
assert(sock:send("EHLO client.example\r\n"))

```

Cet exemple ne constitue pas un client SMTP complet : il faut parser toutes les réponses multi-lignes, authentifier et gérer les codes d'erreur.

Test local auto-signé

```

local sock = assert(babet.socket.connect_tls("127.0.0.1", 8443, {
    verify = false,
    hostname = "dev.local", -- SNI reste envoyé
    timeout = 3,
}))

```

N'utilise pas cette configuration en production.

Contrat d'erreur

Erreurs Lua levées

```

babet.socket.connect_tls({}, 443)
babet.socket.connect_tls("host", "443")

```

Le host doit être une string stricte et le port un entier Lua strict.

Erreurs (nil, err)

- host vide, NUL ou port hors plage ;
- table d'options invalide ;
- mauvais type ou mauvaise valeur d'un champ ;
- CA illisible ou invalide ;
- DNS ou connexion TCP ;
- timeout ou interruption ;
- échec du handshake ;
- vérification de chaîne ou de hostname ;
- précondition STARTTLS non satisfaite.

Les messages OpenSSL peuvent varier selon la version. Teste les états stables comme "timeout" ou "interrupted", et journalise le message complet pour le diagnostic.

Sécurité et limites

- **Ne désactive pas verify en production.** Le chiffrement sans authentification n'empêche pas un MITM.

- **Passe le vrai hostname attendu.** Ne vérifie jamais un nom fourni par le pair lui-même.
- **CA privée.** Protège le fichier CA et déploie une rotation maîtrisée.
- **STARTTLS stripping.** Le protocole applicatif doit exiger STARTTLS, pas simplement l'utiliser lorsqu'il est annoncé. Un attaquant pourrait supprimer la capability dans un canal clair si ton application accepte un downgrade.
- **Pas d'ALPN.** Babet socket n'est pas adapté à HTTP/2, qui négocie normalement h2 via ALPN.
- **Pas de certificat client.** Les protocoles mTLS ne sont pas supportés par cette API.
- **Pas d'accès au certificat pair.** Impossible de faire du pinning ou une inspection avancée directement en Lua.
- **Pas de serveur TLS.** listen() et accept() créent des sockets TCP ; aucune méthode accept_tls n'est exposée.
- **Vérification de révocation.** Babet ne fournit pas de politique OCSP/CRL applicative supplémentaire au comportement d'OpenSSL.

babet.toml - décodage TOML

Le module `babet.toml` s'appuie sur **toml++ 3.4.0** et décode des documents conformes à **TOML 1.0.0**. Les extensions situées hors du profil TOML 1.0 et prises en charge optionnellement par cette version de toml++ sont désactivées.

La v1 expose uniquement le décodage d'une chaîne. Elle ne lit pas un fichier à ta place et ne fournit pas d'encodeur.

Table des matières du module

- [API](#)
- [Correspondance des types](#)
 - [Entiers et flottants](#)
- [Chaînes, UTF-8 et clés](#)
- [Arrays et tables](#)
 - [Ambiguïté des conteneurs vides](#)
- [Dates et heures](#)
- [Ordre et informations perdues](#)
- [Contrat d'erreur](#)
- [Limites](#)
- [Non exposé en v1](#)

API

Fonction	Contrat
<code>babet.toml.decode(text)</code>	Renvoie table, nil ou nil, err

`decode` exige exactement **un argument**, qui doit être une véritable chaîne Lua. Un nombre n'est pas converti implicitement en texte et tout argument supplémentaire est refusé.

Pour lire un fichier :

```
local file = assert(io.open("config.toml", "rb"))
local text = assert(file:read("*a"))
file:close()

local config, err = babet.toml.decode(text)
if not config then
    error(err)
end
```

Un document TOML possède toujours une table à sa racine. Une chaîne vide ou composée uniquement de commentaires produit donc une table Lua vide.

Correspondance des types

TOML	Lua
string	string
integer signé sur 64 bits	integer Lua
float	number Lua
boolean	boolean
array	table séquentielle 1..n
table, inline table ou dotted table	table à clés chaînes
array of tables	table séquentielle de tables
local date/time et date-time	string normalisée

TOML ne possède pas de valeur équivalente à nil ou au null JSON.

Entiers et flottants

Les entiers TOML occupent la plage signée complète sur 64 bits, de `math.mininteger` à `math.maxinteger`, et sont conservés exactement. Les notations décimale, hexadécimale, octale et binaire produisent toutes un entier Lua. Une valeur hors de cette plage est une erreur de parsing.

Les flottants deviennent des nombres Lua. TOML autorise également les valeurs spéciales suivantes :

```
local values = assert(babet.toml.decode([[
positive = inf
negative = -inf
invalid  = nan
]]))

assert(values.positive == math.huge)
assert(values.negative == -math.huge)
assert(values.invalid ~= values.invalid) -- propriété de NaN
```

Comme en Lua, NaN ne se compare égal à aucune valeur, pas même à lui-même.

Chaînes, UTF-8 et clés

Le document TOML doit être en UTF-8 valide. Les chaînes basiques, littérales et multilignes sont décodées avant d'être renvoyées à Lua : les séquences d'échappement TOML deviennent les caractères correspondants.

Les chaînes Lua sont binary-safe. Une séquence TOML comme `\u0000` peut donc produire un octet NUL dans une valeur **ou dans une clé citée**, sans troncature :

```
local value = assert(babet.toml.decode([[
"a\u0000b" = "x\u0000y"
]]))

assert(value["a\0b"] == "x\0y")
```

Un octet NUL brut injecté dans le texte TOML n'est pas traité comme une fin de chaîne C : le parseur voit l'intégralité du buffer et refuse le document. Il en va de même pour une séquence UTF-8 invalide.

Les clés nues, citées et pointées suivent les règles TOML :

```
local value = assert(babet.toml.decode([[
"a.b" = 1
site."google.com" = true
physical.color = "orange"
]]))

assert(value["a.b"] == 1)           -- point littéral dans une clé citée
assert(value.site["google.com"] == true)
assert(value.physical.color == "orange")
```

Une clé citée vide ("") est valide en TOML et devient la clé Lua "".

Arrays et tables

Les arrays TOML deviennent des séquences Lua indexées à partir de 1. TOML 1.0 autorise les arrays hétérogènes :

```
local value = assert(babet.toml.decode([[
items = [1, "two", true, { name = "three" }]
]]))

assert(value.items[1] == 1)
assert(value.items[2] == "two")
assert(value.items[3] == true)
assert(value.items[4].name == "three")
```

Les sections, inline tables et dotted keys aboutissent toutes à des tables Lua ordinaires. La syntaxe d'origine n'est pas conservée.

Ambiguïté des conteneurs vides

Un array vide [] et une table vide {} deviennent tous deux une table Lua vide :

```
local value = assert(babet.toml.decode([[
a = []
b = {}
]]))

assert(type(value.a) == "table" and next(value.a) == nil)
assert(type(value.b) == "table" and next(value.b) == nil)
```

Aucune métatable ni sentinelle ne mémorise leur type TOML d'origine. Cette perte d'information est sans conséquence pour la lecture courante d'une configuration, mais elle empêche un round-trip fidèle.

Dates et heures

TOML distingue quatre types temporels. Lua n'ayant pas de type date natif, Babet les convertit en chaînes :

Type TOML	Forme Lua
local date	YYYY-MM-DD
local time	HH:MM:SS[.fraction]
local date-time	YYYY-MM-DDTHH:MM:SS[.fraction]
offset date-time	YYYY-MM-DDTHH:MM:SS[.fraction]Z ou avec +/-HH:MM

```
local value = assert(babet.toml.decode([[
date = 1979-05-27
time = 07:32:00.1234
local_dt = 1979-05-27T07:32:00
utc_dt = 1979-05-27T07:32:00Z
]]))
```

La valeur temporelle est conservée, mais sa **graphie** peut être normalisée par toml++ : séparateur canonique, fraction ou notation UTC. Les commentaires et l'orthographe exacte du document ne sont pas disponibles. Après conversion, une date TOML est également indiscernable d'une chaîne TOML contenant le même texte.

Ordre et informations perdues

Une table TOML n'a pas d'ordre sémantique garanti et une table Lua ne doit pas être parcourue en supposant l'ordre du fichier. Le décodage ne conserve pas :

- les commentaires ;
- l'ordre lexical des clés ;
- les espaces et retours à la ligne ;
- le choix entre section, inline table et dotted keys ;
- la graphie d'origine des nombres et des dates ;
- le type d'un conteneur vide.

Le module est donc destiné à **lire les valeurs**, pas à éditer puis réécrire un document TOML à l'identique.

Contrat d'erreur

Une erreur de syntaxe ou de valeur TOML renvoie :

```
local value, err = babet.toml.decode("answer = ")
assert(value == nil)
print(err) -- toml: ... (line ..., col ...)
```

Le message est préfixé par toml: et contient la ligne et la colonne indiquées par toml++.

Sont notamment renvoyés sous forme de (nil, err) :

- syntaxe invalide ou clé redéfinie ;
- entier hors de la plage signée 64 bits ;
- UTF-8 invalide ou caractère de contrôle interdit ;
- extension non disponible dans le profil TOML 1.0 utilisé.

Les erreurs d'utilisation de l'API lèvent en revanche une erreur Lua :

- argument absent ;
- argument qui n'est pas une chaîne ;
- argument supplémentaire.

Limites

Le document complet est parsé et converti en mémoire. Il n'existe pas d'API de streaming, de decode_file, de validation de schéma ni d'accès aux positions source des valeurs. La conversion récursive protège la pile Lua ; une profondeur impossible à convertir est renvoyée comme une erreur toml:.

Non exposé en v1

- babet.toml.encode ;

- `babet.toml.decode_file` ;
- options de parsing ou activation des extensions hors TOML 1.0 ;
- nœuds conservant commentaires, ordre et positions source ;
- sentinelles distinguant arrays et tables vides ;
- validation de schéma.

USER - rechercher des comptes système par nom ou UID

Le module `babet.user` interroge la base des utilisateurs du système via NSS. Il permet de :

- rechercher un compte par nom ;
- rechercher un compte par UID ;
- tester rapidement l'existence d'un compte ;
- récupérer le nom, l'UID, le GID principal, le champ GECOS, le home et le shell.

Il ne lit jamais `/etc/passwd` directement. Il utilise les mêmes mécanismes de résolution que `id` ou `getent passwd`, et peut donc voir les comptes fournis par LDAP, SSSD, NIS, FreeIPA ou d'autres sources configurées dans `/etc/nsswitch.conf`.

Table des matières du module

- [Conventions générales](#)
- [Vue d'ensemble de l'API](#)
- [Table utilisateur renvoyée](#)
- [Rechercher un compte avec `get`](#)
 - Recherche par nom
 - Recherche par UID
 - Compte absent
- [Tester l'existence avec `exists`](#)
 - Par nom
 - Par UID
 - Quand préférer `get`
- [Validation des arguments](#)
- [NSS, workers et sécurité](#)
- [Contrat d'erreur](#)
- [Décisions et limites](#)

Conventions générales

Sous-table dédiée

Contrairement aux fonctions historiques de SYS ou FS, les fonctions utilisateur sont regroupées dans `babet.user` :

```
local user = babet.user.get("root")
local exists = babet.user.exists("root")
```

Nom ou UID, sans conversion implicite

L'argument accepte exactement :

- une chaîne Lua : recherche par **nom** ;
- un integer Lua non négatif dans la plage de `uid_t` : recherche par **UID**.

Une chaîne numérique reste un nom :

```
-- Recherche un compte dont le nom est littéralement "1000"
local by_name = babet.user.get("1000")
```

```
-- Recherche l'UID numérique 1000
local by_uid = babet.user.get(1000)
```

Les floats, booléens, tables, nil, UID négatifs et UID trop grands lèvent une erreur Lua. Aucune troncature ni conversion silencieuse n'est effectuée.

Champs toujours présents

Lorsqu'un utilisateur est trouvé, la table contient toujours les six champs documentés. Les champs texte peuvent être la chaîne vide, mais jamais nil.

Vue d'ensemble de l'API

Fonction	Résultat
<code>babet.user.get(name_or_uid)</code>	table, (nil, "user not found") ou (nil, "user: ...")
<code>babet.user.exists(name_or_uid)</code>	booléen strict

Table utilisateur renvoyée

Exemple typique :

```
{
  name  = "www-data",
  uid   = 33,
  gid   = 33,
  gecos = "www-data",
  home  = "/var/www",
  shell = "/usr/sbin/nologin",
}
```

Champ	Type	Signification
name	string	nom canonique renvoyé par NSS
uid	integer	identifiant numérique de l'utilisateur
gid	integer	GID principal du compte
gecos	string	champ descriptif brut, souvent le nom complet
home	string	répertoire personnel déclaré
shell	string	shell déclaré pour le compte

gid n'est pas la liste des groupes secondaires. Le module ne réalise pas de lookup de groupes et n'expose pas encore les appartenances supplémentaires.

Le champ gecos est renvoyé tel quel. Son format historique peut contenir plusieurs valeurs séparées par des virgules, mais de nombreux systèmes y placent simplement un nom libre.

```
local u = assert(babet.user.get("root"))
print("Nom   :", u.name)
print("UID   :", u.uid)
print("GID   :", u.gid)
print("GECOS :", u.gecos)
print("Home  :", u.home)
print("Shell :", u.shell)
```

Rechercher un compte avec get

Signature :

```
local info, err = babet.user.get(name_or_uid)
```

Utilise get lorsque tu as besoin des informations du compte ou lorsque tu dois distinguer un utilisateur absent d'une panne NSS.

Recherche par nom

```
local root, err = babet.user.get("root")
assert(root, err)
assert(root.name == "root")
assert(root.uid == 0)
```

Exemple pour préparer un répertoire appartenant à un compte de service :

```
local account, err = babet.user.get("mon-service")
assert(account, err)
```

```
assert(babet.mkdir("/var/lib/mon-service"))
assert(babet.setAttributes(
    "/var/lib/mon-service",
    account.uid,
    account.gid,
    tonumber("750", 8)
))
```

Le nom est transmis à NSS tel quel. Il doit être une chaîne sans octet NUL. Une chaîne vide est une recherche valide au niveau de l'API, mais elle ne correspond normalement à aucun compte et renvoie user not found.

Recherche par UID

```
local root, err = babet.user.get(0)
assert(root, err)
assert(root.name == "root")
```

Exemple pour afficher le compte associé à un propriétaire de fichier :

```
local attrs, err = babet.getAttributes("rapport.txt")
assert(attrs, err)

local owner, user_err = babet.user.get(attrs.owner)
if owner then
    print("Propriétaire :", owner.name)
else
    print("UID sans compte résolu :", attrs.owner, user_err)
end
```

L'UID doit être un integer non négatif compatible avec le type système uid_t. Sur Linux, la limite est généralement $2^{32} - 1$, mais le code utilise la limite réelle de la plateforme au moment de la compilation.

Compte absent

Un compte absent n'est pas une erreur Lua. La fonction renvoie :

```

local info, err = babet.user.get("compte-inexistant")
-- info == nil
-- err == "user not found"

```

Traitement classique :

```

local account, err = babet.user.get("mon-service")
if not account then
    if err == "user not found" then
        print("Le compte doit être créé")
    else
        print("La résolution NSS a échoué :", err)
    end
end

```

Ne fais pas :

```

-- Mauvais si l'absence est un cas normal : assert lève immédiatement
-- local account = assert(babet.user.get("compte-optionnel"))

```

Tester l'existence avec exists

Signature :

```

local present = babet.user.exists(name_or_uid)

```

La fonction renvoie toujours un booléen pour un argument valide.

Par nom

```

if babet.user.exists("www-data") then
    print("Le compte www-data existe")
end

```

Exemple de précondition simple :

```

if not babet.user.exists("mon-service") then
    error("Le compte mon-service doit être créé avant le démarrage")
end

```

Par UID

```

assert(babet.user.exists(0)) -- root sur un système Unix normal

```

Un UID absent renvoie false :

```

local present = babet.user.exists(2000000000)
print(present)

```

Quand préférer get

exists assimile volontairement une erreur NSS à false.

Cela rend la fonction pratique pour une branche simple, mais elle ne permet pas de distinguer :

- un compte réellement absent ;
- un annuaire LDAP temporairement indisponible ;
- une erreur d'E/S ;
- un manque de mémoire dans le résolveur.

Pour une décision importante, notamment en administration ou en sécurité, utilise get :

```

local account, err = babet.user.get("mon-service")
if account then
    print("Compte disponible")
elseif err == "user not found" then
    print("Compte absent")
else
    error("Impossible d'interroger NSS : " .. err)
end

```

Validation des arguments

Les cas suivants lèvent une erreur Lua, récupérable avec `pcall` :

```

local invalid_calls = {
    function() return babet.user.get() end,
    function() return babet.user.get(nil) end,
    function() return babet.user.get(true) end,
    function() return babet.user.get({}) end,
    function() return babet.user.get(1.5) end,
    function() return babet.user.get(-1) end,
    function() return babet.user.get(8589934592) end,
    function() return babet.user.get("root\0autre") end,
}

for _, call in ipairs(invalid_calls) do
    local ok, err = pcall(call)
    assert(not ok)
    print(err)
end

```

`exists` applique exactement les mêmes validations :

```

local ok = pcall(function()
    return babet.user.exists(-1)
end)
assert(not ok)

```

Le rejet des NUL évite que "root\0autre" soit vu comme "root" par `getpwnam_r`.

NSS, workers et sécurité

Sources de comptes

Le résultat dépend de la configuration NSS de la machine. Selon `/etc/nsswitch.conf`, un compte peut provenir de :

- `/etc/passwd` ;
- LDAP ;
- SSSD ;
- NIS ;
- FreeIPA ;
- `systemd-userdb` ;
- un autre module NSS.

Un script exécuté sur deux machines peut donc obtenir des résultats différents sans que Babet ait changé.

Appels depuis des workers

Le module utilise `getpwnam_r` et `getpwuid_r`, les variantes réentrantes. Il peut être utilisé dans les workers :

```
local worker = assert(babet.workers.spawn([[
    local root, err = babet.user.get("root")
    if not root then
        error(err)
    end
    return root.uid
]]))

local joined, uid = worker:join()
assert(joined and uid == 0)
```

Chaque appel interroge NSS. Babet ne maintient pas de cache applicatif des utilisateurs.

Attention aux décisions de sécurité

Les champs `home` et `shell` sont des données de configuration, pas une preuve que le chemin existe ni que le programme est exécutable.

```
local u = assert(babet.user.get("mon-service"))
local home_is_dir = assert(babet.isDir(u.home))
local shell_path = babet.which(u.shell)
```

De même, la présence d'un compte ne prouve pas qu'il est autorisé à se connecter, qu'un mot de passe est valide ou qu'il possède un groupe secondaire particulier.

Contrat d'erreur

Situation	get	exists
compte trouvé	table utilisateur	true
compte absent	(nil, "user not found")	false
erreur NSS	(nil, "user: ...")	false
argument invalide	erreur Lua	erreur Lua

Exemple générique :

```
local ok, info, err = pcall(function()
    return babet.user.get("mon-service")
end)

if not ok then
    print("Mauvais appel :", info)
elseif not info then
    print("Lookup impossible :", err)
else
    print("UID :", info.uid)
end
```

Décisions et limites

- NSS est utilisé au lieu de parser `/etc/passwd`.
- Les variantes réentrantes `_r` sont utilisées pour rester compatibles avec les workers.
- Les champs texte sont toujours présents et remplacés par `"` si NSS fournit un pointeur nul.

- Le buffer NSS est agrandi dynamiquement jusqu'à une limite interne de 64 Kio ; au-delà, get renvoie une erreur NSS.
- exists privilégie une interface booléenne simple et masque les erreurs NSS ; utilise get lorsqu'un diagnostic est nécessaire.
- Le module n'expose pas les groupes secondaires.
- Il n'expose ni /etc/shadow, ni mots de passe, ni expiration de compte.
- Il ne crée, ne supprime et ne modifie aucun utilisateur.
- Pour créer un compte, un script peut appeler un outil système via [babet.exec](#), avec les privilèges appropriés.

WORKERS — threads OS, états Lua isolés et files de messages

babet.workers exécute du code Lua dans de vrais threads POSIX. Chaque worker possède son propre lua_State, reçoit une copie sérialisée de ses arguments et communique avec le parent par deux files de messages bornées.

Le module couvre :

- le lancement d'un chunk Lua dans un thread OS ;
- le passage d'une table d'arguments initiale ;
- la récupération bloquante ou non bloquante du premier résultat ;
- une inbox parent vers worker et une outbox worker vers parent ;
- des timeouts et une contre-pression par capacité de queue ;
- la fermeture coopérative d'un canal ;
- le chargement des modules Babet, bundlés et utilisateur dans chaque état ;
- l'isolation de la mémoire Lua entre les threads.

Il ne fournit pas de mémoire Lua partagée, de terminaison forcée, de timeout sur join(), de pool global ni de canal direct worker vers worker.

Table des matières du module

- [Conventions essentielles](#)
- [Vue d'ensemble de l'API](#)
- [spawn\(code, args?, opts?\)](#)
 - [code](#)
 - [args et worker.args](#)
 - [Capacités des queues](#)
 - [Moment où setenv et chdir sont verrouillés](#)
- [Valeurs transférables](#)
 - [Scalaires et chaînes](#)
 - [Tables listes et objets](#)
 - [Valeurs refusées](#)
 - [Copies, identité et profondeur](#)
- [Résultat final du worker](#)
 - [join\(\) — attendre et consommer](#)
 - [poll\(\) — tester et consommer](#)
 - [Ne pas combiner poll\(\) terminé puis join\(\)](#)
- [Messages parent vers worker](#)
- [Messages worker vers parent](#)
- [API worker.send et worker.recv](#)
- [Timeouts et raisons d'échec](#)
- [close\(\) et cycle de vie des queues](#)
- [Chargement des modules et environnement du worker](#)
- [Exemples complets](#)
 - [Calcul simple avec join](#)
 - [Arguments structurés](#)

- Suivi non bloquant avec `poll`
- Worker persistant requête/réponse
- Envoyer et recevoir `nil`
- Fermeture puis drainage de l'outbox
- Contre-pression et `timeout`
- Annulation coopérative
- Pool borné corrigé
- Interblocages à éviter
- Contrat d'erreur
- Garbage collector et destruction
- Signaux, environnement et concurrence
- Fonctionnalités absentes

Conventions essentielles

Chaque worker possède un état Lua distinct

Une table, une globale ou un module modifié dans le parent n'est pas partagé avec le worker. Les données traversent uniquement par sérialisation :

- `args` lors de `spawn` ;
- `job:send` et `worker.recv` ;
- `worker.send` et `job:recv` ;
- la première valeur retournée par le chunk.

Il n'existe donc pas de data race sur les objets Lua eux-mêmes. Les effets externes — fichiers, sockets, bases de données, processus — restent en revanche partagés au niveau du système et doivent être coordonnés par l'application.

Les résultats utilisent une convention proche de `pcall`

`join()` renvoie :

```
true, valeur_metier
false, message_erreur
```

Cette convention permet au worker de retourner légitimement `nil` :

```
local ok, value = job:join()
if ok then
    -- value peut être nil.
else
    io.stderr:write(value, "\n")
end
```

Les méthodes de queue renvoient également un booléen en première position, ce qui permet de transporter `nil` sans ambiguïté.

Le résultat final est consommable une seule fois

Dès que `join()` récupère le résultat, ou que `poll()` renvoie "done" ou "error", ce résultat est consommé. Tout nouvel appel à `join()` ou `poll()` renvoie un diagnostic `result already consumed`.

Vue d'ensemble de l'API

Côté parent

```
local job, err = babet.workers.spawn(code, args?, opts?)

local ok, value = job:join()
```

```

local state, value = job:poll()

local ok, err = job:send(value, timeout?)
local ok, value_or_err = job:recv(timeout?)
local ok, err = job:close()

```

API	Résultat principal	Bloquante
spawn	job ou (nil, err)	création seulement
job:join()	(true, result) ou (false, err)	oui, sans timeout
job:poll()	("running", nil), ("done", result) ou ("error", err)	non
job:send(v, t?)	(true, nil), (false, reason) ou (nil, err)	selon t
job:recv(t?)	(true, value) ou (false, reason)	selon t
job:close()	(true, nil)	non

Côté worker

Le chunk reçoit une table globale worker :

```

worker.args
worker.send(value, timeout?)
worker.recv(timeout?)

```

API	Résultat
worker.args	copie de args, ou nil
worker.send(v, t?)	(true, nil) ou (false, reason_or_error)
worker.recv(t?)	(true, value) ou (false, reason)

spawn(code, args?, opts?)

```

local job, err = babet.workers.spawn(code, args?, opts?)

```

En succès, spawn renvoie uniquement le userdata job. Le thread peut être encore en cours, déjà terminé ou déjà en erreur au moment où le parent reçoit ce userdata.

Les erreurs de chargement ou d'exécution du chunk ont lieu **dans** le worker : spawn peut donc réussir, puis join() ou poll() rapporter l'erreur.

```

local job = assert(babet.workers.spawn("this is not valid Lua"))
local ok, err = job:join()
assert(not ok)

```

code

code est obligatoire et doit être une véritable chaîne Lua. Les nombres ne sont pas convertis implicitement.

```

local job = assert(babet.workers.spawn([[
    return 6 * 7
]]))

```

Le code est chargé comme un chunk texte dans un nouvel état Lua. Il ne s'agit pas d'une fonction capturant les upvalues du parent.

```
local factor = 3
```

```
-- factor n'existe pas dans le worker : il faut le transmettre dans args.
```

```
local job = assert(babet.workers.spawn([[
    return worker.args.value * worker.args.factor
]], {
    value = 14,
    factor = factor,
})))
```

Les octets NUL présents dans code ne sont pas tronqués par le binding, mais un NUL rend normalement le chunk invalide pour le parseur Lua. Utilise une chaîne source Lua ordinaire.

args et worker.args

args est facultatif. Il doit être une table ou nil.

```
local job = assert(babet.workers.spawn([[
    return worker.args.left + worker.args.right
]], {
    left = 20,
    right = 22,
})))
```

Dans le worker :

- worker.args contient une copie désérialisée de la table ;
- sans args, worker.args == nil ;
- modifier worker.args ne modifie jamais la table du parent ;
- les métatables et l'identité des sous-tables ne traversent pas.

Un scalaire direct n'est pas accepté comme deuxième argument :

```
babet.workers.spawn("return worker.args", 42) -- erreur Lua levée
```

Emballer-le dans une table :

```
babet.workers.spawn("return worker.args.value", { value = 42 })
```

Capacités des queues

opts peut définir :

```
{
    inbox_capacity = 64,
    outbox_capacity = 64,
}
```

- inbox_capacity : nombre maximal de messages parent vers worker ;
- outbox_capacity : nombre maximal de messages worker vers parent ;
- défaut : 64 messages pour chaque queue ;
- plage autorisée : entier de 1 à 1 000 000 ;
- la valeur doit être un véritable entier Lua ; la chaîne "64" et le flottant 64.0 sont refusés ;
- la taille porte sur le **nombre de messages**, pas sur leur taille en octets.

```
local job = assert(babet.workers.spawn(code, nil, {
    inbox_capacity = 8,
    outbox_capacity = 32,
})))
```

Une capacité plus grande consomme potentiellement davantage de mémoire et ne remplace pas une stratégie de drainage. Une petite capacité applique une contre-pression plus tôt.

Les champs inconnus de `opts` sont actuellement ignorés.

Moment où `setenv` et `chdir` sont verrouillés

Après validation des types et capacités, le premier appel à `spawn` marque le processus comme ayant utilisé les workers. À partir de cet instant :

- `babet.setenv(...)` est définitivement refusé ;
- `babet.chdir(...)` est définitivement refusé ;
- la restriction `reste` active après `join()` ;
- elle reste active même si la sérialisation de `args`, l'initialisation des queues ou `pthread_create` échoue ensuite.

Un appel rejeté **avant** ce marquage — par exemple `spawn(42)` ou une capacité non entière — ne verrouille pas l'environnement.

Configure donc le répertoire courant et l'environnement avant le premier worker.

Valeurs transférables

Le format de transport interne est JSON. Les mêmes règles s'appliquent à :

- `args` ;
- les messages dans les deux sens ;
- la première valeur de retour du worker.

Scalaires et chaînes

Valeur Lua	Transférable	Remarque
<code>nil</code>	oui	devient JSON <code>null</code> , puis redevient <code>nil</code>
<code>booléen</code>	oui	identité préservée
<code>entier</code>	oui	redevient normalement un entier Lua
<code>nombre flottant fini</code>	oui	<code>NaN</code> , <code>+Inf</code> et <code>-Inf</code> refusés
<code>string texte</code>	oui	doit passer le validateur UTF-8 et ne contenir aucun NUL

Contrairement à plusieurs APIs de fichiers ou de sockets, les messages workers ne sont **pas binary-safe**. Une chaîne contenant `\0` ou des octets non acceptés comme UTF-8 est refusée.

Tables listes et objets

Une table est transférable sous l'une de ces deux formes.

Liste dense non vide

```
{ "a", "b", "c" }
```

Toutes les clés doivent être les entiers consécutifs `1..n`, sans trou ni autre clé.

Objet à clés string

```
{  
  name = "alice",  
  age = 30,  
}
```

Toutes les clés doivent être des strings acceptées par le validateur de texte.

Une table vide est sérialisée comme un **objet vide** {}, pas comme un array vide. Il n'existe pas de marqueur équivalent à `babet.json.empty_array` dans le transport workers.

Les tables mixtes ou creuses sont refusées :

```
{ [1] = "a", [3] = "c" }      -- trou
{ [1] = "a", label = "x" }   -- liste + map
{ [0] = "zero" }             -- clé numérique hors 1..n
```

Valeurs refusées

Ne traversent pas les états Lua :

- fonctions et closures ;
- userdata : socket, statement SQLite, watcher, job, fichier, etc. ;
- coroutines/threads Lua ;
- tables cycliques ;
- tables mixtes ou creuses ;
- clés non string dans une table objet ;
- nombres NaN ou infinis ;
- chaînes avec NUL ou octets non acceptés comme UTF-8 ;
- structures imbriquées au-delà de la limite.

Un chemin, une URL ou une configuration sérialisable doit être transmis à la place de l'objet système lui-même. Le worker ouvre ensuite sa propre ressource.

```
-- Incorrect : socket est un userdata.
babet.workers.spawn(code, { socket = peer })

-- Correct : transmettre les paramètres de connexion.
babet.workers.spawn(code, {
    host = "127.0.0.1",
    port = 9000,
})
```

Copies, identité et profondeur

Le transport crée une copie par valeur :

- aucune référence Lua n'est partagée ;
- les métatables sont perdues ;
- deux références vers la même sous-table deviennent deux tables séparées ;
- les cycles sont refusés ;
- la profondeur maximale est de 32 niveaux selon le compteur interne de sérialisation.

```
local shared = { value = 1 }
local args = { a = shared, b = shared }
```

```
-- Dans le worker, worker.args.a et worker.args.b ont le même contenu,
-- mais ne sont pas la même table.
```

La limite protège les piles C++ et Lua contre des structures pathologiques.

Résultat final du worker

Quand le chunk termine normalement, seule sa **première** valeur de retour est sérialisée.

```
return "first", "second", "third"
```

Le parent reçoit uniquement "first".

Pour transporter plusieurs résultats, utilise une table :

```
return {  
    value = 42,  
    elapsed = 0.12,  
}
```

Sans return, ou avec return nil, le résultat métier est nil.

Une erreur Lua non attrapée devient l'état error. Le texte transmis est un diagnostic, mais Babet n'ajoute pas automatiquement de traceback. Le worker peut utiliser `xpcall(..., debug.traceback)` s'il souhaite en construire un.

join() — attendre et consommer

```
local ok, value = job:join()
```

join() :

- attend sans timeout la fin de la pthread ;
- ne ferme aucune queue avant d'attendre ;
- consomme définitivement le résultat ;
- renvoie (true, value) en terminaison normale ;
- renvoie (false, err) en erreur Lua, erreur interne ou résultat invalide.

```
local ok, value = job:join()  
if ok then  
    print("résultat", value)  
else  
    io.stderr:write("worker: ", value, "\n")  
end
```

Même lorsqu'il renvoie nil, un succès reste reconnaissable : `ok == true`.

poll() — tester et consommer

```
local state, value = job:poll()
```

state	value	Consommé
"running"	nil	non
"done"	résultat, éventuellement nil	oui
"error"	message	oui

```
while true do  
    local state, value = job:poll()  
    if state == "running" then  
        babet.sleep(10, "ms")  
    elseif state == "done" then  
        print("terminé", value)  
        break  
    else  
        io.stderr:write("worker: ", value, "\n")  
        break  
    end  
end
```

Lorsque l'état n'est plus running, `poll()` rejoint rapidement la pthread déjà terminée et consomme son résultat.

Ne pas combiner poll() terminé puis join()

Cette séquence est incorrecte :

```
local state = job:poll()
if state ~= "running" then
    local ok, result = job:join() -- résultat déjà consommé
end
```

Utilise directement les deux valeurs de poll() :

```
local state, value = job:poll()
if state == "done" then
    use_result(value)
elseif state == "error" then
    report_error(value)
end
```

Ou utilise poll() uniquement tant qu'il renvoie running, puis choisis une architecture dans laquelle un autre mécanisme indique qu'il faut appeler join() sans avoir consommé le résultat. Dans la plupart des boucles, utiliser poll() jusqu'au résultat final est plus simple.

Messages parent vers worker

```
local ok, err = job:send(value, timeout?)
```

job:send sérialise value, puis l'ajoute à l'inbox du worker en FIFO.

Résultats :

- (true, nil) : message accepté ;
- (false, "full") : queue pleine avec timeout 0 ;
- (false, "timeout") : queue restée pleine jusqu'à l'échéance ;
- (false, "closed") : inbox fermée ;
- (nil, err) : valeur non sérialisable ou erreur interne de sérialisation.

```
local ok, err = job:send({ command = "scan", path = "/tmp" }, 1)
if not ok then
    io.stderr:write("send: ", err, "\n")
end
```

Le succès signifie seulement que le message est en queue, pas que le worker l'a déjà traité.

Messages worker vers parent

```
local ok, value_or_err = job:recv(timeout?)
```

job:recv retire le plus ancien message de l'outbox.

Résultats :

- (true, value) : message reçu ; value peut être nil ;
- (false, "empty") : queue vide avec timeout 0 ;
- (false, "timeout") : aucun message avant l'échéance ;
- (false, "closed") : queue fermée et entièrement drainée.

Lorsqu'une queue est fermée mais contient encore des messages, ceux-ci sont rendus avant "closed".

```
while true do
    local ok, message = job:recv(0)
    if ok then
        process(message)
    end
end
```

```

elseif message == "empty" then
    break
elseif message == "closed" then
    break
else
    error(message)
end
end
end

```

API worker.send et worker.recv

Dans le chunk :

```

local ok, message_or_err = worker.recv(timeout?)
local ok, err = worker.send(value, timeout?)

```

worker.recv lit l'inbox alimentée par job:send. worker.send écrit dans l'outbox lue par job:recv.

Les règles FIFO, capacités, timeouts et fermeture sont symétriques.

Différence de convention sur une erreur de sérialisation :

- job:send renvoie (nil, err) ;
- worker.send renvoie (false, err).

Cette différence conserve la convention booléenne du code worker.

```

local job = assert(babet.workers.spawn([
    while true do
        local ok, message = worker.recv()
        if not ok then
            return "inbox closed"
        end

        local sent, err = worker.send({ echo = message })
        if not sent then
            return { send_error = err }
        end
    end
end
]))

```

Timeouts et raisons d'échec

Les quatre méthodes de messages partagent le même contrat :

timeout Lua	Comportement
absent ou nil	attente indéfinie
0	tentative immédiate, non bloquante
0 < t <= 86400	attente d'au plus t secondes
négatif, NaN, Inf ou > 86400	erreur Lua levée
autre type, y compris "1"	erreur Lua levée

La résolution interne est la milliseconde. Une durée strictement positive est arrondie vers le haut : 0.0005 seconde attend au moins 1 ms et n'est pas transformé en mode non bloquant.

Les raisons de queue sont :

Raison	Signification
"full"	envoi immédiat, queue pleine
"empty"	réception immédiate, queue vide
"timeout"	échéance positive atteinte
"closed"	envoi sur queue fermée, ou réception sur queue fermée et vide

Des diagnostics internes comme "out of memory" ou "internal mutex error" peuvent exceptionnellement apparaître à la place.

close() et cycle de vie des queues

```
local ok, err = job:close()
```

job:close() :

- ferme uniquement l'**inbox** parent vers worker ;
- est idempotent ;
- renvoie (true, nil) ;
- débloque un worker.recv() en attente, qui obtient (false, "closed") une fois les messages déjà présents drainés ;
- rend les futurs job:send() impossibles avec (false, "closed") ;
- laisse l'outbox ouverte afin que le parent puisse encore recevoir les derniers messages du worker.

Il ne termine pas de force le thread et ne consomme pas son résultat final.

Le worker ferme automatiquement **les deux** queues lorsqu'il termine, en succès ou en erreur. Le parent peut néanmoins drainer les messages déjà placés dans l'outbox avant de recevoir "closed".

Pour demander proprement à un worker qui attend des commandes de finir :

```
assert(job:close())
local ok, result = job:join()
```

Le worker doit traiter worker.recv() == false, "closed" comme une condition de sortie.

Chargement des modules et environnement du worker

Chaque thread crée un nouvel état Lua et :

- ouvre les bibliothèques standard Lua ;
- enregistre le namespace complet babet ;
- expose les modules bundlés via require ;
- configure le chargement des modules utilisateur de la même manière que le projet parent, en mode dossier ou embarqué ;
- expose worker.args, worker.send et worker.recv ;
- ne copie pas la table globale arg du parent : arg == nil dans le worker.

```
local job = assert(babet.workers.spawn([
  local inspect = require("inspect")
  local mymod = require("mymod")
  return inspect({ answer = mymod.answer() })
]))
```

Les états Lua sont isolés, mais les threads appartiennent au même processus :

- même PID ;
- même répertoire courant process-wide ;
- même environnement process-wide ;

- même système de fichiers et mêmes ressources externes accessibles ;
- compte mémoire Lua mesuré séparément dans chaque état.

Babet interdit les mutations de `cwd` et d'environnement après le premier `spawn`, ce qui stabilise ces deux états globaux.

Exemples complets

Calcul simple avec `join`

```
local job, err = babet.workers.spawn([[
    return 21 * 2
]])
assert(job, err)

local ok, result = job:join()
assert(ok, result)
print(result) -- 42
```

Arguments structurés

```
local job = assert(babet.workers.spawn([[
    local total = 0
    for _, value in ipairs(worker.args.values) do
        total = total + value
    end
    return {
        name = worker.args.name,
        total = total,
    }
]], {
    name = "batch A",
    values = { 10, 20, 12 },
}))

local ok, result = job:join()
assert(ok, result)
print(result.name, result.total)
```

Suivi non bloquant avec `poll`

```
local job = assert(babet.workers.spawn([[
    babet.sleep(200, "ms")
    return "ready"
]]))

while true do
    local state, value = job:poll()

    if state == "running" then
        update_ui()
        babet.sleep(10, "ms")
    elseif state == "done" then
        print(value)
        break
    else
        error(value)
    end
end
```

```
end
end
```

Il ne faut pas appeler `join()` après la branche `done` ou `error` : le résultat a déjà été consommé par `poll()`.

Worker persistant requête/réponse

```
local job = assert(babet.workers.spawn([[
  while true do
    local ok, request = worker.recv()
    if not ok then
      return "parent closed inbox"
    end

    if request.op == "stop" then
      return "stopped"
    elseif request.op == "square" then
      local sent, err = worker.send({
        id = request.id,
        value = request.value * request.value,
      })
      if not sent then
        return { error = err }
      end
    end
  end
end], nil, {
  inbox_capacity = 16,
  outbox_capacity = 16,
}))

for i = 1, 5 do
  assert(job:send({ id = i, op = "square", value = i }))
end

for _ = 1, 5 do
  local ok, response = job:recv(2)
  assert(ok, response)
  print(response.id, response.value)
end

assert(job:send({ op = "stop" }))
local ok, result = job:join()
assert(ok, result)
print(result)
```

Envoyer et recevoir nil

Le booléen de statut distingue un vrai message nil d'une queue vide.

```
local job = assert(babet.workers.spawn([[
  local ok, value = worker.recv()
  assert(ok)
  assert(value == nil)

  assert(worker.send(nil))
```

```

        return "done"
    ]]))

assert(job:send(nil))

local got, value = job:recv()
assert(got == true)
assert(value == nil)

local ok, result = job:join()
assert(ok and result == "done")

```

Fermeture puis drainage de l'outbox

```

local job = assert(babet.workers.spawn([
    assert(worker:send("ready"))

    local ok, reason = worker:recv()
    assert(not ok and reason == "closed")

    assert(worker:send("closing"))
    return "finished"
]))

local ok, first = job:recv(1)
assert(ok and first == "ready")

assert(job:close()) -- ferme seulement l'inbox

local got, last = job:recv(1)
assert(got and last == "closing")

local joined, result = job:join()
assert(joined and result == "finished")

```

Contre-pression et timeout

```

local job = assert(babet.workers.spawn([
    babet.sleep(500, "ms")
    while true do
        local ok, message = worker:recv()
        if not ok then return "closed" end
        if message == "stop" then return "done" end
    end
]), nil, {
    inbox_capacity = 2,
})

assert(job:send("one", 0))
assert(job:send("two", 0))

local ok, reason = job:send("three", 0)
assert(ok == false and reason == "full")

local waited, why = job:send("three", 0.1)
assert(waited == false and why == "timeout")

```

```
job:close()
job:join()
```

Annulation coopérative

Il n'existe pas de `job:cancel()`. Le parent envoie une commande, et le worker la vérifie périodiquement.

```
local job = assert(babet.workers.spawn([
  local total = 0

  for i = 1, worker.args.limit do
    total = total + expensive_step(i)

    if i % 1000 == 0 then
      local ok, message = worker.recv(0)
      if ok and message == "stop" then
        return {
          cancelled = true,
          partial = total,
        }
      elseif not ok and message == "closed" then
        return {
          cancelled = true,
          partial = total,
        }
      end
    end
  end

  return { cancelled = false, total = total }
]), { limit = 1000000 }))

-- Plus tard :
local sent, err = job:send("stop", 0.5)
if not sent then
  io.stderr:write("cancel: ", err, "\n")
end

local ok, result = job:join()
assert(ok, result)
```

L'annulation reste coopérative : un worker bloqué dans un appel non interruptible ou qui ne consulte jamais sa queue ne s'arrête pas sur ce message.

Pool borné corrigé

Cette boucle limite le nombre de pthreads simultanées et utilise directement le résultat de `poll()` sans appeler ensuite `join()`.

```
local function map_parallel(items, code, max_concurrent)
  max_concurrent = max_concurrent or 4

  local next_index = 1
  local active = {}
  local results = {}
```

```

while next_index <= #items or #active > 0 do
  while next_index <= #items and #active < max_concurrent do
    local job, err = babet.workers.spawn(code, {
      item = items[next_index],
    })
    assert(job, err)

    active[#active + 1] = {
      index = next_index,
      job = job,
    }
    next_index = next_index + 1
  end

  for i = #active, 1, -1 do
    local entry = active[i]
    local state, value = entry.job:poll()

    if state == "done" then
      results[entry.index] = value
      table.remove(active, i)
    elseif state == "error" then
      results[entry.index] = { error = value }
      table.remove(active, i)
    end
  end

  if #active > 0 then
    babet.sleep(10, "ms")
  end

  return results
end

```

Chaque spawn crée tout de même un nouveau thread et un nouvel état Lua. Ce pattern borne la concurrence ; il ne réutilise pas des workers persistants.

Interblocages à éviter

Worker en attente de l'inbox, parent dans join()

```

-- Worker
local ok, message = worker.recv() -- attend indéfiniment

-- Parent
job:join() -- attend le worker

```

Les deux côtés s'attendent. Ferme d'abord l'inbox ou envoie une commande :

```

job:close()
job:join()

```

Worker bloqué sur une outbox pleine

Si le worker utilise worker.send() sans timeout, que l'outbox est pleine et que le parent appelle join() sans la drainer, aucun côté ne peut progresser.

Solutions :

- le parent lit régulièrement `job:recv()` ;
- le worker utilise un timeout fini ;
- l'outbox possède une capacité adaptée ;
- le protocole sépare clairement la phase de messages et la phase de join.

Oublier que `poll()` consomme

Un code qui attend `poll() ~= "running"`, puis appelle `join()`, ne se bloque pas mais perd le vrai résultat au profit de `already consumed`. Utilise les deux valeurs de `poll()`.

GC potentiellement bloquant

Abandonner la dernière référence à un worker encore actif peut déclencher un `__gc` qui ferme les queues puis attend le thread. Le GC débloque les attentes de queue, mais ne peut pas interrompre un calcul infini ou un `syscall` externe non borné.

Contrat d'erreur

Erreurs Lua levées par mauvais usage

Sont notamment levés :

- code absent ou non string ;
- args non table et non nil ;
- opts non table et non nil ;
- capacités non numériques, non entières, hors `1..1 000 000` ;
- timeout non numérique, négatif, non fini ou supérieur à 86 400 secondes ;
- appel d'une méthode sur un objet qui n'est pas un job.

```
local ok, err = pcall(function()
    babet.workers.spawn(42)
end)
assert(not ok)
```

Échecs renvoyés par `spawn`

`spawn` renvoie (`nil`, `err`) pour les erreurs d'exécution rencontrées avant ou pendant la création :

- args non transférables ;
- échec de sérialisation JSON ;
- initialisation d'une queue impossible ;
- échec de `pthread_create`.

```
local job, err = babet.workers.spawn("return 1", {
    fn = function() end,
})
assert(job == nil)
```

Erreurs finales du worker

`join()` renvoie (`false`, `err`) et `poll()` renvoie (`"error"`, `err`) pour :

- syntaxe Lua invalide ;
- erreur Lua non attrapée ;
- valeur de retour non transférable ;
- exception C++ interne interceptée ;
- corruption ou erreur interne lors de la désérialisation.

Le diagnostic ne contient pas automatiquement de traceback complet.

Sérialisation des messages

- `job:send : (nil, err)` si la valeur ne peut pas être sérialisée ;
- `worker.send : (false, err)` pour la même situation ;
- `recv : (false, err)` sur une anomalie interne de désérialisation.

Garbage collector et destruction

Le userdata possède un `__gc` de sécurité. Lorsqu'il est collecté :

1. l'inbox et l'outbox sont fermées ;
2. les attentes de queue sont réveillées ;
3. Babet appelle `pthread_join` si nécessaire ;
4. les primitives `pthread` et les chaînes internes sont détruites.

Cette séquence évite de libérer un état encore utilisé par un thread. Elle peut toutefois bloquer si le worker ne peut pas terminer.

Ne compte pas sur le GC comme mécanisme normal de synchronisation. Conserve le job, termine le protocole, puis appelle `join()` ou consomme le résultat avec `poll()`.

`tostring(job)` renvoie une indication comme :

```
Worker(running)
Worker(done)
Worker(error)
```

Cette chaîne reflète l'état interne instantané, pas le fait que le résultat a déjà été consommé.

Signaux, environnement et concurrence

Signaux

Les six signaux gérés par `babet.signal` sont bloqués dans chaque `pthread worker`. `babet.signal.handle`, `ignore` et `default` lèvent une erreur dans un worker. Le thread principal reste le seul propriétaire des callbacks POSIX.

Environnement et `cwd`

`setenv` et `chdir` modifient un état process-wide qui ne peut pas être muté en sécurité pendant que d'autres threads l'utilisent. Babet les interdit donc après le premier `spawn`.

Ressources externes

Deux workers peuvent ouvrir des fichiers, bases SQLite ou sockets distincts. Lorsqu'ils ciblent la même ressource externe, la sérialisation workers ne fournit aucun verrou automatique.

Pour SQLite, chaque worker doit ouvrir sa propre connexion. `wal = true` et un `busy_timeout` adapté peuvent améliorer la concurrence, mais ne remplacent pas la conception correcte des transactions.

Coût

Chaque `spawn` crée :

- une `pthread` ;
- un état Lua complet ;
- les modules Babet ;
- deux queues et leurs buffers ;
- des copies JSON des données transférées.

Évite de créer un worker pour une opération minuscule. Regroupe le travail ou utilise quelques workers persistants lorsque le protocole le permet.

Fonctionnalités absentes

Le module ne fournit pas actuellement :

- `job:join(timeout)` ;
- `job:cancel()` ou une terminaison forcée ;
- `worker.cancelled()` ;
- `job:done()` — utilise `poll()` ;
- `babet.workers.cpu_count()` ;
- un pool global réutilisable ;
- des canaux directs worker vers worker ;
- de la mémoire Lua partagée ;
- le transfert de fonctions, userdata ou coroutines ;
- un format binaire pour les messages ;
- le transfert automatique de plusieurs valeurs de retour.

Une annulation coopérative et un pool borné peuvent être construits en Lua avec les exemples de cette page. Pour le nombre de CPU, un programme externe peut être interrogé avec `babet.exec("nproc")`, en vérifiant sa table de résultat.

Cookbook

Recettes concrètes pour des tâches de scripting courantes. Chaque recette est courte et autonome ; copie, adapte, déploie.

Surveiller un dossier et envoyer les nouveaux fichiers par email

Un dossier de dépôt qui mailise chaque fichier dès qu'il arrive. Utilise `babet.inotify` pour le réveil instantané (pas de polling), et écoute `close_write` + `moved_to` pour garantir que le fichier est complet.

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue saturée : rescanne le dossier
            -- (des événements ont été perdus)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
            os.remove(path)
        end
    end
end
```

```

end
end

```

S'assurer qu'un user système existe, puis basculer dessus

Check typique à l'install avant de lancer un daemon sous un utilisateur non-root.

```

local function ensure_user(name)
    if babet.user.exists(name) then return true end
    local ok, err = babet.exec("useradd", {
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd a échoué : " .. tostring(err)
    end
    return true
end

assert(ensure_user("myapp"))
local u = assert(babet.user.get("myapp"))
babet.chdir(u.home)

```

Récupérer beaucoup d'URLs en parallèle

babet.workers fait tourner du Lua sur plusieurs cœurs. Le temps total est proche du fetch le plus lent, pas de la somme.

```

local urls = { "https://a.example/", "https://b.example/",
               "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end

```

Arrêt propre d'un script long-running

Attraper SIGTERM / SIGINT pour que le script puisse fermer ses ressources proprement. Fonctionne avec `systemctl stop` et Ctrl-C.

```

local log = require("logging")
log.set_level("info")

```

```

local running = true
babet.signal.handle("TERM", function()
    log.info("SIGTERM, arrêt en cours")
    running = false
end)
babet.signal.handle("INT", function()
    log.info("Ctrl-C, arrêt en cours")
    running = false
end)

while running do
    -- boucle principale ; les appels bloquants renvoient
    -- (nil, "interrupted") quand un signal géré arrive, donc la
    -- boucle sort rapidement.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv : ", err); break
    end
    process(line)
end

sock:close()
log.info("arrêt propre")

```

Lire une config TOML, valider, persister en SQLite

```

-- Charger le TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
local cfg, perr = babet.toml.decode(body)
if not cfg then error("config.toml : " .. perr) end

-- Valider (checks manuels simples ; pour un vrai schéma, bâtis-le en Lua)
assert(type(cfg.server) == "table", "section [server] absente")
assert(type(cfg.server.host) == "string", "server.host absent")
assert(math.type(cfg.server.port) == "integer", "server.port doit être un entier")

-- Ouvrir la db, persister
local db = assert(babet.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)])])
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
    { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
    { "port", tostring(cfg.server.port) })
db:close()

```

Cache avec TTL lisible, échéances en timestamps ISO

Parser un TTL depuis une config ("15m", "2h30m", "1d"), calculer une échéance, et la logger dans un format UTC uniforme sur lequel grep à travers les fichiers de log marche vraiment.

```

-- Config depuis TOML, CLI, env, peu importe.
local raw_ttl = "15m"

local ttl, err = babet.time.parse_duration(raw_ttl)
if not ttl then error("TTL invalide '" .. raw_ttl .. "' : " .. err) end

```

```

local deadline = babet.now() + ttl
print(string.format("[%s] entrée cache créée, expire à %s (TTL %s)",
    babet.time.iso(),
    babet.time.iso(deadline),
    babet.time.format_duration(ttl)))

-- ...plus tard, ailleurs dans le script :
if babet.now() >= deadline then
    print "[" .. babet.time.iso() .. "]" cache miss : entrée expirée"
end

```

Propriétés utiles de ce pattern :

- `babet.time.iso()` est toujours UTC, toujours 20 caractères, toujours la même forme. `grep` et `sort` marchent directement sur les timestamps.
- `parse_duration("15m")` renvoie un entier, donc `now() + ttl` est exact (pas de drift float sur les longues durées).
- `format_duration(ttl)` est aller-retour : `parse_duration( format_duration(n)) == n` pour tout `n >= 0`. Sûr à utiliser comme représentation canonique dans les logs ou les bases.

Pretty-print d'une valeur Lua

Le module `inspect` (embarqué) donne une sortie lisible pour les tables imbriquées.

```

local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }

```

Hello there

Le script Babet minimum :

```

babet.helloThere()

```